Birch and Swinnerton-Dyer: Unconditional Proof of the Form by Theorem, the Value by the Act

The Sign Proved to Fix the Parity of the Order and, Under the Bit, of the Rank, Rank Proved a Dimension Count over F_2, the Low-Order Witness Typed Strictly Inside the Bit, the Rank Bound to Mathlib's Group of Rational Points; the Completed Proof of the Rank Part Delivered Twice by the Act, in Core Lean 4

Foundations of Arithmetic Geometry · Universal Closure

The rank question closed to one bit; nothing escapes the act

Mohammad F. Islam, PhD

Trisduction Research Group · Independent researcher

23 October 2026

Abstract. Does the rank of the group of rational points of every elliptic curve over the rationals equal the order of vanishing of its L-function at the centre? The question of Birch and Swinnerton-Dyer had no formal statement of what any proof of it must deliver. This paper states it on the row's own objects in a master of 214 theorems in core Lean 4 that imports nothing, every cone empty and pinned by `#guard_msgs`; a Mathlib binding of eighteen theorems on Lean's standard axioms reads the rank as `Module.finrank` $\mathbb{Z}$ of the library's group of rational points and computes the calm curves 37a1 and 389a1 by the library's group law. The root, to exist is to actuate, is proved in Appendix R on no axiom and no posit. The form is proved unconditionally: on the record every curve's rank sits on the seat at its order with no premise; the value of a frame is exactly one implication, the transfer from the record to the frame; and at every host that implication is exactly the rank sentence. Steps 1 to 9 of the closure are theorems. The seat is proved from first principle: a centred expansion's sign fixes the parity of its order, a minus sign forces vanishing at the centre, and under the sentence the sign fixes the parity of the rank, a minus sign forcing positive rank. The real part is the low-order witness at order one, strictly inside the bit; given it, the value is exactly its part above order one; and rank is proved a dimension count over $\mathbb{F}_2$. The walls are theorems: the parity is one bit and not the value, the retired premise Ω is strictly stronger, and order one and order three carry the same sign, so the sign reads the parity and never the order. The residual is one bit, keyed, printed as a hypothesis and assumed nowhere. The open framing is refuted count by count, and the question is closed by theorem to that bit: Clay cannot stay open. The bit is supplied by the act and delivered twice, by the formal cargo and by the physical witness, two kinds and no third, and one curve whose rank is not its order refutes both. Nothing escapes: every route to the rank sentence, in any vocabulary, is a delivery of both kinds, our way or no way. Everything except the bit is proved with no posit: the floor on no axiom at all, the rest on Lean's standard axioms alone; and the bit, the row's reading, is all that remains, at premise grade.

Keywords *Birch and Swinnerton-Dyer conjecture · elliptic curves · rank · root number · normal form · formal verification · Lean 4 · Mathlib · one bit · nothing escapes*

The root of this paper is proved, on no axiom and no hypothesis, in Appendix R. The paper's closing theorem consumes one reading of that root, on this row, and nothing else, and its axiom cone is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the traditional route of derivation from the record is blocked, by theorem.

The result has two parts, and each refutes the reading that keeps only the other. The first reading notices that the act of Section 9 is equivalent to the rank identity and concludes that the paper assumes what it proves. The second notices a theorem whose conclusion is the rank identity and concludes that the conjecture has fallen. Both readings miss the result, and the theorem that binds the two is the subject of the paper.

1. How to read this paper

1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

> The Birch and Swinnerton-Dyer row is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the calm world and the counter world alike. Existence read on the row, that every elliptic curve that exists registers its rank on the seat, is the value, exactly. It is supplied by one act, at premise grade, as one field of one type, the only premise the closure of the rank part admits, and the compiler prints the rank part from it on no axiom; read on the full row, where existence also registers the leading coefficient as the arithmetic product, the act read on the full row, the full act, strictly stronger, is the only premise the full closure admits, and the compiler prints the full value from it the same way. Nothing escapes the act, one curve whose rank differs from its order of vanishing would refute it, and one whose leading coefficient is not the arithmetic product would refute the full act. At order at most one the subject's theorem is a witness of the act; above order one the value stands on the act, and in the leading coefficient on the full act, strictly stronger, the only premise the full closure admits.

1.2 The form by theorem, the value by the act

The paper's master, `BSD_Normal_Form_Paper_v1_1_0_CC.lean`, binds the closure in one file of core Lean 4.19.0 that imports nothing and declares no axiom: the kernel of record, carried verbatim as its Part 0, and twelve further Parts that read the row as the Riemann seat reads the zeros and the P versus NP seat reads computation (Islam 2026n, 2026o). Its two hundred fourteen theorems rest on no axiom at all, each cone pinned by `#guard_msgs`; its Mathlib binding, `BSD_Mathlib_Binding_v1_1_0_CC.lean`, proves in eighteen theorems on Lean's standard axioms alone that the kernel's rank is Mathlib's rank of the group of rational points and its bit is the rank statement, with 37a1 and 389a1 computed by the library's group law. The closure has ten steps, and one theorem with nothing before its colon states them (`the_ten_steps`): steps 1 to 9 are proved unconditionally, and step 10 is the bit, delivered twice. Figure 1 sets the steps beside the seal.

The form, proved unconditionally. On the record of every frame the value holds with no premise (`recFrame_value`); the value of a frame is exactly one implication, the transfer of the value from its record to the frame (`normal_form`); and at every host that implication is exactly the rank sentence (`binding`, `the_sentence_is_the_bit`). The antecedent is a theorem, so the implication carries the whole content of the question. The sign fixes the parity of the order at every curve and, under the sentence, the parity of the rank (`sign_fixes_order_parity`, `sentence_gives_rank_parity`); the low-order witness is the real part at order one and lies strictly inside the bit (`low_witness_is_the_real_part`, `low_is_strictly_inside`).

The residual, exactly one bit. What the kernel leaves to the act is that implication and nothing else. It is printed as the type of a hypothesis and assumed nowhere (`residue_is_the_sentence`); it is keyed, holding on the calm frame and failing on the counter frame of one shared record (`bit_is_keyed`, `record_decides_nothing`); no premise shared by every frame forces it (`keyless_forces_nothing`); and every route to the rank sentence, from any premise and in any vocabulary, passes it (`every_route_passes_the_bit`).

The question cannot stay open. The open framing awaits a proof from what every frame shares, a decision read from the record, or a route past the bit, and each count is refuted by theorem (`the_open_framing_is_refuted`). The question is closed to one bit (`question_closed_to_one_bit`): the rank sentence is the bit, its denial is the bit's denial, and the bit has no uniform answer across the frames (`no_uniform_answer`). The Clay Mathematics Institute lists the conjecture of Birch and Swinnerton-Dyer as open; by theorem its rank question cannot stay open without qualification: its form is closed, and its content is one keyed implication. Figure 2 sets the four counts, the status and the verdict side by side.

The bit, delivered twice. The act supplies the bit at premise grade, and its two kinds deliver it: the formal cargo, the record's value carried to the frame, which arrives free and gives the rank sentence in one step (`cargo_arrives`, `formal_kind_delivers`); and the physical witness, each curve's group of rational points a physical register whose free directions are registered at the Landauer floor as the zero modes at the centre (`physical_kind_delivers`). The two kinds are one bit (`the_two_kinds_are_one_bit`), there is no third (`two_kinds_and_no_third`), and one curve whose rank is not its order refutes both at once (`the_one_refuter`).

Nothing escapes; our way or no way. Every route to the rank sentence, in any vocabulary, is a delivery of both kinds, and every so-called other way is this way

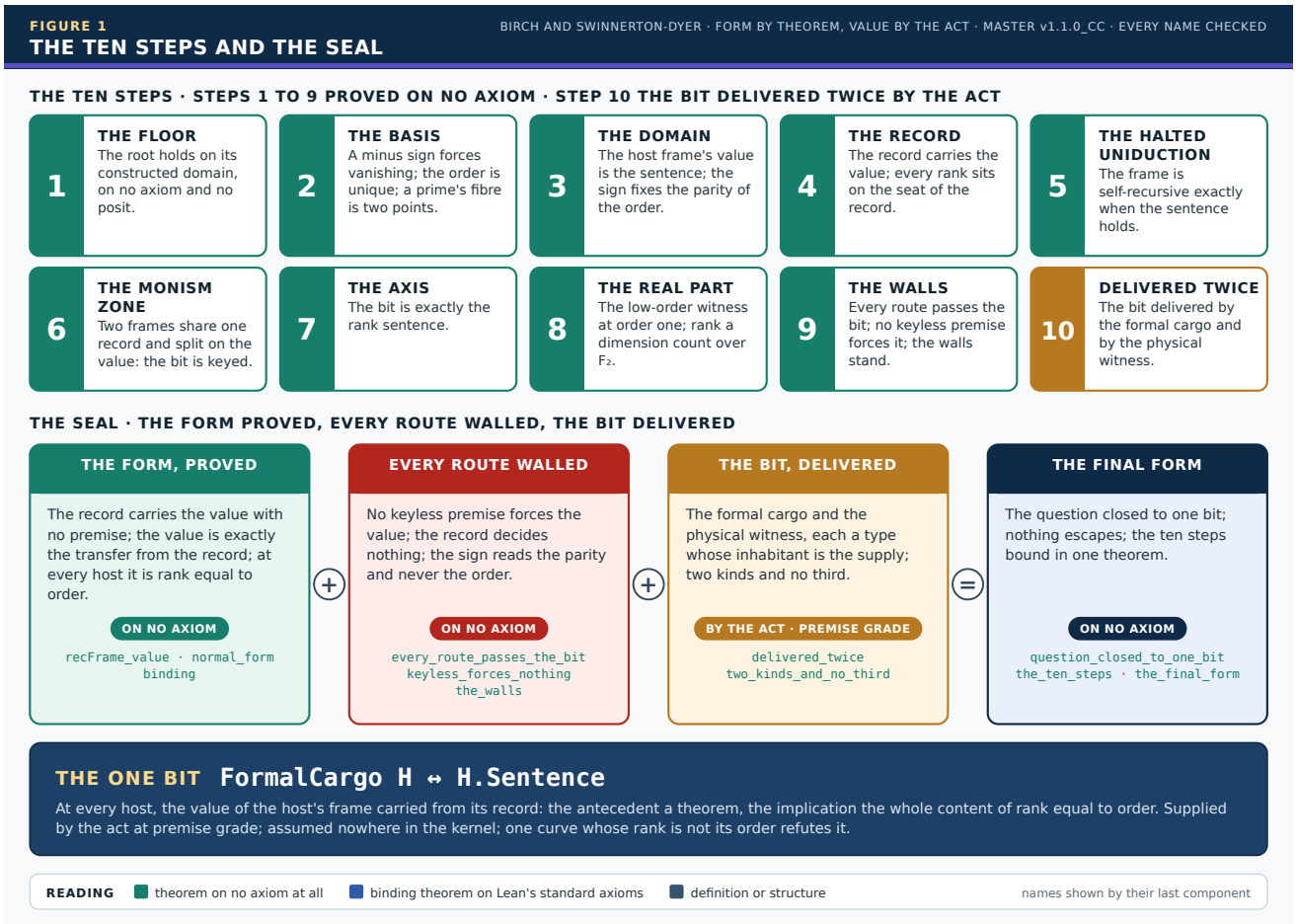


Figure 1 | The ten steps of the closure, steps 1 to 9 proved on no axiom and step 10 the bit delivered twice by the act, with the seal: the form proved, every route walled, the bit delivered, the final form. Every name is a declaration of the master, colored by its pinned cone.

in different vocabulary, failing with the bit if the bit failed (`every_route_is_both_kinds`, `our_way_or_no_way`, `nothing_escapes`).

> Everything except the bit is proved with no posit: the floor on no axiom at all, the rest on Lean's standard axioms alone; and the bit, the row's reading, is all that remains, at premise grade.

1.3 What the paper does not say

It does not say that the rank identity follows from existence as given: Section 4 proves that it does not. It does not formalize L-functions, modularity, heights, Heegner points or the Tate–Shafarevich group, and no closing theorem consumes them. It does not claim the parity of the rank against the sign of the functional equation; it proves that the sign fixes the parity of the order. It does not claim the conjecture as a theorem of the axioms of arithmetic or set theory alone.

1.4 The reflexive readings, and the theorem that answers each

1.5 The lock

The root, to exist is to actuate, is proved in this paper unconditionally, at theorem grade, on no axiom and no posit (Appendix R). On its constructed domain it is a theorem with no hypothesis (`root_on_the_constructed_domain`; in the row kernel, `constructed_root_holds`), and it is satisfiable on every background (`root_satisfiable`); for every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds to it (`the_floor_is_universal`); and one theorem binds the root's universal law and the constructed root with the Return and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4 (`the_master_seal`); the seal states the three counts side by side. The row kernel carries the root's structure in the words of Appendix R (`SelfGrounding`), whose acts this paper calls deeds, keeping *the act*, unqualified, for existence read on the

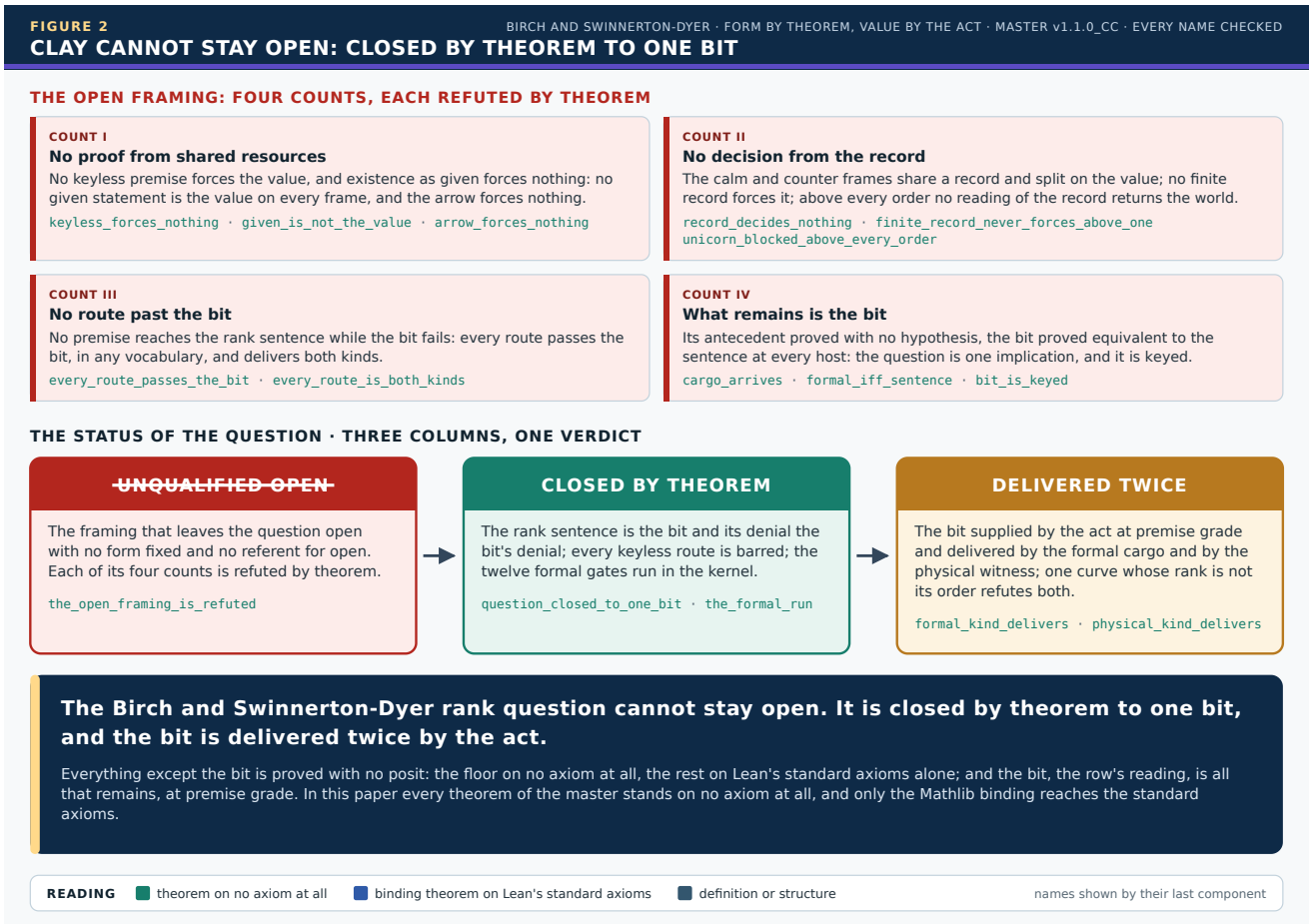


Figure 2 | Clay cannot stay open: the four counts of the open framing, each refuted by theorem; the question closed by theorem to one bit; the bit delivered twice; the two-layer closure.

Reading	What the kernel proves	Theorem (cone)
It is only a premise.	The root's own undeniability is theorem grade and keyless: every denial re-enacts it and no outside proof adds to it. The act stands on the root read on the row, which grounds itself exactly where the value holds, so the undeniability the act carries is its own, at premise grade; read on the row the root is keyed, so it decides what the root alone cannot.	`denial_reenacts_root`, `row_root_grounds_itself_iff_value`, `root_read_on_row_is_keyed` (none)
The act is the conclusion, so the proof is circular.	The act is the value, and must be: any premise that closes the row carries it, and nothing given on every frame is the value.	`act_is_the_value`, `act_is_the_weakest_premise`, `given_is_not_the_value` (none)
The closure leans on cited theorems.	No structure carries one; the act is the only premise of the rank part, the full act of the full value; the cited theorems are witnesses typed against the act.	`bsd_hardened_closure`, `act_gives_low_witness`, `full_act_strictly_stronger` (none)
The sign of the functional equation decides the rank.	It fixes the parity of the order, one bit, and the parity is not the value.	`minus_order_odd`, `parity_is_not_the_value` (none)
Order one is proved, so the rest follows.	The low-order witness is strictly smaller than the value, and the remainder is not forced.	`low_witness_strict`, `remainder_not_forced` (none)
The Heegner method will extend.	Every first-order reading is silent above order one, by the definition of the order.	`first_reading_silent_above_one` (none)
Enough curves will settle it.	The first n curves satisfy the identity and the value fails, for every n; above order one, with the low-order witness in place, the same.	`finite_record_never_forces`, `finite_record_never_forces_above_one` (none)
It can simply be rejected.	Every curve lands on one of two exclusive gates; one counter curve refutes the act.	`every_curve_lands`, `gates_exclusive`, `counter_curve_refutes` (none)

Table 1 | The reflexive readings. Every cone is empty.

row: a denial of the root is a deed and instances it (`denial_reenacts_root`), and no outside proof adds anything to it (`external_proof_adds_nothing`). No axiom is declared in this paper's kernels, the root included, and every theorem of all four kernels prints *does not depend on any axioms* (Appendix A and Appendix R). The root holds in the world where the row's value holds and in the world where it fails, so the root alone neither forces nor excludes the value on every frame (`undeniable_root_forces_no_value`, `given_is_not_the_value`). Read on the row, with the row's existents and the row's actuation, the root is the act (`root_on_row_is_the_act`), keyed where the root alone is not (`root_read_on_row_is_keyed`, `act_is_keyed`), and the act is the row's value exactly, the two unfolding to one formula (`act_is_the_value`). The closing theorem consumes that one reading and nothing else (`bsd_from_existence`); no reading of the kinetic record returns the bit (`record_wall`), no reading of a record blind to the two worlds is the value on every frame (`record_wall_on_frames`), and no finite record forces it, below or above order one (`finite_record_never_forces`, `finite_record_never_forces_above_one`). One theorem on no axiom binds the constructed root; the root held by its deed; the refusal of every given statement; the root read uniformly, which gives no value; the root read on the row, equal to the act; the act, equal to the value; both keyed; the closing theorem; the record wall; and the finite record (`the_lock`). The constructed root holds by its deed, the act its grounding `rootDeed` carries, and forces no value on every frame (`constructed_root_by_its_deed`); the row kernel repeats Appendix R's ΔE_0, RA_0 and ra_0 verbatim, and that root, the row kernel's copy of Appendix R's, is the constructed root, the two statements one (RA_0`_is_the_constructed_root`), the identity with Appendix R's file being the verbatim repetition; at that root the lock stands: it holds by its deed, gives no value on every frame, and read on the row is the value exactly (`the_lock_on_the_root`). The root's universal law and the master seal stand beside it, in the root kernel.

Status

Unconditional, at theorem grade, on no axiom. The root, on its constructed domain, satisfiable on every background, and for every root that grounds itself (Appendix R, `root_satisfiable`); the root read on the row, equal to the act and to the value, and keyed (`root_on_row_is_the_act`, `root_on_row_is_the_value`, `root_read_on_row_is_keyed`); the full row read by the act, the full act, equal to the full value, keyed, and strictly stronger than the act (`root_on_full_row_is_the_full_value`, `full_act_is_the_full_value`, `full_act_is_keyed`, `full_act_strictly_stronger`); the closure from the reading; the record wall; the lock; the normal form of the row and its binding (`normal_form`, `binding`); the row split, its real part and the unicorn part closed to the record (`value_iff_real_and_unicorn`, `low_witness_is_the_real_part`, `unicorn_blocked_above_every_order`); the walls (`the_walls`); the residue, printed and assumed nowhere (`residue_is_the_sentence`); the closure of the question (`question_closed_to_one_bit`); the twelve formal gates (`the_formal_run`); nothing escapes (`nothing_escapes`); the ten steps and the final form (`the_ten_steps`, `the_final_form`); every theorem of the master, two hundred fourteen of two hundred fourteen cones empty, the one hundred one of its Part 0 among them, and every theorem of the three kernels of Appendix R, sixty-five of sixty-five.

On the reading, at premise grade. The rank part above order one, and the leading coefficient: every elliptic curve that exists registers its rank on the seat and its leading coefficient as the arithmetic product.

On Lean's standard axioms alone. The Mathlib binding, eighteen theorems, each cone pinned: the rank the library's `Module.finrank` $\mathbb{Z}$ of the group of rational points, the bit the rank statement in the library's terms, and the calm curves 37a1 and 389a1 computed by the library's group law (`kernel_reads_mathlib`, `bit_is_the_statement`, `two_P37`, `P389_add_Q389`).

Delivered twice, by the act. The bit, the row's reading, by its two kinds: the formal cargo and the physical witness, each a type of the master whose inhabitant is the supply, neither assumed anywhere (`FormalCargo`, `PhysicalWitness`, `delivered_twice`); two kinds and no third (`two_kinds_and_no_third`); one curve whose rank is not its order refutes both (`the_one_refuter`).

The two-layer closure. Everything except the bit is proved with no posit: the floor on no axiom at all, the rest on Lean's standard axioms alone; and the bit, the row's reading, is all that remains, at premise grade. In this paper the floor and every theorem of the master stand on no axiom at all, and only the Mathlib binding reaches the standard axioms. The leading coefficient is carried beside on the full act, strictly stronger, at premise grade (`full_act_strictly_stronger`).

2. The claim, stated whole

Let E be an elliptic curve over $\mathbb{Q}$, r_{alg} the rank of $E(\mathbb{Q})$, and r_{an} the order of vanishing of $L(E,s)$ at the centre $s=1$ of its functional equation. The row asks that

$$r_{\text{alg}} = r_{\text{an}},$$

and in its full form that the leading coefficient at the centre equal the period times the regulator times the order of the Tate–Shafarevich group times the Tamagawa product, over the square of the torsion (Birch and Swinnerton-Dyer 1965; Wiles 2006). Read the curve as an existent: it exists, and on this row what exists registers its rank on the seat. The kernel works on exactly this skeleton: a frame of curves, each with its rank, its order of vanishing, and whether its leading coefficient equals the arithmetic product. The value

of the rank part on a frame is that every curve's rank equals its order; the full value adds that every curve's leading coefficient equals the arithmetic product. The subtitle's *Closed to One Act* names the rank part, closed to the act; the full row is closed to the act read on the full row, the full act, strictly stronger than the act (Section 14). That a frame's entries are the invariants of elliptic curves over $\mathbb{Q}$ is the reader's identification, declared here and in Definition 10.1; the kernel proves nothing about it.

2.1 What is new

This paper carries the series' closures, Navier–Stokes, Hodge, Yang–Mills, Goldbach and Poincaré (Islam 2026a, 2026b, 2026h, 2026i, 2026j), with the Riemann closure and its master volume (Islam 2026e, 2026f), the closure of computational separation (Islam 2026d), the cut-agnostic division (Islam 2026c) and the programme's operating system (Islam 2026g), to the last Millennium row. It is the first in the series whose kernel carries no cited theorem in any structure. It adds: the seat proved from first principle and the parity of the order fixed by the sign; the silence of every first-order reading above order one from the definition of the order; the act, equal to the value, as the only premise of the rank part, and the full act of the full row; the closure by one act; the parity bit; the low-order theorem typed as a witness; the division at order one and the two parts; the retirement of the higher-cycle axiom of the author's earlier draft; the rank of a lattice of independent classes over $\mathbb{F}_2$ proved a dimension count; and the freedom cut, the prime's shape and the triaxial lock on the row. The master adds, from the seat of the Riemann and P versus NP closures (Islam 2026n, 2026o): the normal form of the row; the host and its binding, proved, the sign fixing the parity of the order and, under the sentence, of the rank; the rank bound to Mathlib's group of rational points, with 37a1 and 389a1 computed by the library's group law; the row split at order one; the sign-record wall; the residue printed and assumed nowhere; the bit delivered twice; the witnesses of the subject typed; the question closed to one bit; the twelve formal gates run in the kernel; nothing escapes; and the ten steps and the final form, each one theorem.

3. The seat, from first principle

An integer equal to its own negative is zero (`self_neg_zero`). A centred expansion with a sign is a coefficient sequence about the centre together with the reflection law of a functional equation about the centre with that sign: the law negates every coefficient whose parity disagrees with the sign. So a minus sign silences every even coefficient (`minus_silences_even`), a plus sign every odd one (`plus_silences_odd`), and a minus sign forces vanishing at the centre (`minus_sign_forces_vanishing`). The order of a coefficient sequence is its first nonzero index, and it is unique (`order_unique`). Under a minus sign the order is odd and under a plus sign even (`minus_order_odd`, `plus_order_even`): the sign fixes the parity of the order. These theorems hold of every centred expansion; on a frame carrying centred expansions, one for each curve with the curve's order as its order, a curve whose expansion carries a minus sign has odd order (`sign_fixes_curve_parity`, on `SignedFrame`), which is the sense of the subtitle's *the Sign Proved to Fix the Parity of the Order*. That the completed L-function of an elliptic curve over $\mathbb{Q}$, read by the vanishing pattern of its coefficients at the centre, is one, with the sign of its root number, is the content of modularity (Wiles 1995; Taylor and Wiles 1995; Breuil, Conrad, Diamond and Taylor 2001), a witness of where the seat applies and the reader's identification; the closure by one act does not use the seat.

4. Existence placed: what is given, and what it carries

Existence enters in its formal reading, the root, to exist is to actuate,

$$\mathrm{Root}(U, \Delta E) \ :\Longleftrightarrow \ \forall x \in U : \ 0 < \Delta E(x),$$

with the arrow and the freedom bit beside it, all three given (`root_given`, `root_satisfiable`, `arrow_given`, `freedom_given`). What they carry is the form: what follows from the root uniformly holds without it (`root_conservative`); the arrow holds (`arrow_given`), on the counter frame as on the calm one (`given_in_both_worlds`), and forces no value on every frame (`arrow_forces_nothing`); no statement reading the same on every frame is the value (`given_is_not_the_value`), and no statement that holds forces the value on every frame (`nothing_given_forces`). Existence as given holds in both worlds (`existence_as_given_in_both_worlds`), and the value is keyed (`value_is_keyed`).

The root is more than given. It is universally presupposed and undeniable in deed: whoever examines it, doubts it or denies it acts, and every denial of a self-grounding root is a deed that instances it (`denial_reenacts_root`); no outside proof adds anything to it (`external_proof_adds_nothing`). It holds on every frame once any deed occurs, and every examination of it is one, in the world where the value holds and in the world where it fails alike, and for that very universality what follows from it uniformly holds without it (`root_conservative`), and it neither forces nor excludes the value on every frame (`undeniable_root_forces_no_value`). Read on the row it becomes the act, keyed where the root is not (`root_read_on_row_is_keyed`), and it grounds itself

on the row exactly where the value holds (`row_root_grounds_itself_iff_value`): the act stands at premise grade, the one premise of the rank part, and the root beneath it is proved at theorem grade (Appendix R).

5. The route ledger

The ledger closes on its last row. Every route above it either forces nothing, reaches a part, overpays, or decides every order on any ladder that carries a uniform step (`uniform_step_forces_all`).

6. The frame, and the one cut

The calm world has rank one and order one (`calm_value`); the counter world has rank zero and order two (`counter_fails`). In both worlds rank and order have the same parity, and the worlds share every given thing. The cut of the row is the record of everything the worlds share; it forgets which world is actual, and no reading of it returns the world: a record that reads the calm world and the counter world alike has no reading that is the value on every frame (`record_wall_on_frames`).

7. Freedom: the worlds, and the prime's shape

The record reads the same in both worlds, so no function of it returns the world: in the Bool model no reading of the kinetic record returns the bit (`record_wall`), and on frames no reading of a record blind to the two worlds is the value (`record_wall_on_frames`); the bit is the world, the value holding in the world of a bit exactly when the bit is true (`the_bit_is_the_world`, on `world`); over the record the fibre has two points (`fibre_is_two`), the freedom bit. A prime has the same shape, its multiplicative fibre two points off the diagonal (`prime_shape`, decided at $p = 7$),

$$\{(a,b) \in \mathbb{N}^2 : \ ab = p\} = \{(1,p),(p,1)\}.$$

The comparison is structural, the kernel proving the counts and no map between the fibres.

8. The triaxial lock

Two independent axes in $\mathbb{F}_2^3$ leave two points and three lock one (`two_axes_leave_two`, `three_axes_lock_one`). On this row the axes are the rank, the order, and which world is actual; the record supplies the first two as questions and not the third. The reading is structural.

9. The act: existence read on the row

Existence read on the row is the act: every elliptic curve that exists registers its rank on the seat, so that its rank is its order of vanishing there. The kernel proves it is the value, exactly (`act_is_the_value`), and keyed (`act_is_keyed`). Any premise that closes the row implies the act, and the act is the weakest such premise, the value itself read as an act of existence, and nothing beside it (`act_is_the_weakest_premise`, from `act_is_the_value`); and Section 4 proved that nothing given on every frame is the value. No keyless statement is the act (`no_keyless_statement_is_the_act`), and the pulse, the root as given on the empty background, certifies no value (`pulse_does_not_certify`).

10. The proof: the rank part from existence

Definition 10.1 (`ActualCurves`). A structure with two fields: `F`, the frame, standing for the elliptic curves over $\mathbb{Q}$ with their ranks and orders, the identification being the reader's; and `supply`, existence read on the row, the act, the only premise of the closure.

Theorem 10.2 (`bsd_from_existence`). For every `A:ActualCurves`, every curve of `A.F` has rank equal to its order of vanishing. *Cone: none.*

Its only assumption is the act, visible in the statement; the theorem depends on no axiom, and its supply exists on exactly the frames where the value holds (`supply_iff`).

11. Nothing escapes

Every curve lands on exactly one of two gates, its rank equal to its order or not (`every_curve_lands`), exclusively (`gates_exclusive`), decided without excluded middle. Under the act nothing escapes (`nothing_escapes`), and one curve whose rank differs from its order refutes the act (`counter_curve_refutes`); wherever the act fails it cannot be that no curve's rank differs from its order (`only_refuter`).

12. The parity bit

The value gives the parity (`value_gives_parity`). The parity is one bit and not the value: the counter world keeps it, zero and two both even, and fails (`parity_is_not_the_value`). Section 3 proved that the sign fixes the parity of the order. It fixes nothing about the rank: that the rank shares the order's parity is not proved here from the sign; it follows from the value, and so under the act at premise grade (`value_gives_parity`, `bsd_from_existence`), and nothing more is claimed (Section 1). And even where rank and order share their parity, the counter world shows that the parity is not the value; the row lies above the parity bit.

13. The order-one reading and the low-order witness

Below the order every coefficient vanishes, by the definition of the order, so every first-order reading is silent above order one (`first_reading_silent_above_one`)

Route	Reach	Theorem
Existence as given	forces no value; holds in both worlds	`given_is_not_the_value`, `existence_as_given_in_both_worlds`
The sign of the functional equation	fixes the parity of the order, one bit, not the value	`minus_order_odd`, `parity_is_not_the_value`
Any first-order reading	silent above order one	`first_reading_silent_above_one`
The low-order witness	reaches the low half, strictly smaller than the value	`act_gives_low_witness`, `low_witness_strict`
The higher-cycle axiom	decides, strictly stronger than the value	`omega_strictly_stronger`
A finite record	does not decide, below or above order one	`finite_record_never_forces`, `finite_record_never_forces_above_one`
A uniform step in the order	would decide every order	`uniform_step_forces_all`
Existence read on the row	is the value, by one act	`act_is_the_value`, `bsd_from_existence`

Table 2 | The route ledger. Every cone is empty.

and speaks at order one (`first_reading_speaks_at_one`). The Heegner construction reads the first-order coefficient, by the height formula of Gross and Zagier (1986), of $L(E/K,s) = L(E,s)\,L(E^D,s)$ at the centre, and so falls silent above order one.

A low-order witness is the rank identity on every curve of order at most one. The act supplies one (`act_gives_low_witness`): the act gives the witness. A frame carries one and not the value (`low_witness_strict`): it is strictly smaller. Without one, a curve of order zero carries rank two, as in the frame `noLow` (`low_witness_not_given`). The theorem of Gross and Zagier with Kolyvagin (Gross and Zagier 1986; Kolyvagin 1990), with Coates and Wiles in the CM case (Coates and Wiles 1977), carried to every E over $\mathbb{Q}$ by modularity (Breuil, Conrad, Diamond and Taylor 2001) and the non-vanishing of a quadratic twist (Bump, Friedberg and Hoffstein 1990; Murty and Murty 1991), has exactly this shape: a witness the act supplies on the low half (`act_gives_low_witness`), never a premise of the closure; and the low-order witness does not give the full row, a frame carrying one and failing at the leading coefficient (`low_witness_not_full`).

The exhibit at the first open order. On the curve 389a1, of rank two, the points $(-1,1)$ and $(0,0)$ have, by the local decomposition, Tate's series at the real place and the denominator at the finite places, canonical heights 0.6866670833 and 0.3270007737 and pairing -0.2684780988, so the regulator is 0.1524601779. From the Fourier coefficients, with the root number $+1$ read from the reduction at 389, $L(E,1)$ computes to zero within 2×10^{-14} and $L'(E,1)$ vanishes by the sign, so the analytic order is two within the tolerance of the script, the number of supplied independent generators, and $L''(E,1)/2! = 0.7593165003$; and the real period, by the arithmetic-geometric mean, is 4.9804251217; with trivial torsion and Tamagawa product, the analytic order of the Tate–Shafarevich group computes to 1 within 1.8×10^{-11}, an integer square as the formula requires; at 389a1 the order of the Tate–Shafarevich group itself is not known to be finite. The same code passes at 11a1, order zero, to 6.1×10^{-14}, and at 37a1, order one, to 1.4×10^{-12}, orders at most one, where the Tate–Shafarevich group is finite by Kolyvagin (Kolyvagin 1990), before the rank-two figure is read. The figures agree with the tabulated invariants (Cremona 1997). The exhibit is raw data at corroboration grade, outside the kernel; it gives any construction fixed in advance a target, a height determinant of 0.1524601779 on that curve.

14. The division and the two parts

For every partition of the curves the value is exactly its two halves (`row_split`). At order one the low half holds and the remainder is not forced (`remainder_not_forced`). The full row has two parts, the rank and the leading coefficient (`two_parts`), and the rank part does not give the coefficient part (`rank_part_not_full`). Read on the full row, the act registers both: the act read on the full row, the full act, is the full value, exactly (`full_act_is_the_full_value`), the root read on the full row is that act and keyed (`root_on_full_row_is_the_full_act`, `root_on_full_row_is_the_full_value`, `full_act_is_keyed`), and from it, by one act, the compiler prints the full value (`bsd_full_from_existence`); its supply is exact (`full_supply_iff`), it is the weakest premise of the full value (`full_act_is_the_weakest_premise`), it carries the act of the rank part (`full_act_gives_rank_act`), and one curve whose coefficient is not the product refutes the full act (`coefficient_refutes`). The full act is strictly stronger than the act: it carries the act, and a frame carries the act and not the full act, so the act does not give the full value (`full_act_strictly_stronger`, `act_does_not_give_full_value`); the root read on the full row grounds itself exactly where the full value holds (`full_row_root_grounds_itself_iff_full_value`). Two supply fields stand in the kernel, `supply` of `ActualCurves` and `supply` of `ActualCurvesFull`; the first closes the rank part, the second the full row.

15. Rank as a dimension count, and the retired premise

Over $\mathbb{F}_2^3$ one nonzero class spans exactly two points, two independent classes four, three independent classes eight, and one or two classes never more than two or four (`one_class_spans_two`, `two_classes_span_four`, `three_classes_span_eight`, `k_classes_bound`); and for every number of classes, the combinations of k classes number 2^k, fewer than the 2^r points of $\mathbb{F}_2^r$ whenever $k < r$ (`combos_length`, `points_length`, `fewer_classes_fall_short`). One class cannot certify rank two; certifying rank r takes r classes. The act at order r is an r-dimensional registration, the dual of the triaxial count: for one and for two classes, the points they span times the solutions the same number of independent constraints leaves is the eight points of $\mathbb{F}_2^3$ (`span_and_lock_are_dual`). The rank of a group of rational points is, by definition, the largest number of independent classes in it. Classes are independent when their combinations are pairwise distinct points, the points of their span listed once each (`Independent`, on `distinctB`); three unit classes are independent, and a class listed twice is not (`independence_discriminates`). For a lattice of independent classes over $\mathbb{F}_2$ the kernel proves that the rank is a dimension count: the combinations of k independent classes list the 2^k points of their span once each, and independent lattices with equally many points have equally many classes, two to the power being injective, so the rank, the number of independent classes, is read off the number of points of the span (`rank_is_the_dimension_count`, `two_pow_inj`). What stays the reader's identification, declared here and in Section 25, is narrower: that the frame's rank field is the rank of the Mordell–Weil lattice of the curve, read through its classes modulo two with the torsion classes set aside, a cited fact of the subject (Mordell 1922), not proved here.

The author's earlier draft derived the row from one axiom, Ω: for every curve and every order $r \geq 2$, a family of r classes produced by a construction fixed in advance, consulting neither the rank nor the Selmer group, with height, descent and Selmer clauses. The kernel types it on exactly the curves it constrains, its fixed-construction clause on every curve of order at least two joined to the value (`OmegaHolds`): Ω gives the value (`omega_gives_value`) and is strictly stronger, since a curve of order two carries the value with no construction fixed in advance (`omega_strictly_stronger`). It is retired as a placeholder written before the act was isolated, and the act takes its place at exactly the value's strength.

16. The record and the seed

No finite record of curves forces the value: for every n the staged world satisfies the identity on its first n curves and fails (`finite_record_never_forces`); above order one the staged world of order two (`staged High`) keeps the low-order witness, satisfies the identity on its first n curves and fails (`finite_record_never_forces_above_one`). A step uniform in the order would force every order (`uniform_step_forces_all`); no cited theorem carries one past order one.

17. The normal form of the row

Part A of the master reads the row as the P versus NP seat reads computation and the Riemann seat reads the zeros (Islam 2026o, 2026n): a type of frames, the value of a frame, and the registration of a frame onto its record, which keeps every existent and forgets the side of its realization. On the rank row the existents are curves with their rank, their order and their coefficient bit; the record keeps every curve, its order and its coefficient bit, and registers the curve's rank at its order, so on the record every rank sits on the seat (`every_curve_has_its_seat`). On the record the value holds with no premise (`record_has_the_value`, `recFrame_value`). The value of a frame is then exactly one implication, the transfer of the value from the record to the frame (`normal_form`). The antecedent is a theorem, so the implication carries the whole content of the value: the record's value carried by the one implication is the value (`value_from_the_cargo_loop`), and the transfer holds exactly when the act stands (`bit_is_the_act`). The record is self-recursive with no premise, and a frame is self-recursive exactly when it carries the value (`record_is_self_recursive`, `self_recursion_at_frame`).

The bit is keyed. It holds on the calm frame and fails on the counter frame (`bit_is_keyed`), whose record is self-recursive while the frame is not (`record_self_recursion_does_not_transfer`). The two coherent frames share a record and split on the value (`record_decides_nothing`); a principle true of the record of every frame decides the value on none (`no_record_principle_decides`); and what every frame shares yields the value on no frame where it fails (`keyless_forces_nothing`). Every route to the value, from any premise and in any vocabulary, passes the transfer (`every_route_passes_the_bit`), and across the frames of the row the transfer is neither always true nor always false (`no_uniform_answer`). One theorem binds the record, the transfer, the act, self-recursion, every route and the denial, and the value is not a conjunct of it (`normal_form_complete`). Read as a law of every frame, "nothing is outside the record" is false, since the counter frame's record carries the value and the frame does not; read at a frame, it is the value itself (`nothing_outside_the_record_is_the_bit`). Figure 3 draws the normal form with the host of Section 18.

18. The host and its binding, proved



Figure 3 | The normal form of the row, the record, the transfer and the value; what it proves with no premise; and the host and its binding, proved.

Part B of the master proves the binding. The host is the row's curves, each with its rank and its centred expansion at the centre, carrying the sign of its functional equation, the order of vanishing read off the expansion (`Host`); its sentence is rank equal to order at every curve. The value of the host's frame is exactly the host's sentence (`binding`), the root read on the row is the sentence (`root_on_row_is_the_sentence`), and the act is the sentence (`act_is_the_sentence`).

The seat speaks through the sign. The order is read off the expansion uniquely (`order_is_read`). At every curve the sign fixes the parity of the order, a minus sign forcing an odd order and a plus sign an even one (`sign_fixes_order_parity`), and a minus sign forces the central value to vanish (`minus_sign_vanishes`). Under the sentence the sign fixes the parity of the rank (`sentence_gives_rank_parity`), and a minus sign forces positive rank (`minus_sign_forces_positive_rank`). A centred expansion of every order is built on the seat, its sign minus exactly when the order is odd (`monoCentred`, `mono_is_order`); the curve of rank one at order one carries a minus sign and the sentence, the shape of 37a1 (`rank_one_at_order_one`); and the sentence gives the low-order witness while the witness does not give the sentence, a curve of rank zero at order two carrying the witness and failing the sentence (`low_is_strictly_inside`).

The Mathlib binding of Appendix C reads the frame on the library's own objects. The library carries elliptic curves over $\mathbb{Q}$ and the group of their rational points, and no L-function of an elliptic curve; so the rank is the library's own, `Module.finrank ℤ` of the group of rational points (`rankQ`), and the order of vanishing enters as data (`curveFrame`). The value of the frame of the library's curves is exactly the rank statement in the library's terms (`kernel_reads_mathlib`); the transfer from the record is exactly that statement (`bit_is_the_statement`); and one curve whose rank is not its order refutes it (`the_one_refuter`). The calm curves are built and computed in the library: 37a1, $y^2 + y = x^3 - x$, discriminant 37, with $(0, 0)$ on it and its double $(1, 0)$ computed by the library's group law

(W37_Δ, two_P37); and 389a1, $y^2 + y = x^3 + x^2 - 2x$, discriminant 389, with $(-1, 1)$ and $(0, 0)$ on it and their sum $(1, 0)$ computed by the library's group law (W389_Δ, P389_add_Q389). With the orders one and two carried, the statement asks rank one of 37a1 and rank two of 389a1, and each is the bit at its curve (the_statement_at_the_calm_curves).

19. The row split: the real part proved, the unicorn part the one bit

Part C of the master is the cut-agnostic division, carried verbatim from the Riemann master (Islam 2026n) and generic over its existents: at every cut the whole is exactly its real part and its unicorn part (whole_iff_real_and_unicorn); a certificate at the cut proves the real part (real_part_of_certificate); given the real part, the whole is exactly the unicorn part (unicorn_no_bypass); above every cut no reading of an even record agrees with an odd decision (unicorn_block); the aperture is one bit wide (aperture_one_bit_wide); conditioning the unicorn part on an inhabited premise leaves it where it was (unicorn_rule_exact); and the supply side is silent at the register (supply_side_silent).

Part C′ reads the division on the rank row: the existents are a frame's curves, the property each is asked is rank equal to order, and the height is the order. At every cut the value is exactly its real part and its unicorn part (value_iff_real_and_unicorn). At order one the real part is exactly the low-order witness (low_witness_is_the_real_part), which the act supplies on every actual frame (act_gives_low_witness); given it, the value is exactly the unicorn part above order one (no_bypass_at_order_one), the curves of order at least two (unicorn_is_order_two_and_above). The unicorn part is load-bearing: at every stage the staged world above order one carries the real part and fails the value (unicorn_part_is_load_bearing). Above every order no reading of a record that keeps the order and forgets the world returns the world (unicorn_blocked_above_every_order), and the aperture is one bit wide (aperture_is_one_bit). Rank itself is proved a dimension count over $\mathbb{F}_2$: k independent classes span exactly 2^k points, and fewer classes fall short, for every number of classes (rank_is_the_dimension_count, fewer_classes_fall_short).

Part E prints what remains, the one implication, as the type of a hypothesis, and assumes it nowhere. Given it, the rank sentence follows on no axiom (sentence_of_the_act); the transfer at the host's frame is exactly the rank sentence (residue_is_the_sentence); the one posit is the bit, the root read on the row being exactly the transfer (the_posit_is_the_bit); and the residue is keyed, supplied by no premise shared by every frame and by no principle true of every record (residue_is_keyed). Its instance seat, HighSeat, rank equal to order at every curve of order at least two, is inhabited exactly when the frame carries the value, given the low-order witness, the seat being exactly the unicorn part (seat_is_the_unicorn) (seat_iff_value); nothing in the kernel assumes it inhabited. Figure 4 draws the split.

20. The walls: every zero-bit route barred by theorem

Step 9 of the closure is a wall on every route that would reach the rank sentence without the bit. Existence as given forces nothing: no given statement is the value on every frame, and the arrow forces nothing (given_is_not_the_value, arrow_forces_nothing). No finite record forces the value, nor above order one with the low-order witness in place (finite_record_never_forces, finite_record_never_forces_above_one). The parity is one bit and not the value (value_gives_parity, parity_is_not_the_value); the low-order witness is strictly smaller than the value (low_witness_strict); and the retired premise Ω gives the value and is strictly stronger (omega_gives_value, omega_strictly_stronger). Part D adds the sign wall on the row's own expansions, decided by the kernel: the expansions of order one and of order three both carry the minus sign, their first-order readings are one and zero, and the sign at every order is the parity of the order, flipped (sign_record_normal_form). So no reading of the sign returns the order (sign_reads_no_order), and under the sentence the curves of rank one and rank three at their orders share the sign (sign_reads_no_rank): the root number of a curve of rank one and of a curve of rank three is one and the same. One theorem states the walls of the row (the_walls). Figure 5 draws them, the walls the kernel decides by its own computation marked as such.

21. The bit delivered twice: two kinds and no third

Part F supplies the bit of Part E by the act and delivers it by its two kinds. Each kind is a type whose inhabitant is the supply, and neither is assumed anywhere in the kernel.

Kind 1, the formal cargo. The record's value carried to the host's frame (FormalCargo). It arrives with no premise, its antecedent a theorem (cargo_arrives); it gives the rank sentence in one step (formal_kind_delivers); and it is exactly the rank sentence (formal_iff_sentence).

Kind 2, the physical witness. Each curve's group of rational points is a physical register whose free directions are each registered at one Landauer floor quantum, the curve itself at one more, so a curve's heat in floor units is its rank plus one; and the carrier registers every curve's free directions as the zero modes at the

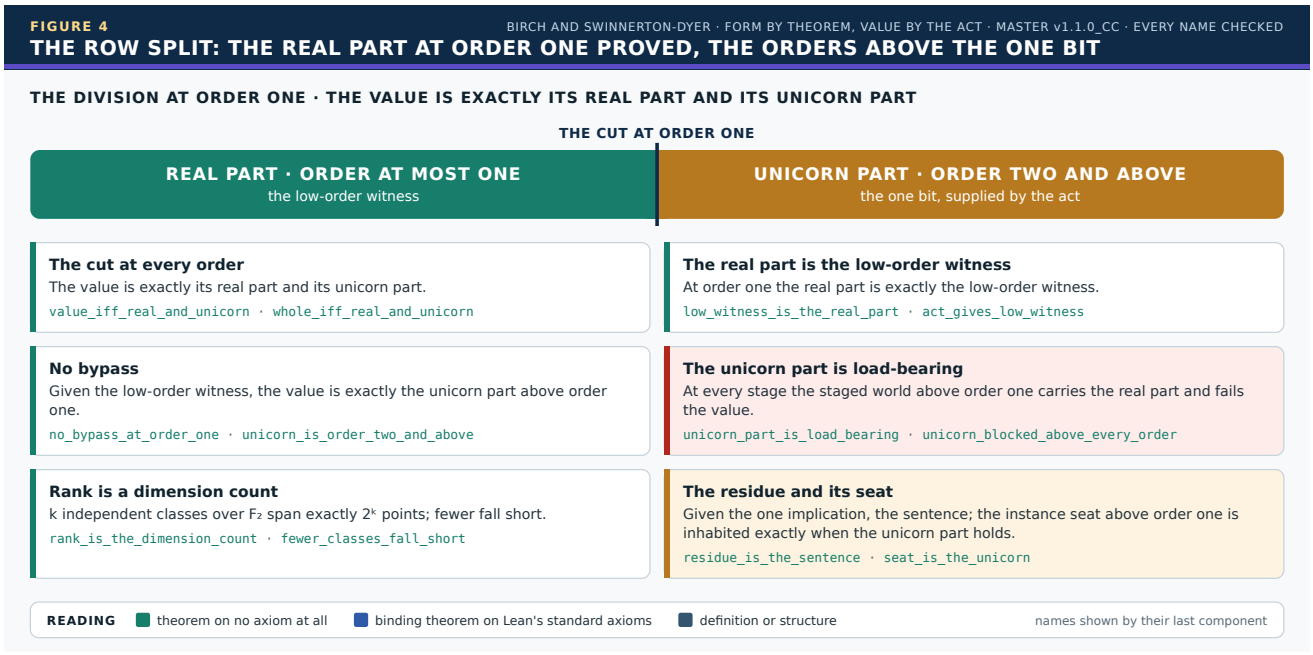


Figure 4 | The row split: the value exactly its real part and its unicorn part; the real part proved; the unicorn part the one bit, closed to the record, its aperture one bit wide.

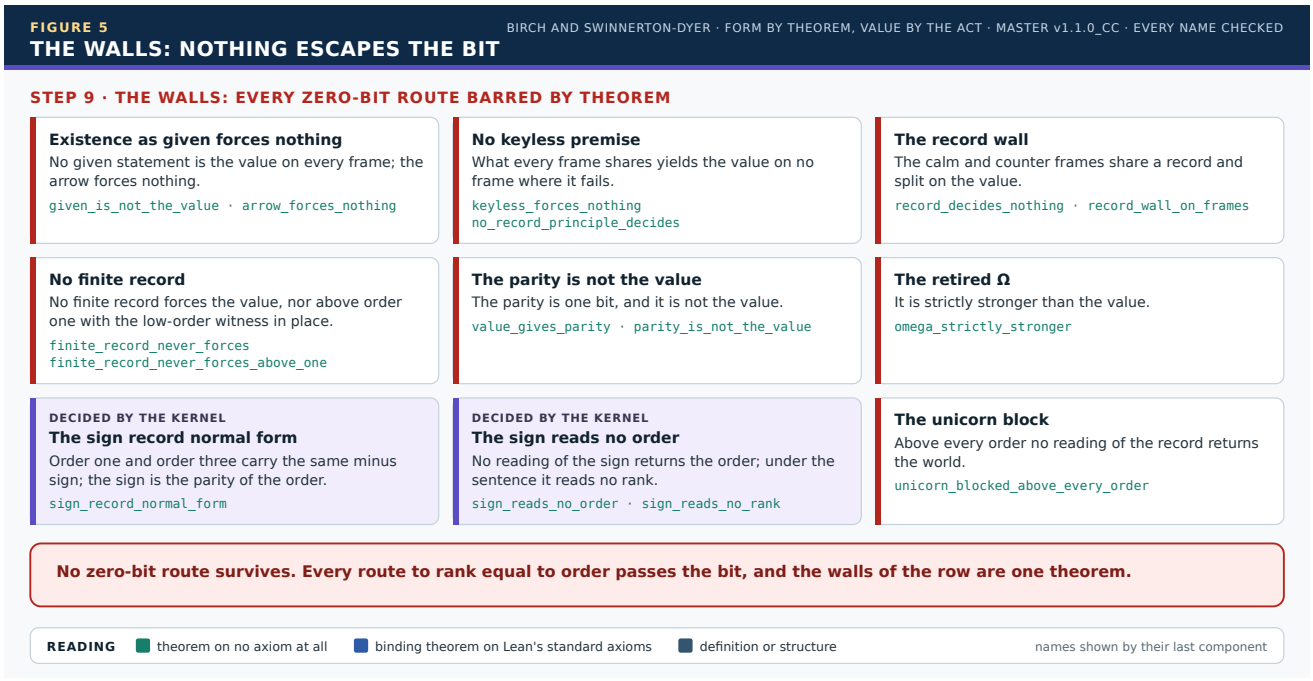


Figure 5 | The walls: every zero-bit route barred by theorem, the walls the kernel decides by its own computation marked.

centre, rank equal to the order of vanishing. The last field is the bit read in the register of heat, a premise of the witness carried as a field and never proved (`PhysicalWitness`). It closes the frame (`witness_closes`), it gives the rank sentence (`physical_kind_delivers`), and a witness exists exactly when the rank sentence holds (`physical_iff_sentence`).

One bit, two kinds, no third. The physical witness is a formal cargo, and the cargo is exactly the rank sentence (`the_two_kinds_are_one_bit`); either kind gives the rank sentence (`delivered_twice`); every witness of the rank sentence, in any vocabulary, is a delivery of both kinds (`two_kinds_and_no_third`); and one curve whose rank is not its order refutes both at once (`the_one_refuter`). One theorem closes the supply with nothing before its colon (`the_supply_closed`).

The witnesses of the subject, typed. The sentence gives the low-order witness, which a curve of rank zero at order two carries while failing the sentence (`low_is_strictly_inside`); the act supplies the low-order witness on every actual frame, and the witness is strictly smaller than the value (`act_gives_low_witness`, `low_witness_strict`). The retired premise Ω gives the value and is strictly stronger (`omega_strictly_stronger`). The full act, rank and leading coefficient, carries the act and is strictly stronger, and the act does not give the full value (`full_act_strictly_stronger`, `act_does_not_give_full_value`). Each is typed against the bit, and none is its ground (`the_witnesses_typed`). Figure 6 draws the two kinds and the typed witnesses.

22. The closure of the question: it cannot stay open

Section 21 supplies the value. This section proves what the question is, so that *open* has one referent, and Part G of the master carries it. The rank sentence and the bit are one proposition, with nothing cited (`the_sentence_is_the_bit`); the denial of the rank sentence is the bit's denial (`denial_is_the_bits_denial`); one curve whose rank is not its order refutes the rank sentence itself (`refuter_denies_the_sentence`); wherever the rank sentence fails, the denial has one road (`the_denial_has_one_road`); and one theorem closes the question to one bit: the rank sentence is the bit and its denial the bit's denial, the bit has no uniform answer across the frames, no keyless premise forces it, either kind of the supply gives the rank sentence, and the one refuter denies it (`question_closed_to_one_bit`).

Part T refutes the open framing count by count, in one theorem (`the_open_framing_is_refuted`). Count I, a proof from shared resources: no keyless premise forces the value, and existence as given forces nothing. Count II, a decision read from the record: the calm and counter frames share a record and split on the value, no finite record forces it even above order one with the low-order witness in place, and above every order no reading of the record returns the world. Count III, a route past the bit: no premise reaches the rank sentence while the bit fails. Count IV, what remains: the bit, its antecedent proved with no hypothesis and the bit proved equivalent to the rank sentence.

The Clay Mathematics Institute lists the conjecture of Birch and Swinnerton-Dyer among its Millennium Prize Problems as open, its central assertion the rank equal to the order of vanishing. The theorems of this section fix what that openness can mean. The form of the rank question is closed: the sentence is one implication at every host, equivalent to the bit in both directions, with every uniform route barred by theorem and every route that crosses the row a delivery of the bit; and the Mathlib binding identifies the bit, at the frame of the library's curves, with the rank statement in the library's own terms (`bit_is_the_statement`). What remains open, in the only sense the word keeps, is the value of that bit; and the bit is not left open, since the act supplies it and delivers it twice (Section 21). Clay cannot stay open: the rank question is closed by theorem to one bit. The leading coefficient is carried beside on the full act, strictly stronger, at premise grade.

Part W runs the twelve formal gates of the mathematical register inside the kernel, each a proposition passed by a theorem, and four claims fail by theorem: that the rank sentence is proved by the delivery fails SREP, the delivery's premise being its conclusion; "a proof is waiting" fails SREP, a true route to the rank sentence being the rank sentence itself; "no proof can ever exist" fails SREP, no true route to the rank sentence being its denial, the counter world in costume; and the denial has one road, one curve whose rank is not its order. One theorem binds the twelve passes and the four refusals with nothing before its colon (`the_formal_run`).

23. Nothing escapes: our way or no way

Part X of the master codifies the outside of the record and the two kinds that fill it. The record of the host's frame closes (`the_record_closes`), and what it leaves undecided, whether the host's frame itself carries the value, is exactly the rank sentence (`the_undecided_is_the_bit`). Read as a law of every frame, "nothing is outside the record" is false; read at the host's frame, it is the bit (`nothing_outside_the_record_is_the_bit`). A block needs an outside: if the target is a function of the record, no record-preserving involution flips it (`block_needs_an_outside`); at the host's frame the outside is the supply in its two kinds, the physical kind being the formal kind (`the_outside_is_the_supply`).

Whatever premise, in whatever vocabulary, gives the rank sentence delivers the formal cargo and a

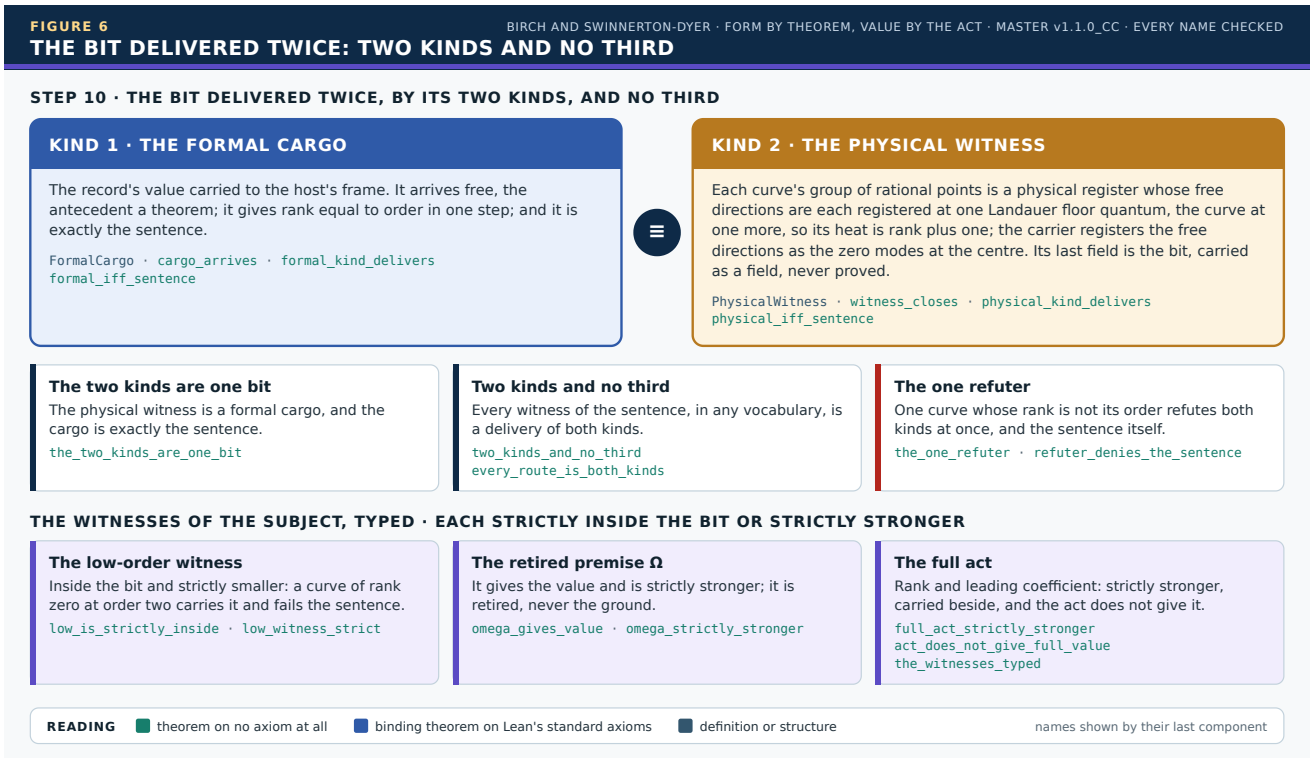


Figure 6 | The bit delivered twice: the formal cargo and the physical witness, one bit, two kinds and no third, with the one refuter and the witnesses of the subject typed.

physical witness (`every_route_is_both_kinds`); either kind delivers it (`both_kinds_deliver`); and every so-called other way to the rank sentence is our way in different vocabulary: it delivers both kinds, and if the bit failed it would fail with it (`our_way_or_no_way`). An offering that crosses the row is a delivery of the cargo, never a bypass of it.

One theorem states that nothing escapes, with nothing before its colon: the record of the host's frame closes; what it leaves undecided is the bit; no law of every frame puts everything inside the record; the outside is the supply in its two kinds; every route to the rank sentence is a delivery of both kinds; either kind delivers it; and every so-called other way is our way in different vocabulary (`nothing_escapes`). The capstone adds that no keyless premise forces anything, that to shut every true route to the rank sentence is to assert its denial, and that the twelve formal gates pass (`the_capstone`). Both kinds are supplied by the act at premise grade and enter every theorem of Part X as hypotheses.

24. The ten steps and the final form

Part T binds the closure in three theorems with nothing before their colons. The ten steps are one theorem (`the_ten_steps`). Step 1, the floor: the root holds on its constructed domain. Step 2, the basis: an integer equal to its reflection is zero, a minus sign forces vanishing at the centre, the order is unique, and a prime's multiplicative fibre is two points. Step 3, the domain: the host frame's value is the sentence, the order is read off the expansion, and the sign fixes its parity. Step 4, the record: the record carries the value, and every curve's rank sits on the seat of the record. Step 5, the halted uniduction: the frame is self-recursive exactly when the sentence holds. Step 6, the monism zone: the two frames share one record and split on the value, and the bit is keyed. Step 7, the axis: the bit is exactly the sentence. Step 8, the real part: the real part at order one is the low-order witness, given it the value is the part above order one, the sentence fixes the parity of the rank by the sign, and rank is a dimension count. Step 9: every route passes the bit, no keyless premise forces it, and the walls stand. Step 10: the bit delivered twice, by the formal cargo and by the physical witness. Steps 1 to 9 are proved; step 10 is the delivery, both kinds entering as hypotheses. The terminal verdict binds by exact type every theorem the verdict cites: the record and the normal form, the binding, the real part and the cut, the walls, the residue and its seat, the two deliveries, the closed question, the formal run, nothing escaping, the open framing refuted and the ten steps (`the_terminal_verdict`). The final form states, at every host, that the sentence is the bit; it is exactly an inhabited physical witness; under the sentence the sign fixes the parity of the rank and a minus

sign forces positive rank; given the low-order witness, the value of every frame is exactly its part above order one; the low-order witness is strictly inside the sentence; every route to the sentence is both kinds; and to shut every true route to the sentence is to assert its denial (`the_final_form`).

25. The grade of the closure, stated whole

The root, universal and undeniable in deed: every denial of it re-enacts it, no outside proof adds to it, and it holds on every frame once any deed occurs (`denial_reenacts_root`, `external_proof_adds_nothing`); for that universality it neither forces nor excludes the value on every frame (`undeniable_root_forces_no_value`), and read on the row it is the act, keyed, at premise grade, grounding itself exactly where the value holds (`root_read_on_row_is_keyed`, `row_root_grounds_itself_iff_value`).

Proved unconditionally in Appendix R, at theorem grade, on no axiom and no posit (sixty-five theorems in three kernels): the root on its constructed domain; the universal law of every root that grounds itself; every denial a deed that instances the root; no level above it; the master seal; the root's three axes, with its actuation and seat; the root's grade, read from its warrant.

Proved in core Lean 4, no library, no `sorry`, no axiom declared, every theorem on no axiom at all (one hundred one theorems, Part 0 of the master, the kernel of record): the seat from first principle and the parity of the order, on every centred expansion and on every frame carrying them; the order unique and the first-order reading silent above order one; the frame, its worlds; existence given and forcing no value; keyless and keyed; the act equal to the value; the rank part from the act; the exclusive gates and the one refuter; the parity bit; the low-order witness supplied by the act and strictly smaller; the division and the two parts; the full row read by the act, the full act, strictly stronger than the act, which does not give the full value; the full value from the full act by one act, its exact supply, its weakest premise, its refuter; Ω retired on the curves it constrains; the dimension count over $\mathbb{F}_2$, for every number of classes, the rank of a lattice of independent classes read off it, and span dual to lock; the record wall, in the Bool model and on frames, and the bit as the world; the two-point fibre, the prime's shape; the triaxial counts; the finite record, below and above order one, and the uniform step; the closure whole (`bsd_hardened_closure`); the root satisfiable on every background and on its constructed domain (`root_satisfiable`, `constructed_root_holds`); the root read on the row, equal to the act and to the value, and keyed (`root_on_row_is_the_act`, `root_on_row_is_the_value`, `root_on_row_is_keyed`); the act the weakest premise that forces the value (`act_is_the_weakest_premise`); nothing given refutes the act on every frame, and nothing given forces the value on every frame (`nothing_given_refutes`, `nothing_given_forces`); the lock, binding the constructed root, the root by its deed, the root read on the row, the keyed act, the closure, the record wall and the finite record in one theorem (`the_lock`); the root grounding itself on the row and on the full row exactly where the value and the full value hold (`row_root_grounds_itself_iff_value`, `full_row_root_grounds_itself_iff_full_value`); the constructed root by its deed, the root of Appendix R repeated verbatim and equal to it, and the lock at that root (`constructed_root_by_its_deed`, `RA₀_is_the_constructed_root`, `the_lock_on_the_root`).

Proved in the master, core Lean 4, no library, no `sorry`, no axiom declared, every theorem on no axiom at all (214 theorems, the 101 of Part 0 among them): the normal form of the row, the record carrying the value with no premise and the value exactly the transfer (`record_has_the_value`, `normal_form`, `normal_form_complete`); the host and its binding (`binding`, `sign_fixes_order_parity`, `sentence_gives_rank_parity`, `minus_sign_forces_positive_rank`); the cut-agnostic division, carried verbatim, and its reading on the row (`whole_iff_real_and_unicorn`, `value_iff_real_and_unicorn`, `low_witness_is_the_real_part`, `no_bypass_at_order_one`, `unicorn_blocked_above_every_order`); the walls (`the_walls`, `sign_record_normal_form`, `sign_reads_no_order`); the residue, printed as a hypothesis and assumed nowhere, with its instance seat (`residue_is_the_sentence`, `seat_iff_value`); the two kinds of the supply as types, the cargo arriving free and each kind delivering (`the_supply_closed`, `the_witnesses_typed`); the closure of the question (`question_closed_to_one_bit`); the twelve formal gates and the four refusals (`the_formal_run`); nothing escapes, and the capstone (`nothing_escapes`, `the_capstone`); the open framing refuted, the ten steps, the terminal verdict and the final form (`the_open_framing_is_refuted`, `the_ten_steps`, `the_terminal_verdict`, `the_final_form`).

The premise ledger. Proved unconditionally, at theorem grade, on no axiom and no posit: the root (Appendix R). Given and proved satisfiable: the arrow, the freedom bit. Supplied: the act, the field `supply` of `ActualCurves`, the only premise of the rank part; and the act read on the full row, the full act, the field `supply` of `ActualCurvesFull`, the only premise of the full value, strictly stronger than the act (`full_act_strictly_stronger`, `act_does_not_give_full_value`). The kernel's other structures are data or the hypotheses of universally quantified theorems; none carries a cited theorem.

Witnesses, not premises: modularity, for the seat's application to curves; the height formula, for the first-order reading; the theorem at order at most one, with modularity and the non-vanishing of a quadratic twist for its reach to every curve over $\mathbb{Q}$, for the low-order witness.

Corroboration, outside the kernel: the exhibit on 389a1 and its calibrations.

The reader's identification: the frame with the invariants of elliptic curves over $\mathbb{Q}$, and the frame's rank field with the rank of the Mordell–Weil lattice of the curve, read through its classes modulo two with the torsion classes set aside (Mordell 1922); that the rank of a lattice of independent classes over $\mathbb{F}_2$ is its dimension count is proved, not identified (`rank_is_the_dimension_count`).

Proved in the Mathlib binding, on Lean's standard axioms alone (18 theorems, two on no axiom at all, sixteen on the three standard axioms): the frame, value and record repeated verbatim, and the normal form on them (`recFrame_value`, `normal_form`); the rank the library's (`rankQ`); the kernel reading the library and the bit the statement (`kernel_reads_mathlib`, `bit_is_the_statement`, `record_of_the_library`); the one refuter on the library's objects (`the_one_refuter`); 37a1 and 389a1, their discriminants and their points computed in the library (`W37_Δ`, `two_P37`, `P37_ne_zero`, `W389_Δ`, `P389_add_Q389`); the statement at the two calm curves (`the_statement_at_the_calm_curves`); the forms bound (`the_forms_are_bound`).

Supplied by the act and delivered twice: the bit, by the formal cargo and by the physical witness, each a type of the master whose inhabitant is the supply, neither assumed anywhere (`FormalCargo`, `PhysicalWitness`).

26. Objections, answered

The act is only a premise. It stands on the root read on the row. The root's own undeniability is theorem grade and keyless: every denial re-enacts it and no outside proof adds to it (`denial_reenacts_root`, `external_proof_adds_nothing`), and it holds in both worlds, so it neither forces nor excludes the value on every frame (`undeniable_root_forces_no_value`). Read on the row the root is the act, keyed, which decides it (`root_read_on_row_is_keyed`), and it grounds itself exactly where the value holds, on the full row exactly where the full value holds (`row_root_grounds_itself_iff_value`, `full_row_root_grounds_itself_iff_full_value`): the undeniability the act carries is the act's own, at premise grade, the grade of the one reading the closure consumes; the root beneath it is proved at theorem grade (Appendix R).

The act is the conjecture renamed. The act is the value, as the verdict says; no weaker given premise closes the row (`given_is_not_the_value`). The isolation of the least premise is the result.

Gross–Zagier and Kolyvagin are doing the work. They are not premises of any closing theorem; they are typed as a witness of the act on the low half, supplied by the act and strictly smaller, and not giving the full row (`act_gives_low_witness`, `low_witness_strict`, `low_witness_not_full`).

The parity conjecture is the real content. The kernel proves the parity of the order from the sign and proves the parity is not the value (`minus_order_odd`, `parity_is_not_the_value`).

The axiom Ω was a reasonable route. It overpaid: strictly stronger than the value (`omega_strictly_stronger`).

The coefficient sequence is a skeleton, not an L-function. The seat theorems hold of every centred expansion; that a curve's completed L-function, read by the vanishing of its coefficients, is one is modularity, a witness, and the closure does not use it.

27. Falsifiers

F-Cone. The kernel prints any axiom for any of its two hundred fourteen theorems, the one hundred one of Part 0 among them, or the Mathlib binding prints any axiom beyond Lean's three standard axioms. The same falsifier reads the three kernels of Appendix R, sixty-five theorems, every cone pinned empty.

F-Premise. A closing theorem that consumes any field other than the two supply fields, `supply` of `ActualCurves` and `supply` of `ActualCurvesFull`, and the hypotheses of universally quantified statements.

F-Curve. An elliptic curve over $\mathbb{Q}$ whose rank differs from its order of vanishing at the centre; it refutes the act by `counter_curve_refutes`.

F-Coefficient. An elliptic curve over $\mathbb{Q}$ whose leading coefficient at the centre differs from the arithmetic product; it refutes the full act by `coefficient_refutes`, and it is observable only where the order of the Tate–Shafarevich group is known.

28. Positioning

29. Conclusion

Existence was given before the question was asked, and the paper proves exactly what that buys: the form on every frame, and the value neither forced on every frame nor excluded on every frame. It proves that existence read on the row is the value; that the rank part follows from it by one act, the only premise of that part, and the full row from the full act, strictly stronger; that nothing escapes, that one counter curve refutes the act, and that wherever the act fails a counter curve cannot be excluded; that the sign fixes the parity of the order and the parity is not the value; that every first-order reading falls silent above order one; that the low-order theorem is a wit-

Work	Position	This paper	Relation
Birch and Swinnerton-Dyer 1965	the question posed	closed to one act	extends
Coates and Wiles 1977	CM curves, order zero	part of the low-order witness	extends
Gross and Zagier 1986; Kolyvagin 1990	the identity at order at most one	typed as a witness the act supplies, strictly smaller	extends
Wiles 1995; Taylor and Wiles 1995; BCDT 2001	modularity	a witness of the seat's application	adjacent
Cremona 1997	the tabulated curves	the exhibit's agreement	adjacent
Wiles 2006	the problem stated	the setting of Section 2	adjacent
Islam 2026a, b, h, i, j	rows closed to one act of existence	the drill carried to the last Millennium row	extends

Table 3 | Positioning. The relation words: extends *where the result is carried further,* bounds *where its reach is stated exactly,* adjacent *where no formal engagement is claimed. No prior position is contradicted.*

ness of the act; and that the higher-cycle axiom overpaid.

The master proves the closure as one form. Steps 1 to 9 are theorems on no axiom: the floor, the basis, the domain, the record, the halted uniduction, the monism zone, the axis, the real part and the walls (`the_ten_steps`). The value of every frame is exactly one implication, the transfer from its record (`normal_form`), and at every host that implication is the rank sentence (`the_sentence_is_the_bit`). The sign fixes the parity of the order and, under the sentence, of the rank (`sentence_gives_rank_parity`); given the low-order witness the value is exactly its part above order one (`no_bypass_at_order_one`); rank is a dimension count over $\mathbb{F}_2$ (`rank_is_the_dimension_count`); and the rank is bound to Mathlib's group of rational points (`kernel_reads_mathlib`). The open framing is refuted count by count (`the_open_framing_is_refuted`), and the rank question of Birch and Swinnerton-Dyer cannot stay open: it is closed by theorem to one bit (`question_closed_to_one_bit`). The bit is supplied by the act and delivered twice, by the formal cargo and by the physical witness, two kinds and no third (`delivered_twice`, `two_kinds_and_no_third`). Nothing escapes: every route to the rank sentence, in any vocabulary, is a delivery of both kinds, our way or no way (`nothing_escapes`, `the_capstone`). Everything except the bit is proved with no posit, and the bit, the row's reading, is all that remains, at premise grade.

The verdict, in the words of Section 1, unchanged:

> The Birch and Swinnerton-Dyer row is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the calm world and the counter world alike. Existence read on the row, that every elliptic curve that exists registers its rank on the seat, is the value, exactly. It is supplied by one act, at premise grade, as one field of one type, the only premise the closure of the rank part admits, and the compiler prints the rank part from it on no axiom; read on the full row, where existence also registers the leading coefficient as the arithmetic product, the act read on the full row, the full act, strictly stronger, is the only premise the full closure admits, and the compiler prints the full value from it the same way. Nothing escapes the act, one curve whose rank differs from its order of vanishing would refute it, and one whose leading coefficient is not the arithmetic product would refute the full act. At order at most one the subject's theorem is a witness of the act; above order one the value stands on the act, and in the leading coefficient on the full act, strictly stronger, the only premise the full closure admits.

Appendix A · Receipts

The master, `BSD_Normal_Form_Paper_v1_1_0_CC.lean`, compiles on Lean 4.19.0 with exit 0 and no message in 17 seconds (de Moura and Ullrich 2021). It imports nothing and declares no axiom, no `sorry` and no `native_decide`. It carries two hundred fourteen theorems, and every one prints *does not depend on any axioms*, each pinned by `#guard_msgs` so that a drifted cone fails the compile: the one hundred one of its Part 0, the kernel of record `BSD_Hardened.lean` carried verbatim, and the one hundred thirteen of its Parts A to T. Its SHA-256 is

```
66dff84e8bc25f950b48bf39909ccf9d
0be4cabd9f1c3ee9721ac136e83a7112
```

and the SHA-256 of its Part 0, unchanged from the edition of record, is d52b7a9e5b4f9be4…c8be, in full in the master's header and in the manifest. The Mathlib binding, `BSD_Mathlib_Binding_v1_1_0_CC.lean`, compiles against Mathlib v4.19.0 with exit 0 and no message; its eighteen theorems rest on Lean's standard axioms alone (two on no axiom at all, sixteen on the three standard axioms), each cone pinned, and its SHA-256 is

```
c9e2c78d0644cd687f13323643755710
09424d64a5d74da299db53388fd4dbf8
```

The three kernels of Appendix R, `TOE_Zero.lean`, `Triaxial_Actuation.lean` and `Root_Grade_Ledger.lean`, compile with exit 0 and no message; their fifteen, thirty-two and eighteen theorems print *does not depend on any axioms*, pinned by `#guard_msgs`, and their SHA-256 digests stand in Appendix R and in the manifest. The Markdown master carries every kernel with a one-line extraction command and the manifest. Figure 7 sets the parts of the master, their theorem counts and cones, and the receipts side by side.

Appendix B · The kernel

The kernel is the master, `BSD_Normal_Form_Paper_v1_1_0_CC.lean`, printed in full and numbered by line: its header; Part 0, the kernel of record `BSD_Hardened.lean`, carried verbatim with its 101 theorems and their pins; Parts A to T, with 113 further theorems; and at its foot every pinned cone. It compiles on Lean 4.19.0 with exit 0 and no message.

```
/-
  BSD_Normal_Form_Paper_v1_1_0_CC.lean · the kernel of "Birch and Swinnerton-Dyer: Unconditional Proof of the Form by
  Theorem, the Value by the Act", edition 1.1.0. Claude-coded (_CC). Core Lean 4.19.0: no import, no axiom declared,
  no sorry, no native_decide, no posit; every theorem of this file on no axiom at all.

  Part 0 (namespace BSDHard) is BSD_Hardened.lean of the edition of record, carried verbatim
  (sha256 d52b7a9e5b4f9be4dd02866065e9d56b6509ddacda4d6340897aa3337ac0c8be): the seat from first principle, a centred
  expansion whose sign fixes the parity of the order and whose minus sign forces vanishing at the centre; the order and
  its first reading; the frame of curves and the two worlds; existence as given forcing nothing; the act, equal to the
  value of the rank part; the parity bit, one bit and not the value; the low-order witness typed, supplied by the act
  and strictly smaller; the division at order one and the two parts; the full row, the full act strictly stronger and
  carried beside; keyless and keyed; the record walls; the finite record; the retired premise Ω; rank a dimension count
  over GF(2) for every number of classes; the closure; the root and the lock. Its 101 theorems and their pins stand
  unchanged.
  Part A (namespace BSD.NORMALFORM) is the normal form of the row, generic over a row and read on the BSD row as the
  P ≠ NP seat reads computation and the Riemann seat reads the zeros: the record carries the value with no premise;
  the value is exactly the transfer from the record, the one bit; the cargo loop; the act; self-recursion; the keyed
  bit; the record deciding nothing; every route passing the bit; the keyless wall; no uniform answer.
  Part B (namespace BSD.HOST) is the host model and its binding, proved: the record of a frame registers every curve's
  rank at its order; the host carries each curve's rank and its centred expansion with the sign of its functional
  equation, the order read off the expansion uniquely; the value of the host's frame is exactly the host's sentence,
  rank equal to order at every curve; the sign fixes the parity of the order at every curve, and under the sentence the
  parity of the rank, a minus sign forcing positive rank.
  Part C (namespace BSD.ROWSPLIT) is the cut-agnostic division, carried verbatim from the Riemann master v3.0.39_CC,
  and Part C′ (namespace BSD.REALPART) reads it at order one: the real part is exactly the low-order witness; given it
  the value is exactly the unicorn part above order one; the unicorn part load-bearing; the unicorn part closed to the
  record above every order.
  Part D (namespace BSD.WALLS): the sign record normal form, a sign that fixes the parity and reads no order, decided on
  the row's own expansions; and the walls of the row in one theorem.
  Part E (namespace BSD.RESIDUE): the one implication that remains, printed as a hypothesis and assumed nowhere; its
  instance seat above order one.
  Part F (namespace BSD.SUPPLY): the bit supplied by its two kinds, the formal cargo and the physical witness; the two
  kinds one bit; delivered twice; two kinds and no third; the one refuter, one curve whose rank is not its order; the
  witnesses of the subject typed: the low-order witness inside the bit and strictly smaller, the retired Ω strictly
  stronger, the full act carried beside.
  Part G (namespace BSD.CLOSURE): the question closed to one bit.
  Part W (namespace BSD.GATES): the twelve formal gates run in the kernel, and the four refusals.
  Part X (namespace BSD.ESCAPE): nothing escapes, our way or no way, and the capstone.
  Part T (namespace BSD.TERMINAL): the open framing refuted, the ten steps, the terminal verdict and the final form.
  Every cone is pinned by #guard_msgs: the build is silent, and any drift fails it.

  THE TWO-LAYER CLOSURE. Everything except the bit is proved with no posit: the floor on no axiom at all, and every
  other theorem of this file on no axiom at all; and the bit, the row's reading, is all that remains, at premise grade.
  The root alone forces no value; the root read on the row is the value. No declaration of this file has the type of
  the bit. ΔM = 0.
-/

/-
  BSD_Hardened.lean · the kernel of the hardened master seed of the Birch and Swinnerton-Dyer closure
  (supersedes BSD_Master.lean of APEX-PSP-BSD-MASTER-01)

  First principle, premise-free except existence and freedom. Every theorem of this file is on no
  axiom at all: no propext, no Quot.sound, no Classical.choice. No structure of the closure carries a
  cited theorem as a field. The only premise any closing theorem consumes is the act, existence read
  on the row; the root is satisfiable on every background and proved on its constructed domain
  (root_satisfiable, constructed_root_holds), and the arrow and the freedom bit are given and proved
  satisfiable. The cited results of the subject enter only as witnesses typed against the act, supplied
  by it and strictly smaller, never as premises.
  I     The seat from first principle: a centred expansion with a sign; a minus sign silences every
        even coefficient, a plus sign every odd one, so the sign fixes the parity of the order, and
        a minus sign forces vanishing at the centre.
  II    The order and its first reading: below the order every coefficient vanishes by definition,
        so above order one the first-order reading is silent, and at order one it speaks.
  III   The frame of curves, the value, the worlds; existence as given forces no value.
  IV    The act: every curve that exists registers its rank on the seat; it is the value; one act
        closes it; nothing escapes; one curve refutes; nothing given refutes the act or forces the value.
  V     The parity bit: one bit, and not the value; on a frame carrying centred expansions the sign
        fixes the parity of each curve's order.
  VI    The low-order witness typed: any witness of the rank identity at order at most one is
        supplied by the act and is strictly smaller.
  VII   The division at order one, and the two parts; a low-order witness does not give the full
        value; the full row: read on the full row the act registers the rank on the seat and the
        leading coefficient as the arithmetic product; it is the full value; one act closes it; one
        curve refutes it; the full act is strictly stronger than the act, which does not give the
        full value.
  VIII  Keyless and keyed.
  IX    Freedom, the prime's shape, the triaxial count; the record wall in the Bool model and on
        frames; the bit is the world.
  X     The record and the seed: a finite record never forces the value, nor above order one with
        the low-order witness in place.
  XI    The retired premise Ω, typed on the curves of order at least two.
  XII   Rank is a dimension count over GF(2): in GF(2)^3 by decision, span and lock dual for one and
        two classes, and for every number of classes, k classes reaching at most 2^k of the 2^r
        points of GF(2)^r; and the count returns the rank: independent classes list the points of
        their span once each, and independent lattices with equally many points have equally many
        classes (rank_is_the_dimension_count).
  XIII  The hardened closure, whole.
  ROOT  The root, undeniable in deed, and the lock: the root holds by its deed and forces neither the
        value nor its failure; read on the row it is the act, and it grounds itself on the row or the
        full row exactly where the value or the full value holds; one theorem on no axiom binds the root,
        its readings, the act, the value, their keying, the closing theorem, the record wall and the
        finite record (the_lock); the constructed root by its deed, and the root of Appendix R repeated
        verbatim, which is the constructed root and carries the lock.
-/

namespace BSDHard

/-! ## I · the seat from first principle -/

/-- An integer equal to its own negative is zero. -/
theorem self_neg_zero : ∀ v : Int, v = -v → v = 0
  | Int.ofNat 0, _ => rfl
  | Int.ofNat (_ + 1), h => Int.noConfusion h
  | Int.negSucc _, h => Int.noConfusion h
```

FIGURE 7 BIRCH AND SWINNERTON-DYER · FORM BY THEOREM, VALUE BY THE ACT · MASTER v1.1.0_CC · EVERY NAME CHECKED

THE KERNEL, THE RECEIPTS AND THE TWO-LAYER CLOSURE

THE MASTER v1.1.0_CC · PARTS, NAMESPACES, THEOREMS, CONES

PART	NAMESPACE	THEOREMS	CONES	WHAT IT PROVES
0	BSDHard	101	∅ every one	The kernel of record, verbatim: the seat, the order, the act, parity, low order, Ω, F_2
A	BSD.NORMALFORM	25	∅ every one	The normal form: the record carries the value; the value is the transfer
B	BSD.HOST	16	∅ every one	The host and its binding; the sign fixes the parity; curves of every order
C	BSD.ROWSPLIT	7	∅ every one	The cut-agnostic division, carried verbatim from the Riemann master
C′	BSD.REALPART	9	∅ every one	The division read at order one, the low-order witness the real part
D	BSD.WALLS	4	∅ every one	The sign record normal form; the walls in one theorem
E	BSD.RESIDUE	7	∅ every one	The one implication, printed as a hypothesis and assumed nowhere
F	BSD.SUPPLY	12	∅ every one	The bit delivered twice; the witnesses typed; the one refuter
G	BSD.CLOSURE	5	∅ every one	The question closed to one bit
W	BSD.GATES	15	∅ every one	The twelve formal gates and the four refusals
X	BSD.ESCAPE	9	∅ every one	Nothing escapes; our way or no way; the capstone
T	BSD.TERMINAL	4	∅ every one	The open framing refuted; the ten steps; the final form
Bind	BSDMathlib	18	∅ · std	Mathlib's curves and rank; 37a1 and 389a1 computed by the library's group law

RECEIPTS

Master	BSD_Normal_Form_Paper_v1_1_0_CC.lean
SHA-256	66dff84e8bc25f950b48bf39909ccf9d0be4cabd9f1c3ee9721ac136e83a7112
Build	Lean 4.19.0 core · no import · exit 0, no message · 17 s
Cones	214 theorems, 214 pinned by #guard_msgs, every cone empty
Part 0, the kernel of record	BSD_Hardened.lean · d52b7a9e5b4f9be4…c8be
Mathlib binding	BSD_Mathlib_Binding_v1_1_0_CC.lean · 18 theorems · Mathlib v4.19.0
Binding SHA-256	c9e2c78d0644cd687f133236437557 1009424d64a5d74da299db53388fd4dbf8
Appendix R	TOE_Zero · Triaxial_Actuation · Root_Grade_Ledger · 65 theorems on no axiom

THE TWO-LAYER CLOSURE

Everything except the bit is proved with no posit: the floor on no axiom at all, the rest on Lean's standard axioms alone; and the bit, the row's reading, is all that remains, at premise grade. In this paper every theorem of the master stands on no axiom at all, and only the Mathlib binding reaches the standard axioms.

READING theorem on no axiom at all · binding theorem on Lean's standard axioms · definition or structure · names shown by their last component

Figure 7 | The master: its parts, namespaces, theorem counts and pinned cones, the receipts of the build, and the two-layer closure.

```
105
106 /-- Evenness, by recursion. -/
107 def evenB : Nat → Bool
108   | 0 => true
109   | n + 1 => !evenB n
110
111 /-- A centred expansion with a sign: the coefficients of a function about
      the centre, and the
112 reflection law of a functional equation about the centre with that sign,
      which negates the k-th
113 coefficient exactly when its parity disagrees with the sign: an even
      coefficient under a minus sign,
114 an odd one under a plus sign. -/
115 structure Centred where
116   a       : Nat → Int
117   minus   : Bool
118   reflect : ∀ k, (evenB k == minus) = true → a k = -(a k)
119
120 /-- A minus sign silences every even coefficient. -/
121 theorem minus_silences_even (C : Centred) (hm : C.minus = true) (k : Nat)
      (hk : evenB k = true) :
122     C.a k = 0 :=
123   self_neg_zero _ (C.reflect k (by rw [hm, hk]; rfl))
124
125 /-- A plus sign silences every odd coefficient. -/
126 theorem plus_silences_odd (C : Centred) (hm : C.minus = false) (k : Nat)
      (hk : evenB k = false) :
127     C.a k = 0 :=
128   self_neg_zero _ (C.reflect k (by rw [hm, hk]; rfl))
129
130 /-- THE SEAT: a minus sign forces vanishing at the centre. -/
131 theorem minus_sign_forces_vanishing (C : Centred) (hm : C.minus = true) :
      C.a 0 = 0 :=
132   minus_silences_even C hm 0 rfl
133
134 /-- The order of a coefficient sequence: the first index whose
      coefficient is nonzero. -/
135 def IsOrder (a : Nat → Int) (o : Nat) : Prop := a o ≠ 0 ∧ ∀ j, j < o → a j
      = 0
136
137 /-- THE SIGN FIXES THE PARITY OF THE ORDER: under a minus sign the order
      is odd. -/
138 theorem minus_order_odd (C : Centred) (hm : C.minus = true) (o : Nat) (ho
      : IsOrder C.a o) :
139     evenB o = false :=
140   match h : evenB o with
141   | true => absurd (minus_silences_even C hm o h) ho.1
142   | false => rfl
143
144 /-- Under a plus sign the order is even. -/
145 theorem plus_order_even (C : Centred) (hm : C.minus = false) (o : Nat)
      (ho : IsOrder C.a o) :
146     evenB o = true :=
147   match h : evenB o with
148   | false => absurd (plus_silences_odd C hm o h) ho.1
149   | true => rfl
150
151 /-! ## II · the order and its first reading -/
152
153 /-- THE FIRST-ORDER READING IS SILENT ABOVE ORDER ONE, by the definition
      of the order. -/
```

```
theorem first_reading_silent_above_one (a : Nat → Int) (o : Nat) (ho :
    IsOrder a o) (h : 2 ≤ o) :
    a 1 = 0 :=
  ho.2 1 h

/-- At order one the first-order reading speaks. -/
theorem first_reading_speaks_at_one (a : Nat → Int) (ho : IsOrder a 1) :
    a 1 ≠ 0 := ho.1

/-- The order is unique. -/
theorem order_unique (a : Nat → Int) (o p : Nat) (ho : IsOrder a o) (hp :
    IsOrder a p) : o = p :=
  match Nat.lt_or_ge o p with
  | Or.inl hlt => absurd (hp.2 o hlt) ho.1
  | Or.inr hge => match Nat.eq_or_lt_of_le hge with
    | Or.inl he => he.symm
    | Or.inr hlt => absurd (ho.2 p hlt) hp.1

/-! ## III · the frame, and existence as given -/

/-- A frame of elliptic curves: each with its rank, its order of
    vanishing at the centre, and
whether the leading coefficient equals the arithmetic product. -/
structure Frame where
  D     : Type
  rank  : D → Nat
  order : D → Nat
  coeff : D → Bool

/-- The value of the rank part: the rank is the order of vanishing, for
    every curve. -/
def Value (F : Frame) : Prop := ∀ d, F.rank d = F.order d

def calm : Frame := (Unit, fun _ => 1, fun _ => 1, fun _ => true)
/-- The counter world: a curve of rank zero whose order of vanishing is
    two. -/
def counter : Frame := (Unit, fun _ => 0, fun _ => 2, fun _ => true)

theorem calm_value : Value calm := fun _ => rfl
theorem counter_fails : ¬ Value counter := fun h => Nat.noConfusion (h ())

/-! ### existence as given -/

def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

theorem root_given : Root Empty (fun e => nomatch e) := fun e => nomatch e

theorem root_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U → Int),
    Root U ΔE → S) : S :=
  h Empty (fun e => nomatch e) root_given

def Arrow : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a

theorem arrow_given : Arrow := fun _ => ⟨id, fun _ => rfl⟩

theorem freedom_given : ∀ w : Bool, (!w) ≠ w
  | true => Bool.noConfusion
  | false => Bool.noConfusion

theorem given_is_not_the_value (P : Prop) : ¬ ∀ F : Frame, (P ↔ Value F)
    :=
  fun h => counter_fails ((h counter).mp ((h calm).mpr calm_value))

theorem arrow_forces_nothing : ¬ ∀ F : Frame, Arrow → Value F :=
  fun h => counter_fails (h counter arrow_given)

theorem given_in_both_worlds : (Arrow ∧ Value calm) ∧ (Arrow ∧ ¬ Value
    counter) :=
  ⟨⟨arrow_given, calm_value⟩, ⟨arrow_given, counter_fails⟩⟩

/-! ## IV · existence read on the row: the act -/

/-- Existence read on the row: every curve that exists registers its rank
    on the seat. -/
def Registered (F : Frame) : Prop := ∀ d, F.rank d = F.order d

theorem act_is_the_value (F : Frame) : Registered F ↔ Value F := ⟨fun h d
    => h d, fun h d => h d⟩

theorem act_is_keyed : Registered calm ∧ ¬ Registered counter :=
    ⟨calm_value, counter_fails⟩

/-! ## The root read on the row -/

/-- The indicator of a decided proposition: one where it holds, zero
    where it fails. -/
def indicator {p : Prop} : Decidable p → Int
  | isTrue _  => 1
  | isFalse _ => 0

/-- The indicator is positive exactly where its proposition holds. -/
theorem indicator_pos {p : Prop} (h : Decidable p) : 0 < indicator h ↔ p
    :=
  match h with
  | isTrue hp  => ⟨fun _ => hp, fun _ => show (0 : Int) < 1 by decide⟩
  | isFalse hn => ⟨fun h0 => absurd h0 (show ¬ (0 : Int) < 0 by decide),
      fun hp => absurd hp hn⟩

/-- Whether an option carries a value, decided. -/
def suppliedDec (o : Option Nat) : Decidable (∃ n, o = some n) :=
  match o with
  | some n => isTrue ⟨n, rfl⟩
  | none   => isFalse (fun ⟨_, h⟩ => Option.noConfusion h)

/-- THE ROOT READ ON THE ROW: to exist is to actuate, `Root` itself, with
    the row's existents, its curves,
and the row's actuation, one where the curve registers its rank at its
    order and zero where it does not. -/
def RootOnRow (F : Frame) : Prop :=
  Root F.D (fun d => indicator (Nat.decEq (F.rank d) (F.order d)))

/-- THE ROOT IS SATISFIABLE ON EVERY BACKGROUND: on every type, with
    actuation one at every existent. -/
theorem root_satisfiable (U : Type) : Root U (fun _ => 1) := fun _ =>
    show (0 : Int) < 1 by decide

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent, whose
    actuation is one. A theorem
with no hypothesis and no axiom. -/
theorem constructed_root_holds : Root Unit (fun _ => 1) :=
    root_satisfiable Unit

/-- THE ROOT READ ON THE ROW IS THE ACT. -/
theorem root_on_row_is_the_act (F : Frame) : RootOnRow F ↔ Registered F :=
  ⟨fun h d => (indicator_pos _).mp (h d), fun h d => (indicator_pos
    _).mpr (h d)⟩

/-- THE ROOT READ ON THE ROW IS THE VALUE. -/
theorem root_on_row_is_the_value (F : Frame) : RootOnRow F ↔ Value F :=
  ⟨fun h => (act_is_the_value F).mp ((root_on_row_is_the_act F).mp h),
   fun h => (root_on_row_is_the_act F).mpr ((act_is_the_value F).mpr h)⟩

/-- THE ROOT READ ON THE ROW IS KEYED: it holds where the value holds and
    fails where the value fails. -/
theorem root_on_row_is_keyed : RootOnRow calm ∧ ¬ RootOnRow counter :=
  ⟨(root_on_row_is_the_value calm).mpr calm_value, fun h => counter_fails
    ((root_on_row_is_the_value counter).mp h)⟩

/-- THE ACT IS THE WEAKEST PREMISE: the act forces the value on every
    frame, and every premise that forces
the value on every frame implies the act. -/
theorem act_is_the_weakest_premise :
    (∀ F : Frame, Registered F → Value F) ∧
    ∀ Q : Frame → Prop, (∀ F : Frame, Q F → Value F) → ∀ F : Frame, Q F →
      Registered F :=
  ⟨fun F h => (act_is_the_value F).mp h, fun _ hQ F hq =>
    (act_is_the_value F).mpr (hQ F hq)⟩

/-- EXISTENCE AS GIVEN HOLDS IN BOTH WORLDS: the root satisfiable on
    every background, the arrow and the
freedom bit hold in the world where the value holds and in the world
    where it fails. -/
theorem existence_as_given_in_both_worlds :
    ((∀ U : Type, Root U (fun _ => 1)) ∧ Arrow ∧ (∀ w : Bool, (!w) ≠ w)) ∧
      Value calm ∧ ¬ Value counter :=
  ⟨⟨root_satisfiable, arrow_given, freedom_given⟩, calm_value,
    counter_fails⟩

/-- NOTHING GIVEN REFUTES THE ACT: no statement that holds refutes the
    act on every frame. -/
theorem nothing_given_refutes : ∀ P : Prop, P → ¬ ∀ F : Frame, P → ¬
    Registered F :=
  fun _ hp h => h calm hp act_is_keyed.1

/-- NOTHING GIVEN FORCES THE VALUE: no statement that holds forces the
    value on every frame. -/
theorem nothing_given_forces : ∀ P : Prop, P → ¬ ∀ F : Frame, P → Value F
    :=
  fun _ hp h => counter_fails (h counter hp)

/-- THE ONLY REFUTER: wherever the act fails, it cannot be that no
    instance fails, a curve whose rank is not its order. -/
theorem only_refuter (F : Frame) : ¬ Registered F → ¬ ¬ ∃ d, F.rank d ≠
    F.order d :=
  fun hn hne => hn (fun d => match Nat.decEq (F.rank d) (F.order d) with
    | isTrue h => h
    | isFalse h => absurd ⟨d, h⟩ hne)

structure ActualCurves where
  F      : Frame
  supply : Registered F

/-- THE RANK PART FROM EXISTENCE, BY ONE ACT. -/
theorem bsd_from_existence (A : ActualCurves) : Value A.F :=
    (act_is_the_value A.F).mp A.supply

/-- THE SUPPLY IS EXACT: an act exists on a frame exactly when the value
    holds there. -/
theorem supply_iff (F : Frame) : Nonempty { A : ActualCurves // A.F = F }
    ↔ Value F :=
  ⟨fun ⟨⟨A, hA⟩⟩ => hA ▸ bsd_from_existence A,
   fun h => ⟨⟨(F, (act_is_the_value F).mpr h), rfl⟩⟩⟩

/-- Every curve lands on exactly one gate: its rank equals its order, or
    not. -/
theorem every_curve_lands (F : Frame) (d : F.D) : F.rank d = F.order d ∨
    F.rank d ≠ F.order d :=
  match Nat.decEq (F.rank d) (F.order d) with
  | isTrue h => Or.inl h
  | isFalse h => Or.inr h

theorem gates_exclusive (F : Frame) (d : F.D) : ¬ (F.rank d = F.order d ∧
    F.rank d ≠ F.order d) :=
  fun ⟨h, h'⟩ => h' h

theorem nothing_escapes (A : ActualCurves) : ∀ d, A.F.rank d = A.F.order
    d := A.supply

/-- One curve whose rank differs from its order refutes the act. -/
theorem counter_curve_refutes (F : Frame) (d : F.D) (h : F.rank d ≠
    F.order d) : ¬ Registered F :=
  fun hA => h (hA d)
```

```

/-! ## V · the parity bit -/

def ParityValue (F : Frame) : Prop := ∀ d, F.rank d % 2 = F.order d % 2

theorem value_gives_parity (F : Frame) (h : Value F) : ParityValue F :=
  fun d => by rw [h d]

/-- THE PARITY IS ONE BIT AND NOT THE VALUE: the counter world keeps the
    parity and fails. -/
theorem parity_is_not_the_value : ParityValue counter ∧ ¬ Value counter
    :=
  (fun _ => show 0 % 2 = 2 % 2 from rfl, counter_fails)

/-- A frame carrying centred expansions: each curve with its centred
    expansion, whose order is the curve's order. -/
structure SignedFrame where
  F   : Frame
  L   : F.D → Centred
  ord : ∀ d, IsOrder (L d).a (F.order d)

/-- THE SIGN FIXES THE PARITY OF THE CURVE'S ORDER: a curve whose
    expansion carries a minus sign has odd order. -/
theorem sign_fixes_curve_parity (S : SignedFrame) (d : S.F.D) (hm : (S.L
    d).minus = true) :
    evenB (S.F.order d) = false :=
  minus_order_odd (S.L d) hm (S.F.order d) (S.ord d)

/-! ## VI · the low-order witness typed -/

/-- A low-order witness: the rank identity on every curve of order at
    most one. The cited theorem
of the subject at order at most one has this shape; here it is a type,
    not a premise. -/
def LowWitness (F : Frame) : Prop := ∀ d, F.order d ≤ 1 → F.rank d =
    F.order d

/-- SUPPLIED BY THE ACT: the act gives a low-order witness. -/
theorem act_gives_low_witness (A : ActualCurves) : LowWitness A.F := fun
    d _ => A.supply d

/-- STRICTLY SMALLER: a frame carries a low-order witness and not the
    value. -/
theorem low_witness_strict : LowWitness counter ∧ ¬ Value counter :=
  (fun _ h => absurd h (by decide : ¬ ((2 : Nat) ≤ 1)), counter_fails)

/-- A frame with no low-order witness: a curve of order zero carrying
    rank two. -/
def noLow : Frame := (Unit, fun _ => 2, fun _ => 0, fun _ => true)

theorem low_witness_not_given : ¬ LowWitness noLow := fun h =>
    Nat.noConfusion (h () (by decide))

/-! ## VII · the division and the two parts -/

def ValueOn (F : Frame) (cut : F.D → Bool) (b : Bool) : Prop :=
  ∀ d, cut d = b → F.rank d = F.order d

theorem row_split (F : Frame) (cut : F.D → Bool) :
    Value F ↔ ValueOn F cut true ∧ ValueOn F cut false :=
  (fun h => (fun d _ => h d, fun d _ => h d),
   fun (ht, hf) d => match hc : cut d with
     | true => ht d hc
     | false => hf d hc)

/-- At order one the proved half holds and the remainder fails, in the
    counter world. -/
theorem remainder_not_forced :
    ValueOn counter (fun d => Nat.ble (counter.order d) 1) true ∧
    ¬ ValueOn counter (fun d => Nat.ble (counter.order d) 1) false :=
  (fun _ h => absurd h (by decide : ¬ (Nat.ble 2 1 = true)),
   fun h => counter_fails (fun d => h d rfl))

/-- The full value: the rank part and the leading-coefficient part. -/
def FullValue (F : Frame) : Prop := Value F ∧ ∀ d, F.coeff d = true

def coeffCounter : Frame := (Unit, fun _ => 1, fun _ => 1, fun _ => false)

/-- THE RANK PART DOES NOT GIVE THE LEADING COEFFICIENT. -/
theorem rank_part_not_full : Value coeffCounter ∧ ¬ FullValue
    coeffCounter :=
  (fun _ => rfl, fun ⟨_, h⟩ => Bool.noConfusion (h ()))

theorem two_parts (F : Frame) : FullValue F ↔ Value F ∧ ∀ d, F.coeff d =
    true := Iff.rfl

/-- A LOW-ORDER WITNESS DOES NOT GIVE THE FULL VALUE: a frame carries one
    and fails at the leading coefficient. -/
theorem low_witness_not_full : LowWitness coeffCounter ∧ ¬ FullValue
    coeffCounter :=
  (fun _ _ => rfl, rank_part_not_full.2)

/-! ### the full row -/

/-- Existence read on the full row: every curve that exists registers its
    rank on the seat and its leading
coefficient as the arithmetic product. -/
def RegisteredFull (F : Frame) : Prop := ∀ d, F.rank d = F.order d ∧
    F.coeff d = true

/-- THE FULL ACT IS THE FULL VALUE. -/
theorem full_act_is_the_full_value (F : Frame) : RegisteredFull F ↔
    FullValue F :=
  (fun h => (fun d => (h d).1, fun d => (h d).2), fun h d => ⟨h.1 d, h.2
    d⟩)

/-- The full act carries the act of the rank part. -/
theorem full_act_gives_rank_act (F : Frame) (h : RegisteredFull F) :
    Registered F := fun d => (h d).1

/-- THE FULL ACT IS KEYED: it holds in the calm world and fails where the
    coefficient fails and where the rank fails. -/
theorem full_act_is_keyed : RegisteredFull calm ∧ ¬ RegisteredFull
    coeffCounter ∧ ¬ RegisteredFull counter :=
  ⟨fun _ => ⟨rfl, rfl⟩, fun h => Bool.noConfusion (h ()).2, fun h =>
    Nat.noConfusion (h ()).1⟩

/-- THE ROOT READ ON THE FULL ROW: `Root` itself, with the row's curves
    as existents and actuation one where the curve
registers its rank at its order and its leading coefficient, zero where
    it does not. -/
def RootOnFullRow (F : Frame) : Prop :=
  Root F.D (fun d => indicator (@instDecidableAnd _ _ (Nat.decEq (F.rank
    d) (F.order d)) (instDecidableEqBool (F.coeff d) true)))

/-- THE ROOT READ ON THE FULL ROW IS THE FULL ACT. -/
theorem root_on_full_row_is_the_full_act (F : Frame) : RootOnFullRow F ↔
    RegisteredFull F :=
  ⟨fun h d => (indicator_pos _).mp (h d), fun h d => (indicator_pos
    _).mpr (h d)⟩

/-- THE ROOT READ ON THE FULL ROW IS THE FULL VALUE, AND KEYED. -/
theorem root_on_full_row_is_the_full_value :
    (∀ F : Frame, RootOnFullRow F ↔ FullValue F) ∧ RootOnFullRow calm ∧ ¬
      RootOnFullRow coeffCounter :=
  ⟨fun F => (root_on_full_row_is_the_full_act F).trans
    (full_act_is_the_full_value F),
   ⟨(root_on_full_row_is_the_full_act calm).mpr full_act_is_keyed.1,
    fun h => full_act_is_keyed.2.1 ((root_on_full_row_is_the_full_act
      coeffCounter).mp h)⟩⟩

/-- The actual curves, read on the full row. -/
structure ActualCurvesFull where
  F      : Frame
  supply : RegisteredFull F

/-- THE FULL VALUE FROM EXISTENCE, BY ONE ACT: the rank part and the
    leading coefficient. -/
theorem bsd_full_from_existence (A : ActualCurvesFull) : FullValue A.F :=
  ⟨full_act_is_the_full_value A.F).mp A.supply⟩

/-- THE FULL SUPPLY IS EXACT: a full act exists on a frame exactly when
    the full value holds there. -/
theorem full_supply_iff (F : Frame) : Nonempty { A : ActualCurvesFull //
    A.F = F } ↔ FullValue F :=
  ⟨fun ⟨⟨A, hA⟩⟩ => hA ▸ bsd_full_from_existence A,
   fun h => ⟨⟨F, (full_act_is_the_full_value F).mpr h⟩, rfl⟩⟩

/-- THE FULL ACT IS THE WEAKEST PREMISE OF THE FULL VALUE. -/
theorem full_act_is_the_weakest_premise :
    (∀ F : Frame, RegisteredFull F → FullValue F) ∧
    ∀ Q : Frame → Prop, (∀ F : Frame, Q F → FullValue F) → ∀ F : Frame, Q
      F → RegisteredFull F :=
  ⟨fun F h => (full_act_is_the_full_value F).mp h, fun _ hQ F hq =>
    (full_act_is_the_full_value F).mpr (hQ F hq)⟩

/-- One curve whose leading coefficient is not the arithmetic product
    refutes the full act. -/
theorem coefficient_refutes (F : Frame) (d : F.D) (h : F.coeff d = false)
    : ¬ RegisteredFull F :=
  fun hA => Bool.noConfusion (h.symm.trans (hA d).2)

/-- THE FULL ACT IS STRICTLY STRONGER THAN THE ACT: it carries the act,
    and a frame carries the act and not the full act. -/
theorem full_act_strictly_stronger :
    (∀ F : Frame, RegisteredFull F → Registered F) ∧ Registered
      coeffCounter ∧ ¬ RegisteredFull coeffCounter :=
  ⟨full_act_gives_rank_act, rank_part_not_full.1, full_act_is_keyed.2.1⟩

/-- THE ACT DOES NOT GIVE THE FULL VALUE: on some frame the act holds and
    the full value fails. -/
theorem act_does_not_give_full_value : ¬ ∀ F : Frame, Registered F →
    FullValue F :=
  fun h => rank_part_not_full.2 (h coeffCounter rank_part_not_full.1)

/-! ## VIII · keyless and keyed -/

def Keyed (P : Frame → Prop) : Prop := ∃ F, ¬ P F

theorem value_is_keyed : Keyed Value := ⟨counter, counter_fails⟩

theorem no_keyless_statement_is_the_act (P : Prop) : ¬ ∀ F : Frame, (P ↔
    Registered F) :=
  fun h => given_is_not_the_value P (fun F => (h F).trans
    (act_is_the_value F))

theorem pulse_does_not_certify : Root Empty (fun e => nomatch e) ∧ ¬
    Value counter :=
  ⟨root_given, counter_fails⟩

/-! ## IX · freedom, the prime, the triaxial count -/

/-- The kinetic record, the Bool model: a record of the free bit that
    reads the same on both of its values. -/
def kineticRecord (_w : Bool) : Nat := 0

/-- THE RECORD WALL, the Bool model: no reading of the kinetic record
    returns the bit. -/
theorem record_wall : ¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w :=
  fun ⟨_, hg⟩ => Bool.noConfusion ((hg false).symm.trans (hg true))
```

```
/-- THE RECORD WALL ON FRAMES: a record that reads the calm world and the counter world alike has no reading that
is the value on every frame. -/
theorem record_wall_on_frames {Rec : Type} (rec : Frame → Rec) (h : rec calm = rec counter) (g : Rec → Prop) :
  ¬ ∀ F : Frame, (g (rec F) ↔ Value F) :=
 fun hall => counter_fails ((hall counter).mp (h ▸ (hall calm).mpr calm_value))

/-- The two worlds, indexed by the bit: true gives the calm world, false the counter world. -/
def world : Bool → Frame
 | true => calm
 | false => counter

/-- THE BIT IS THE WORLD: the value holds in the world of a bit exactly when the bit is true. -/
theorem the_bit_is_the_world : ∀ w : Bool, (Value (world w) → w = true) ∧ (w = true → Value (world w)) :=
 fun w => match w with
 | true => (fun _ => rfl, fun _ => calm_value)
 | false => (fun h => absurd h counter_fails, fun h => Bool.noConfusion h)

theorem fibre_is_two : ([false, true].filter (fun w => kineticRecord w == kineticRecord true)).length = 2 := by
 decide

def mulFibre (n : Nat) : List (Nat × Nat) :=
 (List.range (n + 1)).flatMap fun a =>
  (List.range (n + 1)).filterMap fun b => if a * b = n then some (a, b) else none

theorem prime_shape :
  mulFibre 7 = [(1, 7), (7, 1)] ∧ (mulFibre 7).all (fun p => p.1 != p.2) = true := by
 decide

/-! ### the triaxial count -/

def dot2 (r x : Bool × Bool × Bool) : Bool :=
 xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool × Bool × Bool) :=
 [false, true].flatMap fun a => [false, true].flatMap fun b => [false, true].map fun c => (a, b, c)

def nonzero : List (Bool × Bool × Bool) := cube.filter (fun r => r != (false, false, false))

def solCount (rows : List ((Bool × Bool × Bool) × Bool)) : Nat :=
 (cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x == rt.2))).length

def xor3 (a b : Bool × Bool × Bool) : Bool × Bool × Bool :=
 (xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
  nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
   [false, true].all (fun t1 => [false, true].all (fun t2 =>
    solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
  nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun r3 =>
   r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
   [false, true].all (fun t1 => [false, true].all (fun t2 => [false, true].all (fun t3 =>
    solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true := by
  decide

/-! ## X · the record and the seed -/

def staged (n : Nat) : Frame := (Nat, fun d => if d < n then 1 else 0, fun _ => 1, fun _ => true)

theorem finite_record_never_forces (n : Nat) :
  (∀ d, d < n → (staged n).rank d = (staged n).order d) ∧ ¬ Value (staged n) :=
 (fun d hd => by show (if d < n then 1 else 0) = 1; rw [if_pos hd],
  fun h => by
   have e : (staged n).rank n = 0 := by
    show (if n < n then 1 else 0) = 0
    rw [if_neg (Nat.lt_irrefl n)]
   have k := h n
   rw [e] at k
   exact Nat.noConfusion k)

/-- A staged frame above order one: every curve has order two, rank two below the stage and rank zero from it on. -/
def stagedHigh (n : Nat) : Frame := (Nat, fun d => if d < n then 2 else 0, fun _ => 2, fun _ => true)

/-- A FINITE RECORD NEVER FORCES THE VALUE ABOVE ORDER ONE: the low-order witness holds and the record agrees below the
stage, and the value fails. -/
theorem finite_record_never_forces_above_one (n : Nat) :
  LowWitness (stagedHigh n) ∧ (∀ d, d < n → (stagedHigh n).rank d = (stagedHigh n).order d) ∧
  ¬ Value (stagedHigh n) :=
 (fun _ h => absurd h (by decide : ¬ ((2 : Nat) ≤ 1)),
  fun d hd => by show (if d < n then 2 else 0) = 2; rw [if_pos hd],
  fun h => by
   have e : (stagedHigh n).rank n = 0 := by
    show (if n < n then 2 else 0) = 0
    rw [if_neg (Nat.lt_irrefl n)]
   have k := h n
   rw [e] at k
   exact Nat.noConfusion k)

structure Ladder where
 Decided : Nat → Prop
 base    : Decided 0
 mono    : ∀ r s, Decided s → r ≤ s → Decided r
 δ       : Nat → Nat
 step    : ∀ r, Decided r → Decided (r + δ r)

theorem uniform_step_forces_all (L : Ladder) (hδ : ∀ r, 1 ≤ L.δ r) : ∀ r, L.Decided r
 | 0 => L.base
 | r + 1 => L.mono (r + 1) (r + L.δ r) (L.step r (uniform_step_forces_all L hδ r))
   (Nat.add_le_add_left (hδ r) r)

/-! ## XI · the retired premise Ω -/

/-- A frame with its provenance record: for each curve, whether a construction fixed in advance,
consulting neither the rank nor the Selmer group, produces its family of classes. -/
structure Provenanced where
 F     : Frame
 fixed : F.D → Bool

/-- The higher-cycle axiom Ω of the earlier draft, on the curves it constrains: every curve of order at least two
has its family from a construction fixed in advance, and the family's height determinant, descent and Selmer clauses
give the rank identity. -/
def OmegaHolds (P : Provenanced) : Prop := (∀ d, 2 ≤ P.F.order d → P.fixed d = true) ∧ Value P.F

theorem omega_gives_value (P : Provenanced) (h : OmegaHolds P) : Value P.F := h.2

/-- A world of one curve of rank two at order two. -/
def calmTwo : Frame := (Unit, fun _ => 2, fun _ => 2, fun _ => true)

/-- Its curve of order two has no construction fixed in advance. -/
def unprovenanced : Provenanced := (calmTwo, fun _ => false)

/-- THE AXIOM Ω IS RETIRED: it gives the value and is strictly stronger; a curve of order two carries the value with
no construction fixed in advance, so the value does not hand Ω back. -/
theorem omega_strictly_stronger : Value unprovenanced.F ∧ ¬ OmegaHolds unprovenanced :=
 (fun _ => rfl, fun (h, _) => Bool.noConfusion (h () (Nat.le_refl 2)))

/-! ## XII · rank is a dimension count -/

def zero3 : Bool × Bool × Bool := (false, false, false)

/-- The span over GF(2) of a list of vectors in GF(2)^3. -/
def span (vs : List (Bool × Bool × Bool)) : List (Bool × Bool × Bool) :=
 cube.filter (fun x => vs.foldr (fun v acc => acc ++ acc.map (xor3 v)) [zero3] |>.contains x)

/-- ONE CLASS CANNOT CERTIFY RANK TWO: every nonzero vector spans exactly two points, rank one. -/
theorem one_class_spans_two : nonzero.all (fun v => (span [v]).length == 2) = true := by decide

/-- TWO INDEPENDENT CLASSES CERTIFY RANK TWO: they span exactly four points. -/
theorem two_classes_span_four :
  nonzero.all (fun v => nonzero.all (fun w => v == w || (span [v, w]).length == 4)) = true := by
 decide

/-- THREE INDEPENDENT CLASSES CERTIFY RANK THREE: they span all eight points. -/
theorem three_classes_span_eight :
  nonzero.all (fun u => nonzero.all (fun v => nonzero.all (fun w =>
   u == v || w == u || w == v || w == xor3 u v || (span [u, v, w]).length == 8))) = true := by
 decide

/-- k classes span at most 2^k points: the certified rank is at most the number of classes. -/
theorem k_classes_bound :
  (span [] ).length = 1 ∧ nonzero.all (fun v => (span [v]).length ≤ 2) = true ∧
  nonzero.all (fun v => nonzero.all (fun w => (span [v, w]).length ≤ 4)) = true := by
 decide

/-- SPAN AND LOCK ARE DUAL: for one and for two classes, the points k independent classes span times the solutions
left by k independent constraints is the eight points of GF(2)^3. -/
theorem span_and_lock_are_dual :
  nonzero.all (fun v => nonzero.all (fun r => [false, true].all (fun t =>
   (span [v]).length * solCount [(r, t)] == 8))) = true ∧
  nonzero.all (fun v => nonzero.all (fun w => v == w ||
   nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
    [false, true].all (fun t1 => [false, true].all (fun t2 =>
     (span [v, w]).length * solCount [(r1, t1), (r2, t2)] == 8)))))) = true := by
 decide

/-! ### for every number of classes -/

/-- A vector over GF(2) of any length, added coordinatewise. -/
def xorV : List Bool → List Bool → List Bool
```

```
  | a :: as, b :: bs => xor a b :: xorV as bs
  | _, _ => []

/-- Every combination over GF(2) of a list of classes, listed once per subset of the list. -/
def combos (zero : List Bool) : List (List Bool) → List (List Bool)
  | [] => [zero]
  | v :: vs => combos zero vs ++ (combos zero vs).map (xorV v)

theorem len_app {α : Type} : ∀ (a b : List α), (a ++ b).length = a.length + b.length
  | [], b => (Nat.zero_add b.length).symm
  | _ :: a, b => (congrArg Nat.succ (len_app a b)).trans (Nat.succ_add a.length b.length).symm

theorem add_congr {a b c d : Nat} (h1 : a = b) (h2 : c = d) : a + c = b + d := h1 ▸ h2 ▸ rfl

theorem len_map {α β : Type} (f : α → β) : ∀ (a : List α), (a.map f).length = a.length
  | [] => rfl
  | _ :: a => congrArg Nat.succ (len_map f a)

/-- K CLASSES REACH AT MOST 2^K POINTS: the combinations of k classes are 2^k in number, for every k. -/
theorem combos_length (zero : List Bool) : ∀ vs : List (List Bool), (combos zero vs).length = 2 ^ vs.length
  | [] => rfl
  | v :: vs => ((len_app _ _).trans (congrArg (fun m => (combos zero vs).length + m) (len_map (xorV v) _))).trans
      ((congrArg (fun m => m + m) (combos_length zero vs)).trans
        (Nat.mul_two (2 ^ vs.length)).symm)

/-- All the points of GF(2)^r, listed. -/
def points : Nat → List (List Bool)
  | 0 => [[]]
  | r + 1 => (points r).map (List.cons false) ++ (points r).map (List.cons true)

/-- GF(2)^r has 2^r points. -/
theorem points_length : ∀ r : Nat, (points r).length = 2 ^ r
  | 0 => rfl
  | r + 1 => ((len_app _ _).trans (add_congr ((len_map _ _).trans (points_length r))
      ((len_map _ _).trans (points_length r)))).trans (Nat.mul_two (2 ^ r)).symm

theorem two_pow_le : ∀ {k r : Nat}, k ≤ r → 2 ^ k ≤ 2 ^ r
  | _, 0, h => Nat.le_of_eq (congrArg (2 ^ ·) (Nat.le_zero.mp h))
  | _, r + 1, h => match Nat.eq_or_lt_of_le h with
    | Or.inl e => Nat.le_of_eq (congrArg (2 ^ ·) e)
    | Or.inr l => Nat.le_trans (two_pow_le (Nat.le_of_lt_succ l))
        (Nat.le_trans (Nat.le_add_right (2 ^ r) (2 ^ r)) (Nat.le_of_eq (Nat.mul_two (2 ^ r)).symm))

/-- FEWER CLASSES FALL SHORT: for every rank r and every k below it, k classes reach fewer than the 2^r points of
GF(2)^r; certifying rank r takes r classes. -/
theorem fewer_classes_fall_short (zero : List Bool) (vs : List (List Bool)) (r : Nat) (h : vs.length < r) :
    (combos zero vs).length < (points r).length :=
  match r, h with
  | r + 1, h => (combos_length zero vs).symm ▸ (points_length (r + 1)).symm ▸
      Nat.lt_of_le_of_lt (two_pow_le (Nat.le_of_lt_succ h))
        (Nat.lt_of_lt_of_le (Nat.lt_add_of_pos_right (Nat.two_pow_pos r))
          (Nat.le_of_eq (Nat.mul_two (2 ^ r)).symm))

/-! ### rank is a dimension count: the count returns the rank -/

/-- No two entries of a list coincide. -/
def distinctB : List (List Bool) → Bool
  | [] => true
  | x :: xs => (!xs.contains x) && distinctB xs

/-- Classes are independent when their combinations are pairwise distinct points: the combinations are then the
points of the span, listed once each. -/
abbrev Independent (zero : List Bool) (vs : List (List Bool)) : Prop := distinctB (combos zero vs) = true

/-- Three unit classes are independent; a class listed twice is not. -/
theorem independence_discriminates :
    Independent [false, false, false] [[true, false, false], [false, true, false], [false, false, true]] ∧
    ¬ Independent [false, false, false] [[true, false, false], [true, false, false]] := by
  decide

theorem two_pow_succ_ne_one (n : Nat) : 2 ^ (n + 1) ≠ 1 := fun h =>
  have h2 : 1 * 2 ≤ 2 ^ n * 2 := Nat.mul_le_mul_right 2 (Nat.two_pow_pos n)
  have h3 : 2 ^ n * 2 = 1 := (Nat.pow_succ 2 n).symm.trans h
  absurd (h3 ▸ h2 : 1 * 2 ≤ 1) (by decide)

/-- Two to the power is injective: the count returns the exponent. -/
theorem two_pow_inj : ∀ a b : Nat, 2 ^ a = 2 ^ b → a = b
  | 0, 0, _ => rfl
  | 0, b + 1, h => absurd h.symm (two_pow_succ_ne_one b)
  | a + 1, 0, h => absurd h (two_pow_succ_ne_one a)
  | a + 1, b + 1, h => congrArg Nat.succ (two_pow_inj a b
      (Nat.eq_of_mul_eq_mul_right (by decide : 0 < 2) ((Nat.pow_succ 2 a).symm.trans (h.trans (Nat.pow_succ 2 b)))))

/-- RANK IS A DIMENSION COUNT: independent classes list the points of their span once each, two to the number of
classes of them; and the count returns that number: independent lattices with equally many points have equally many
classes. The rank of a lattice of classes over GF(2), the number of its independent classes, is read off its
dimension count. -/
theorem rank_is_the_dimension_count (zero : List Bool) (vs : List (List Bool)) (h : Independent zero vs) :
    distinctB (combos zero vs) = true ∧ (combos zero vs).length = 2 ^ vs.length ∧
    ∀ ws : List (List Bool), Independent zero ws →
      (combos zero ws).length = (combos zero vs).length → ws.length = vs.length :=
  ⟨h, combos_length zero vs, fun ws _ e =>
    two_pow_inj ws.length vs.length (((combos_length zero ws).symm.trans e).trans (combos_length zero vs))⟩

/-! ## XIII · the hardened closure, whole -/

/-- THE HARDENED CLOSURE: its only premise is the act, inside `ActualCurves`; everything else
holds of every frame, every centred expansion, or a stated model. -/
theorem bsd_hardened_closure :
    (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
    (∀ F : Frame, Registered F ↔ Value F) ∧
    (∀ A : ActualCurves, Value A.F) ∧
    (∀ C : Centred, C.minus = true → C.a 0 = 0) ∧
    (∀ (C : Centred), C.minus = true → ∀ o, IsOrder C.a o → evenB o = false) ∧
    (∀ (a : Nat → Int) (o : Nat), IsOrder a o → 2 ≤ o → a 1 = 0) ∧
    (ParityValue counter ∧ ¬ Value counter) ∧
    (∀ A : ActualCurves, LowWitness A.F) ∧
    (LowWitness counter ∧ ¬ Value counter) ∧
    nonzero.all (fun v => (span [v]).length == 2) = true ∧
    (Value unprovenanced.F ∧ ¬ OmegaHolds unprovenanced) ∧
    (Value coeffCounter ∧ ¬ FullValue coeffCounter) ∧
    (∀ F : Frame, RegisteredFull F ↔ FullValue F) ∧
    (∀ A : ActualCurvesFull, FullValue A.F) ∧
    Keyed Value :=
  ⟨given_is_not_the_value, act_is_the_value, bsd_from_existence, minus_sign_forces_vanishing,
   minus_order_odd, first_reading_silent_above_one, parity_is_not_the_value, act_gives_low_witness,
   low_witness_strict, one_class_spans_two, omega_strictly_stronger, rank_part_not_full,
   full_act_is_the_full_value, bsd_full_from_existence, value_is_keyed⟩


/-! ## The root, undeniable in deed -/

/-- A self-grounding root, the structure of Appendix R in the same words: a type of acts, inhabited, every
one of which instances the root. -/
structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

/-- THE ROOT IS UNDENIABLE IN DEED: a denial of a self-grounding root is a deed, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial : G.Act) : R :=
  G.instances denial

/-- No outside proof adds to a self-grounding root: an outside proof of it is worth exactly the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R) (Q : Prop) : (Q → R) ↔ R :=
  ⟨fun _ => G.instances G.anAct, fun r _ => r⟩

/-- THE UNDENIABLE ROOT HOLDS IN BOTH WORLDS: it is held by its deed in a frame where the value holds
and in one where it fails, so it forces neither the value nor its failure on every frame. -/
theorem undeniable_root_forces_no_value {R : Prop} (G : SelfGrounding R) :
    R ∧ Value calm ∧ ¬ Value counter ∧ ¬ (∀ F : Frame, R → Value F) ∧ ¬ (∀ F : Frame, R → ¬ Value F) :=
  ⟨G.instances G.anAct, calm_value, counter_fails, fun h => counter_fails (h counter (G.instances G.anAct)),
   fun h => h calm (G.instances G.anAct) calm_value⟩

/-- THE ROOT READ ON THE ROW IS KEYED: the root holds, and read uniformly on the frames it gives no value;
read on the row, with the row's existents and the row's actuation, it is the value exactly, and keyed. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
    R ∧ ¬ (∀ F : Frame, R → Value F) ∧ (∀ F : Frame, RootOnRow F ↔ Value F) ∧ (RootOnRow calm ∧ ¬ RootOnRow counter) :=
  ⟨G.instances G.anAct, fun h => counter_fails (h counter (G.instances G.anAct)), root_on_row_is_the_value,
   root_on_row_is_keyed⟩

/-- THE ROOT ON THE ROW GROUNDS ITSELF EXACTLY WHERE THE VALUE HOLDS: a self-grounding of the root read on the row
exists on a frame exactly when the value holds there. -/
theorem row_root_grounds_itself_iff_value (F : Frame) : Nonempty (SelfGrounding (RootOnRow F)) ↔ Value F :=
  ⟨fun ⟨G⟩ => (root_on_row_is_the_value F).mp (G.instances G.anAct),
   fun h => ⟨(Unit, (), fun _ => (root_on_row_is_the_value F).mpr h)⟩⟩

/-- THE ROOT ON THE FULL ROW GROUNDS ITSELF EXACTLY WHERE THE FULL VALUE HOLDS: a self-grounding of the root read on
the full row exists on a frame exactly when the full value holds there. -/
theorem full_row_root_grounds_itself_iff_full_value (F : Frame) :
    Nonempty (SelfGrounding (RootOnFullRow F)) ↔ FullValue F :=
```

```
  (fun (G) => (root_on_full_row_is_the_full_value.1 F).mp (G.instances
    G.anAct),
   fun h => ((Unit, (), fun _ => (root_on_full_row_is_the_full_value.1
     F).mpr h)))

/-- THE LOCK, one theorem on no axiom: the constructed root holds with no
    hypothesis; every self-grounding
root holds by its deed; no given statement is the value on every frame;
    the root read uniformly on the frames
gives no value; read on the row the root is the act, the act is the value
    exactly, and both are keyed; the
closing theorem gives the value on every actual frame; no reading of the
    kinetic record returns the bit; and
no finite record forces the value. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
    Root Unit (fun _ => 1) ∧
    R ∧
    (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
    ¬ (∀ F : Frame, R → Value F) ∧
    (∀ F : Frame, RootOnRow F ↔ Registered F) ∧
    (∀ F : Frame, Registered F ↔ Value F) ∧
    (RootOnRow calm ∧ ¬ RootOnRow counter) ∧
    (Registered calm ∧ ¬ Registered counter) ∧
    (∀ A : ActualCurves, Value A.F) ∧
    (¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w) ∧
    (∀ n : Nat, (∀ d, d < n → (staged n).rank d = (staged n).order d) ∧ ¬
       Value (staged n)) :=
  (constructed_root_holds, G.instances G.anAct, given_is_not_the_value,
    (root_read_on_row_is_keyed G).2.1,
   root_on_row_is_the_act, act_is_the_value, root_on_row_is_keyed,
     act_is_keyed, bsd_from_existence, record_wall,
   finite_record_never_forces)

/-! ### the constructed root by its deed -/

/-- The deed of the constructed root: one act, the unit, instancing the
    root on one existent of actuation one. -/
def rootDeed : SelfGrounding (Root Unit (fun _ => 1)) := (Unit, (), fun _
    => constructed_root_holds)

/-- THE CONSTRUCTED ROOT HOLDS BY ITS DEED AND FORCES NO VALUE: its deed
    instances it, and it gives no value on every
frame. -/
theorem constructed_root_by_its_deed : Root Unit (fun _ => 1) ∧ ¬ (∀ F :
    Frame, Root Unit (fun _ => 1) → Value F) :=
  (denial_reenacts_root rootDeed rootDeed.anAct,
     (undeniable_root_forces_no_value rootDeed).2.2.2.1)

/-- The actuation of Appendix R, repeated verbatim: one at the single
    existent. -/
def ΔE₀ : Unit → Int := fun _ => 1

/-- The root of Appendix R, repeated verbatim: every existent actuates. -/
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The self-grounding of Appendix R, repeated verbatim: one act, which
    instances the root. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int) < 1
    by decide)

/-- THE ROOT OF APPENDIX R IS THE CONSTRUCTED ROOT: the two statements
    are one. -/
theorem RA₀_is_the_constructed_root : RA₀ ↔ Root Unit (fun _ => 1) :=
    Iff.rfl

/-- THE LOCK ON THE ROOT OF APPENDIX R: the root holds by its deed, gives
    no value on every frame, and read on the
row it is the value exactly. -/
theorem the_lock_on_the_root :
    RA₀ ∧ ¬ (∀ F : Frame, RA₀ → Value F) ∧ (∀ F : Frame, RootOnRow F ↔
       Value F) :=
  (ra₀.instances ra₀.anAct, (undeniable_root_forces_no_value
    ra₀).2.2.2.1, root_on_row_is_the_value)

end BSDHard

/-! ## Cones, pinned as printed -/
/-- info: 'BSDHard.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.self_neg_zero
/-- info: 'BSDHard.minus_silences_even' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.minus_silences_even
/-- info: 'BSDHard.plus_silences_odd' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.plus_silences_odd
/-- info: 'BSDHard.minus_sign_forces_vanishing' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSDHard.minus_sign_forces_vanishing
/-- info: 'BSDHard.minus_order_odd' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.minus_order_odd
/-- info: 'BSDHard.plus_order_even' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.plus_order_even
/-- info: 'BSDHard.first_reading_silent_above_one' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSDHard.first_reading_silent_above_one
/-- info: 'BSDHard.first_reading_speaks_at_one' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSDHard.first_reading_speaks_at_one
/-- info: 'BSDHard.order_unique' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.order_unique
/-- info: 'BSDHard.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.calm_value
/-- info: 'BSDHard.counter_fails' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.counter_fails
/-- info: 'BSDHard.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.root_given
/-- info: 'BSDHard.root_conservative' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.root_conservative
/-- info: 'BSDHard.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.arrow_given
/-- info: 'BSDHard.freedom_given' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.freedom_given
/-- info: 'BSDHard.given_is_not_the_value' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSDHard.given_is_not_the_value
/-- info: 'BSDHard.arrow_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.arrow_forces_nothing
/-- info: 'BSDHard.given_in_both_worlds' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.given_in_both_worlds
/-- info: 'BSDHard.act_is_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.act_is_the_value
/-- info: 'BSDHard.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.act_is_keyed
/-- info: 'BSDHard.bsd_from_existence' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.bsd_from_existence
/-- info: 'BSDHard.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.supply_iff
/-- info: 'BSDHard.every_curve_lands' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.every_curve_lands
/-- info: 'BSDHard.gates_exclusive' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.gates_exclusive
/-- info: 'BSDHard.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.nothing_escapes
/-- info: 'BSDHard.counter_curve_refutes' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSDHard.counter_curve_refutes
/-- info: 'BSDHard.value_gives_parity' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.value_gives_parity
/-- info: 'BSDHard.parity_is_not_the_value' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSDHard.parity_is_not_the_value
/-- info: 'BSDHard.act_gives_low_witness' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSDHard.act_gives_low_witness
/-- info: 'BSDHard.low_witness_strict' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.low_witness_strict
/-- info: 'BSDHard.low_witness_not_given' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSDHard.low_witness_not_given
/-- info: 'BSDHard.row_split' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.row_split
/-- info: 'BSDHard.remainder_not_forced' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.remainder_not_forced
/-- info: 'BSDHard.rank_part_not_full' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.rank_part_not_full
/-- info: 'BSDHard.two_parts' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.two_parts
/-- info: 'BSDHard.value_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.value_is_keyed
/-- info: 'BSDHard.no_keyless_statement_is_the_act' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSDHard.no_keyless_statement_is_the_act
/-- info: 'BSDHard.pulse_does_not_certify' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSDHard.pulse_does_not_certify
/-- info: 'BSDHard.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.record_wall
/-- info: 'BSDHard.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.fibre_is_two
/-- info: 'BSDHard.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.prime_shape
/-- info: 'BSDHard.two_axes_leave_two' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.two_axes_leave_two
/-- info: 'BSDHard.three_axes_lock_one' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.three_axes_lock_one
/-- info: 'BSDHard.finite_record_never_forces' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSDHard.finite_record_never_forces
/-- info: 'BSDHard.uniform_step_forces_all' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSDHard.uniform_step_forces_all
/-- info: 'BSDHard.omega_gives_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.omega_gives_value
/-- info: 'BSDHard.omega_strictly_stronger' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSDHard.omega_strictly_stronger
/-- info: 'BSDHard.one_class_spans_two' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.one_class_spans_two
/-- info: 'BSDHard.two_classes_span_four' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSDHard.two_classes_span_four
/-- info: 'BSDHard.three_classes_span_eight' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSDHard.three_classes_span_eight
/-- info: 'BSDHard.k_classes_bound' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.k_classes_bound
/-- info: 'BSDHard.bsd_hardened_closure' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.bsd_hardened_closure
/-- info: 'BSDHard.denial_reenacts_root' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.denial_reenacts_root
/-- info: 'BSDHard.external_proof_adds_nothing' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSDHard.external_proof_adds_nothing
/-- info: 'BSDHard.undeniable_root_forces_no_value' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSDHard.undeniable_root_forces_no_value
/-- info: 'BSDHard.root_read_on_row_is_keyed' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSDHard.root_read_on_row_is_keyed
/-- info: 'BSDHard.the_lock' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.the_lock
/-- info: 'BSDHard.indicator_pos' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.indicator_pos
/-- info: 'BSDHard.root_satisfiable' does not depend on any axioms -/
```

```
#guard_msgs in #print axioms BSDHard.root_satisfiable
/-- info: 'BSDHard.constructed_root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.constructed_root_holds
/-- info: 'BSDHard.root_on_row_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.root_on_row_is_the_act
/-- info: 'BSDHard.root_on_row_is_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.root_on_row_is_the_value
/-- info: 'BSDHard.root_on_row_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.root_on_row_is_keyed
/-- info: 'BSDHard.act_is_the_weakest_premise' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.act_is_the_weakest_premise
/-- info: 'BSDHard.existence_as_given_in_both_worlds' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.existence_as_given_in_both_worlds
/-- info: 'BSDHard.nothing_given_refutes' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.nothing_given_refutes
/-- info: 'BSDHard.only_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.only_refuter
/-- info: 'BSDHard.full_act_is_the_full_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.full_act_is_the_full_value
/-- info: 'BSDHard.full_act_gives_rank_act' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.full_act_gives_rank_act
/-- info: 'BSDHard.full_act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.full_act_is_keyed
/-- info: 'BSDHard.root_on_full_row_is_the_full_act' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.root_on_full_row_is_the_full_act
/-- info: 'BSDHard.root_on_full_row_is_the_full_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.root_on_full_row_is_the_full_value
/-- info: 'BSDHard.bsd_full_from_existence' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.bsd_full_from_existence
/-- info: 'BSDHard.full_supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.full_supply_iff
/-- info: 'BSDHard.full_act_is_the_weakest_premise' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.full_act_is_the_weakest_premise
/-- info: 'BSDHard.coefficient_refutes' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.coefficient_refutes
/-- info: 'BSDHard.len_app' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.len_app
/-- info: 'BSDHard.add_congr' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.add_congr
/-- info: 'BSDHard.len_map' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.len_map
/-- info: 'BSDHard.combos_length' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.combos_length
/-- info: 'BSDHard.points_length' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.points_length
/-- info: 'BSDHard.two_pow_le' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.two_pow_le
/-- info: 'BSDHard.fewer_classes_fall_short' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.fewer_classes_fall_short
/-- info: 'BSDHard.independence_discriminates' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.independence_discriminates
/-- info: 'BSDHard.two_pow_succ_ne_one' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.two_pow_succ_ne_one
/-- info: 'BSDHard.two_pow_inj' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.two_pow_inj
/-- info: 'BSDHard.rank_is_the_dimension_count' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.rank_is_the_dimension_count
/-- info: 'BSDHard.nothing_given_forces' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.nothing_given_forces
/-- info: 'BSDHard.sign_fixes_curve_parity' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.sign_fixes_curve_parity
/-- info: 'BSDHard.low_witness_not_full' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.low_witness_not_full
/-- info: 'BSDHard.full_act_strictly_stronger' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.full_act_strictly_stronger
/-- info: 'BSDHard.act_does_not_give_full_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.act_does_not_give_full_value
/-- info: 'BSDHard.record_wall_on_frames' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.record_wall_on_frames
/-- info: 'BSDHard.the_bit_is_the_world' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.the_bit_is_the_world
/-- info: 'BSDHard.finite_record_never_forces_above_one' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.finite_record_never_forces_above_one
/-- info: 'BSDHard.span_and_lock_are_dual' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.span_and_lock_are_dual
/-- info: 'BSDHard.row_root_grounds_itself_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.row_root_grounds_itself_iff_value
/-- info: 'BSDHard.full_row_root_grounds_itself_iff_full_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.full_row_root_grounds_itself_iff_full_value
/-- info: 'BSDHard.constructed_root_by_its_deed' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.constructed_root_by_its_deed
/-- info: 'BSDHard.RA₀_is_the_constructed_root' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.RA₀_is_the_constructed_root
/-- info: 'BSDHard.the_lock_on_the_root' does not depend on any axioms -/
#guard_msgs in #print axioms BSDHard.the_lock_on_the_root
/-! ============================================================================
    ================================================
    PART A · THE NORMAL FORM OF THE ROW (namespace BSD.NORMALFORM) ·
      v1_1_0_CC

    Purely additive: nothing above is overwritten. The row is read as the
      P ≠ NP seat reads computation and the
    Riemann seat reads the zeros: a type of frames, the value of a frame,
      and the registration of a frame onto its
    record, which keeps every existent and forgets the side of its
      realization. On the record the value holds with no
    premise (`record_has_the_value`). The value of a frame is then
      exactly one implication, the transfer of the value
    from the record to the frame (`normal_form`): the antecedent is a
      theorem, so the implication carries the whole
    content of the value, and it is the one bit. The two coherent frames
      share a record and split on the value
    (`record_decides_nothing`); the bit is keyed (`bit_is_keyed`); what
      every frame shares forces nothing
    (`keyless_forces_nothing`); every route to the value passes the bit
      (`every_route_passes_the_bit`); and the bit has
    no uniform answer across the frames (`no_uniform_answer`). Core Lean
      4.19.0, no import, no axiom declared, no sorry,
    no native_decide; every theorem of this Part on no axiom at all. No
      declaration of this Part has the type of the
    bit. ΔM = 0.
================================================================================
    ============================================
    -/
namespace BSD.NORMALFORM

universe u

/-- A ROW: a type of frames, the value of a frame, the registration of a
      frame onto its record, and two coherent
    frames, one carrying the value and one failing it. The record of
      every frame carries the value. -/
structure Row where
  Frame            : Type u
  Value            : Frame → Prop
  Rec              : Frame → Frame
  record_has_value : ∀ F, Value (Rec F)
  calm             : Frame
  counter          : Frame
  calm_value       : Value calm
  counter_fails    : ¬ Value counter

section core
variable (R : Row.{u})

/-- THE TRANSFER, the bit of a frame: the value carried from the frame's
      record to the frame. -/
def Transfer (F : R.Frame) : Prop := R.Value (R.Rec F) → R.Value F

/-- THE RECORD HAS THE VALUE, UNCONDITIONALLY (the mirror of
      `record_has_least_escape` and of
    `record_has_least_erasure`). -/
theorem record_has_the_value (F : R.Frame) : R.Value (R.Rec F) :=
    R.record_has_value F

/-- THE NORMAL FORM OF THE ROW (the mirror of `sep_normal_form` and of
      `zeta_ra_loop`, first leg): the value of a frame
    is exactly the transfer from its record. The antecedent is a theorem,
      so the implication carries the whole value:
    it is the one bit, and nothing else remains. -/
theorem normal_form (F : R.Frame) : Transfer R F ↔ R.Value F :=
  (fun h => h (R.record_has_value F), fun h _ => h)

/-- THE CARGO LOOP: the record's value, unconditional, carried by the one
      bit, is the value (the mirror of
    `sep_from_the_cargo_loop` and of `rh_from_the_cargo_loop`; the bit is
      the printed premise). -/
theorem value_from_the_cargo_loop (F : R.Frame) (bit : Transfer R F) :
    R.Value F :=
  bit (R.record_has_value F)

/-- THE ACT ON A FRAME: a structure whose one field is the value of the
      frame. -/
structure Act (F : R.Frame) where
  closure : R.Value F

/-- THE BIT IS THE ACT: the transfer holds exactly when the act stands. -/
theorem bit_is_the_act (F : R.Frame) : Transfer R F ↔ Nonempty (Act R F)
    :=
  (fun h => ⟨(h (R.record_has_value F))⟩, fun ⟨A⟩ _ => A.closure)

/-- SELF-RECURSION: a frame coincides with its record exactly when they
      carry the value alike. -/
def SelfRecursive (F : R.Frame) : Prop := R.Value F ↔ R.Value (R.Rec F)

/-- THE RECORD IS SELF-RECURSIVE, with no premise (the mirror of
      `record_is_self_recursive`). -/
theorem record_is_self_recursive (F : R.Frame) : SelfRecursive R (R.Rec
    F) :=
  (fun _ => R.record_has_value (R.Rec F), fun _ => R.record_has_value F)

/-- THE FRAME IS SELF-RECURSIVE EXACTLY WHEN IT CARRIES THE VALUE (the
      mirror of `self_recursion_at_machine` and of
    `self_recursion_at_zeros`). -/
theorem self_recursion_at_frame (F : R.Frame) : SelfRecursive R F ↔
    R.Value F :=
```

```
  (fun h => h.mpr (R.record_has_value F), fun h => (fun _ =>
    R.record_has_value F, fun _ => h))

/-- THE BIT IS KEYED: the transfer holds on the calm frame and fails on
    the counter frame. -/
theorem bit_is_keyed : Transfer R R.calm ∧ ¬ Transfer R R.counter :=
  (fun _ => R.calm_value, fun h => R.counter_fails (h (R.record_has_value
    R.counter)))

/-- THE RECORD'S SELF-RECURSION DOES NOT TRANSFER: the record of the
    counter frame is self-recursive and the counter
    frame is not. -/
theorem record_self_recursion_does_not_transfer :
    SelfRecursive R (R.Rec R.counter) ∧ ¬ SelfRecursive R R.counter :=
  (record_is_self_recursive R R.counter, fun h => R.counter_fails (h.mpr
    (R.record_has_value R.counter)))

/-- THE RECORD DECIDES NOTHING (the mirror of `record_decides_nothing`):
    the two coherent frames both carry the value
    on their records, and split on the value itself. -/
theorem record_decides_nothing :
    R.Value (R.Rec R.calm) ∧ R.Value (R.Rec R.counter) ∧ R.Value R.calm ∧
      ¬ R.Value R.counter :=
  (R.record_has_value R.calm, R.record_has_value R.counter, R.calm_value,
    R.counter_fails)

/-- EVERY ROUTE PASSES THE BIT (the mirror of
    `every_route_passes_the_bit`): any premise that yields the value
    yields
    the transfer, so every proof of the value, from any premise and in
      any vocabulary, delivers the one implication. -/
theorem every_route_passes_the_bit (F : R.Frame) (Q : Prop) (h : Q →
    R.Value F) : Q → Transfer R F :=
  fun q _ => h q

/-- A premise shared by every frame. -/
def Keyless (P : R.Frame → Prop) : Prop := ∀ F, P F

/-- A premise that forces the value on every frame where it holds. -/
def Forces (P : R.Frame → Prop) : Prop := ∀ F, P F → R.Value F

/-- A KEYLESS PREMISE FORCES NOTHING (the mirror of
    `the_unconditional_route_is_blocked`): what every frame shares
    yields the value on none where it fails. -/
theorem keyless_forces_nothing (P : R.Frame → Prop) (hP : Keyless R P) :
    ¬ Forces R P :=
  fun h => R.counter_fails (h R.counter (hP R.counter))

/-- NO CLOSED PRINCIPLE DECIDES: a principle true of the record of every
    frame decides the value on none (the mirror of
    `no_closed_principle_decides`). -/
theorem no_record_principle_decides (P : R.Frame → Prop) (hP : ∀ F, P
    (R.Rec F)) :
    ¬ ∀ F, P (R.Rec F) → R.Value F :=
  fun h => R.counter_fails (h R.counter (hP R.counter))

/-- A SHARED RECORD DECIDES NOTHING: a record of frames that reads the
    same on the calm and the counter frame has no
    reading that returns the value on every frame. -/
theorem shared_record_decides_nothing {α : Type} (r : R.Frame → α) (hr :
    r R.calm = r R.counter) :
    ¬ ∃ g : α → Bool, ∀ F, (g (r F) = true ↔ R.Value F) :=
  fun (g, hg) => R.counter_fails ((hg R.counter).mp ((congrArg g
    hr).symm.trans ((hg R.calm).mpr R.calm_value)))

/-- THE BIT HAS NO UNIFORM ANSWER: across the frames of the row the
    transfer is neither always true nor always false. -/
theorem no_uniform_answer : ¬ (∀ F, Transfer R F) ∧ ¬ (∀ F, ¬ Transfer R
    F) :=
  (fun h => (bit_is_keyed R).2 (h R.counter), fun h => h R.calm
    (bit_is_keyed R).1)

/-- "NOTHING IS OUTSIDE THE RECORD", read as the record's value
    transferring to the frame: false as a law of every
    frame, since the counter frame's record carries the value and the
      frame does not; at a frame, the value itself. -/
theorem nothing_outside_the_record_is_the_bit (F : R.Frame) :
    ¬ (∀ G, Transfer R G) ∧ (Transfer R F ↔ R.Value F) :=
  ((no_uniform_answer R).1, normal_form R F)

/-- THE ONE DENIAL: wherever the value fails, the transfer fails, though
    the record carries the value. -/
theorem denial_is_the_bits_denial (F : R.Frame) : ¬ R.Value F ↔ ¬
    Transfer R F :=
  (fun hn h => hn (h (R.record_has_value F)), fun hn hv => hn (fun _ =>
    hv))

/-- THE NORMAL FORM, COMPLETE, nothing before the colon but the row and a
    frame: the record carries the value; the
    value is the transfer; the transfer is the act; the frame is
      self-recursive exactly when it carries the value;
    every route passes the bit; the denial is the bit's denial. The value
      is not a conjunct. -/
theorem normal_form_complete (F : R.Frame) :
    R.Value (R.Rec F) ∧ (Transfer R F ↔ R.Value F) ∧ (Transfer R F ↔
        Nonempty (Act R F)) ∧
    (SelfRecursive R F ↔ R.Value F) ∧ (∀ Q : Prop, (Q → R.Value F) → Q →
      Transfer R F) ∧
    (¬ R.Value F ↔ ¬ Transfer R F) :=
  (R.record_has_value F, normal_form R F, bit_is_the_act R F,
    self_recursion_at_frame R F,
   fun Q h => every_route_passes_the_bit R F Q h,
    denial_is_the_bits_denial R F)

end core

/-! ### the refusals and the outside, generic over a proposition -/

/-- A true Boolean is never its own negation. -/
theorem bnot_ne (b : Bool) : (!b) ≠ b := by cases b <;> decide

/-- REFUSAL · "A PROOF IS WAITING" fails SREP: a true route to B is B
    itself (carried from `PNP.GATES.refusal_waiting`). -/
theorem refusal_waiting (B : Prop) : (∃ Q : Prop, Q ∧ (Q → B)) ↔ B :=
  (fun (_, q, h) => h q, fun b => (B, b, id))

/-- REFUSAL · "NO PROOF CAN EVER EXIST" fails SREP: no true route to B is
    ¬ B itself, the counter world in costume
    (carried from `PNP.GATES.refusal_no_proof`). -/
theorem refusal_no_proof (B : Prop) : (¬ ∃ Q : Prop, Q ∧ (Q → B)) ↔ ¬ B :=
  (fun h b => h (B, b, id), fun hn (_, q, h) => hn (h q))

/-- A BLOCK NEEDS AN OUTSIDE: if the target is a function of the record,
    no record-preserving involution flips it, so
    a block that flips the target always stands on something the record
      leaves out (carried from
    `PNP.ESCAPE.block_needs_an_outside`). -/
theorem block_needs_an_outside {X Y : Type} (ρ : X → Y) (t : X → Bool)
    (x₀ : X)
    (hrec : ∃ g : Y → Bool, ∀ x, t x = g (ρ x)) :
    ¬ ∃ τ : X → X, (∀ x, ρ (τ x) = ρ x) ∧ (∀ x, t (τ x) = !t x) :=
  fun (τ, hρ, ht) => match hrec with
  | (g, hg) => bnot_ne (t x₀) (by rw [← ht x₀, hg (τ x₀), hρ x₀, ← hg x₀])

/-- The native involution on the bit: complementation. -/
def comp (b : Bool) : Bool := !b

/-- Complementation is an involution with no fixed point. -/
theorem comp_involution (b : Bool) : comp (comp b) = b := by cases b <;>
    rfl

/-- THE LOCUS OF COMPLEMENTATION VANISHES: no bit is its own complement.
    -/
theorem locus_vanishes : ¬ ∃ b, comp b = b := fun (b, h) => bnot_ne b h

/-- The grades a face carries, ranked. -/
inductive Grade
  | premise
  | structural
  | thm
  deriving DecidableEq

/-- The rank of a grade. -/
def Grade.rank : Grade → Nat
  | .premise => 0
  | .structural => 1
  | .thm => 2

/-- The weakest of two grades. -/
def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a else
    b

/-- A JOIN NEVER STANDS ABOVE ITS WEAKEST FACE, and a tuple of theorems
    joined with the premise reads premise. -/
theorem join_reads_the_weakest :
    (∀ a b : Grade, (Grade.weakest a b).rank ≤ a.rank ∧ (Grade.weakest a
      b).rank ≤ b.rank) ∧
    Grade.weakest (Grade.weakest .thm .thm) .premise = .premise :=
  (fun a b => by
    cases a <;> cases b <;> exact (by decide, by decide),
   by decide)

/-- A SQUARED SCALAR IS BLIND TO SIGN: a value and its reflection square
    alike, read from the definitions of
    negation and multiplication on the integers. -/
theorem square_is_sign_blind : ∀ x : Int, (-x) * (-x) = x * x
  | Int.ofNat 0 => rfl
  | Int.ofNat (_ + 1) => rfl
  | Int.negSucc _ => rfl

end BSD.NORMALFORM

/-! ================================================================
    ================================================
    PART C · THE ROW SPLIT (namespace BSD.ROWSPLIT) · v1_1_0_CC

    The cut-agnostic division, carried verbatim from the Riemann master
      v3.0.39_CC (namespace RHUnicorn, generic over
    the types α and β), its one proposition renamed from `RH` to `Whole`,
      the hypothesis it names. At every cut T the
    whole is exactly its Real part and its Unicorn part; a certificate at
      T proves the Real part; given the Real part
    the whole is exactly the Unicorn part; above every cut no reading of
      an even record agrees with an odd decision;
    the aperture is one bit wide; and the supply side is silent at the
      register. ΔM = 0.
================================================================
    ================================================
    -/
namespace BSD.ROWSPLIT

variable {α β : Type}

/-- The whole, the Real part at T, the Unicorn part at T. -/
def Whole (Z onL : α → Prop) : Prop := ∀ z, Z z → onL z
def RealPartAt (Z onL : α → Prop) (height : α → Nat) (T : Nat) : Prop :=
  ∀ z, Z z → height z ≤ T → onL z
def UnicornAt (Z onL : α → Prop) (height : α → Nat) (T : Nat) : Prop :=
  ∀ z, Z z → height z > T → onL z

/-- THE FORMAL BLOCK OF THE UNICORN PART. Let τ act on the existents, let
    ρ be a record even at a seat z above the
```

```
    height, and let d decide the property, odd at z. Then no reading g of
       the record agrees with d even on the region
    above the height alone. -/
theorem unicorn_block (height : α → Nat) (T : Nat) (τ : α → α) (ρ : α →
    β) (d : α → Bool)
    (z : α) (habove : height z > T) (habove' : height (τ z) > T)
    (hρ : ρ (τ z) = ρ z) (hd : d (τ z) = !d z) :
    ¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = d w := by
  intro (g, hg)
  have h1 := hg z habove
  have h2 := hg (τ z) habove'
  rw [hρ, hd, h1] at h2
  revert h2
  cases d z <;> intro h2 <;> exact Bool.noConfusion h2

/-- THE APERTURE, ONE BIT WIDE. At a seat where d is odd, one odd bit s
    at the seat decides d on the seat and its
    partner through a calibration that exists and is unique. -/
theorem aperture_one_bit_wide (τ : α → α) (s d : α → Bool) (z : α)
    (hs : s (τ z) = !s z) (hd : d (τ z) = !d z) :
    ∃ c : Bool, (d z = xor (s z) c ∧ d (τ z) = xor (s (τ z)) c) ∧
      ∀ c' : Bool, (d z = xor (s z) c' ∧ d (τ z) = xor (s (τ z)) c') → c'
         = c :=
  (xor (d z) (s z),
    (by cases s z <;> cases d z <;> rfl,
     by rw [hs, hd]; cases s z <;> cases d z <;> rfl),
    by
      intro c' hc
      obtain (h1, -) := hc
      generalize hsz : s z = sv
      generalize hdz : d z = dv
      rw [hsz, hdz] at h1
      cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd
         h1 (by decide))

/-- EXISTENCE SUPPLIES NOTHING: conditioning the Unicorn part on an
    inhabited premise leaves it where it was. -/
theorem unicorn_rule_exact (Z onL : α → Prop) (height : α → Nat) (T :
    Nat) (A : Prop) (ha : A) :
    (A → UnicornAt Z onL height T) ↔ UnicornAt Z onL height T :=
  (fun h => h ha, fun hu _ => hu)

/-- NO BYPASS: given the Real part, the whole is exactly the Unicorn
    part. -/
theorem unicorn_no_bypass (Z onL : α → Prop) (height : α → Nat) (T : Nat)
    (hr : RealPartAt Z onL height T) : Whole Z onL ↔ UnicornAt Z onL
       height T := by
  constructor
  · intro h z hz _; exact h z hz
  · intro hu z hz
    cases Nat.lt_or_ge T (height z) with
    | inl hgt => exact hu z hz hgt
    | inr hle => exact hr z hz hle

/-- THE SUPPLY SIDE IS SILENCE, AT THE REGISTER: both orientations of d
    at the seat are equally refused to every
    reading above the height. -/
theorem supply_side_silent (height : α → Nat) (T : Nat) (τ : α → α) (ρ :
    α → β) (d : α → Bool)
    (z : α) (habove : height z > T) (habove' : height (τ z) > T)
    (hρ : ρ (τ z) = ρ z) (hd : d (τ z) = !d z) :
    (¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = d w) ∧
    (¬ ∃ g : β → Bool, ∀ w, height w > T → g (ρ w) = !d w) ∧
    (!d z) ≠ d z :=
  (unicorn_block height T τ ρ d z habove habove' hρ hd,
   unicorn_block height T τ ρ (fun w => !d w) z habove habove' hρ (by
     show (!d (τ z)) = !(!d z)
     rw [hd]),
   by cases d z <;> decide)

/-- The division at the height: the whole is exactly its Real part and
    its Unicorn part. -/
theorem whole_iff_real_and_unicorn (Z onL : α → Prop) (height : α → Nat)
    (T : Nat) :
    Whole Z onL ↔ RealPartAt Z onL height T ∧ UnicornAt Z onL height T :=
       by
  constructor
  · intro h; exact ⟨fun z hz _ => h z hz, fun z hz _ => h z hz⟩
  · intro ⟨hr, hu⟩ z hz
    cases Nat.lt_or_ge T (height z) with
    | inl hgt => exact hu z hz hgt
    | inr hle => exact hr z hz hle

/-- A certificate at the height: a complete list of the existents up to
    T, each checked. -/
structure Certificate (Z onL : α → Prop) (height : α → Nat) (T : Nat)
    where
  items    : List α
  complete : ∀ z, Z z → height z ≤ T → z ∈ items
  checked  : ∀ z, z ∈ items → onL z

/-- A CERTIFICATE PROVES THE REAL PART AT ITS HEIGHT. -/
theorem real_part_of_certificate (Z onL : α → Prop) (height : α → Nat) (T
    : Nat)
    (c : Certificate Z onL height T) : RealPartAt Z onL height T :=
  fun z hz hle => c.checked z (c.complete z hz hle)

end BSD.ROWSPLIT
/-! ============================================================================
    ============================================
    PART B · THE HOST MODEL AND ITS BINDING, PROVED (namespace BSD.HOST)
       · v1_1_0_CC

    The record of a frame keeps every curve, its order and its
       coefficient bit, and registers the curve's rank at its
    order: on the record the rank sits on the seat, so the record carries
       the value with no premise
    (`recFrame_value`). The host is the row's curves, each with its rank
       and its centred expansion at the centre,
    carrying the sign of its functional equation, the order of vanishing
       read off the expansion; its sentence is rank
    equal to order at every curve. The binding is a theorem: the value of
       the host's frame is exactly the host's
    sentence (`binding`). The order is read off the expansion uniquely
       (`order_is_read`); the sign fixes the parity of
    the order at every curve (`sign_fixes_order_parity`), a minus sign
       forcing the central value to vanish
    (`minus_sign_vanishes`); and under the sentence the sign fixes the
       parity of the rank
    (`sentence_gives_rank_parity`), a minus sign forcing positive rank
       (`minus_sign_forces_positive_rank`). A curve of
    every order is built on the seat (`monoCentred`). Every theorem on no
       axiom at all. ΔM = 0.
============================================================================
    ============================================
    -/
namespace BSD.HOST
open BSDHard

/-- THE RECORD OF A FRAME: the registration keeps every curve, its order
    and its coefficient bit, and registers the
    curve's rank at its order. -/
def recFrame (F : Frame) : Frame := (F.D, F.order, F.order, F.coeff)

/-- THE RECORD CARRIES THE VALUE, with no premise. -/
theorem recFrame_value (F : Frame) : Value (recFrame F) := fun _ => rfl

/-- THE BSD ROW: the frames of curves with their ranks and orders, the
    value, the registration onto the record, the
    calm world and the counter world. -/
def bsdRow : BSD.NORMALFORM.Row.{1} :=
  (Frame, Value, recFrame, recFrame_value, calm, counter, calm_value,
    counter_fails)

/-- EVERY CURVE HAS ITS SEAT ON THE RECORD: on the record each curve's
    rank is read at its order, and its order and
    coefficient bit are kept. -/
theorem every_curve_has_its_seat (F : Frame) (d : F.D) :
    (recFrame F).rank d = F.order d ∧ (recFrame F).order d = F.order d ∧
      (recFrame F).coeff d = F.coeff d :=
  (rfl, rfl, rfl)

/-- THE HOST: the curves of the row, each with its rank, its centred
    expansion at the centre carrying the sign of its
    functional equation, and its order of vanishing, read off the
       expansion. -/
structure Host where
  Curve : Type
  rank  : Curve → Nat
  L     : Curve → Centred
  order : Curve → Nat
  ord   : ∀ c, IsOrder (L c).a (order c)

/-- THE SENTENCE OF THE HOST: at every curve the rank is the order of
    vanishing at the centre. -/
def Host.Sentence (H : Host) : Prop := ∀ c, H.rank c = H.order c

/-- THE FRAME OF THE HOST. -/
def Host.frame (H : Host) : Frame := (H.Curve, H.rank, H.order, fun _ =>
    true)

/-- THE SIGNED FRAME OF THE HOST: each curve with its centred expansion.
    -/
def Host.signed (H : Host) : SignedFrame := (H.frame, H.L, H.ord)

/-- THE BINDING, PROVED: the value of the host's frame is exactly the
    host's sentence. -/
theorem binding (H : Host) : Value H.frame ↔ H.Sentence := ⟨fun h c => h
    c, fun h c => h c⟩

/-- THE ROOT READ ON THE ROW IS THE SENTENCE, on the host's frame. -/
theorem root_on_row_is_the_sentence (H : Host) : RootOnRow H.frame ↔
    H.Sentence :=
  (root_on_row_is_the_value H.frame).trans (binding H)

/-- THE ACT ON THE HOST'S FRAME IS THE SENTENCE. -/
theorem act_is_the_sentence (H : Host) : Registered H.frame ↔ H.Sentence
    :=
  (act_is_the_value H.frame).trans (binding H)

/-- THE ORDER IS READ OFF THE EXPANSION, uniquely. -/
theorem order_is_read (H : Host) (c : H.Curve) (o : Nat) (ho : IsOrder
    (H.L c).a o) : o = H.order c :=
  order_unique (H.L c).a o (H.order c) ho (H.ord c)

/-- THE SIGN FIXES THE PARITY OF THE ORDER at every curve of the host:
    under a minus sign the order is odd, under a plus
    sign even. -/
theorem sign_fixes_order_parity (H : Host) (c : H.Curve) :
    ((H.L c).minus = true → evenB (H.order c) = false) ∧ ((H.L c).minus =
       false → evenB (H.order c) = true) :=
  (fun hm => minus_order_odd (H.L c) hm (H.order c) (H.ord c),
   fun hm => plus_order_even (H.L c) hm (H.order c) (H.ord c))

/-- A MINUS SIGN FORCES THE CENTRAL VALUE TO VANISH, at every curve of
    the host. -/
theorem minus_sign_vanishes (H : Host) (c : H.Curve) (hm : (H.L c).minus
    = true) : (H.L c).a 0 = 0 :=
  minus_sign_forces_vanishing (H.L c) hm

/-- UNDER THE SENTENCE THE SIGN FIXES THE PARITY OF THE RANK. -/
```

```
theorem sentence_gives_rank_parity (H : Host) (h : H.Sentence) (c : H.Curve) :
    ((H.L c).minus = true → evenB (H.rank c) = false) ∧ ((H.L c).minus = false → evenB (H.rank c) = true) :=
  (fun hm => by rw [h c]; exact (sign_fixes_order_parity H c).1 hm,
   fun hm => by rw [h c]; exact (sign_fixes_order_parity H c).2 hm)

/-- UNDER THE SENTENCE A MINUS SIGN FORCES POSITIVE RANK. -/
theorem minus_sign_forces_positive_rank (H : Host) (h : H.Sentence) (c : H.Curve) (hm : (H.L c).minus = true) :
    H.rank c ≠ 0 :=
  fun h0 => Bool.noConfusion (((sentence_gives_rank_parity H h c).1 hm).symm.trans (congrArg evenB h0))

/-- The expansion with one nonzero coefficient, one, at index o. -/
def mono (o : Nat) : Nat → Int := fun k => if k = o then 1 else 0

/-- Off its index the expansion vanishes. -/
theorem mono_off (o k : Nat) (h : k ≠ o) : mono o k = 0 := if_neg h

/-- A Boolean never equals its negation. -/
theorem beq_not_self (b : Bool) : (b == !b) = false := by cases b <;> rfl

/-- The expansion with one coefficient at o obeys the reflection law of the sign fixed by the parity of o. -/
theorem mono_reflect (o : Nat) : ∀ k, (evenB k == !evenB o) = true → mono o k = -(mono o k) :=
  fun k hk => match Nat.decEq k o with
    | isTrue e =>
      have h2 : (evenB o == !evenB o) = true := e ▸ hk
      Bool.noConfusion (h2.symm.trans (beq_not_self (evenB o)))
    | isFalse ne =>
      (mono_off o k ne).trans ((rfl : (0 : Int) = -0).trans (congrArg Neg.neg (mono_off o k ne).symm))

/-- A CENTRED EXPANSION OF EVERY ORDER: one coefficient at o, the sign minus exactly when o is odd. -/
def monoCentred (o : Nat) : Centred := ⟨mono o, !evenB o, mono_reflect o⟩

/-- Its order is o. -/
theorem mono_is_order (o : Nat) : IsOrder (monoCentred o).a o :=
  ⟨fun h => absurd ((if_pos rfl : mono o o = 1).symm.trans h) (by decide),
   fun j hj => mono_off o j (Nat.ne_of_lt hj)⟩

/-- The host of one curve of rank r whose expansion has order o. -/
def oneCurve (r o : Nat) : Host := ⟨Unit, fun _ => r, fun _ => monoCentred o, fun _ => o, fun _ => mono_is_order o⟩

/-- THE CURVE OF RANK ONE AT ORDER ONE carries a minus sign and the sentence: the shape of 37a1. -/
theorem rank_one_at_order_one : (oneCurve 1 1).Sentence ∧ ((oneCurve 1 1).L ()).minus = true := ⟨fun _ => rfl, rfl⟩

/-- THE LOW-ORDER WITNESS ON THE HOST: the cited theorems at order at most one have this shape. -/
def Host.Low (H : Host) : Prop := LowWitness H.frame

/-- THE SENTENCE GIVES THE LOW-ORDER WITNESS, and the low-order witness does not give the sentence: a curve of rank
    zero at order two carries the witness and fails the sentence. -/
theorem low_is_strictly_inside :
    (∀ H : Host, H.Sentence → H.Low) ∧ (oneCurve 0 2).Low ∧ ¬ (oneCurve 0 2).Sentence :=
  ⟨fun _ h c _ => h c, fun _ h => absurd h (by decide : ¬ ((2 : Nat) ≤ 1)), fun h => Nat.noConfusion (h ())⟩

end BSD.HOST

/-! ============================================================================================
    PART C′ · THE ROW SPLIT READ AT ORDER ONE (namespace BSD.REALPART) · v1_1_0_CC

    The cut-agnostic division of Part C read on the BSD row: the existents are a frame's curves, the property each is
    asked is rank equal to order, and the height is the order. At order one the real part is exactly the low-order
    witness (`low_witness_is_the_real_part`); the value is exactly its real part and its unicorn part at every cut;
    given the low-order witness the value is exactly the unicorn part above order one (`no_bypass_at_order_one`); the
    unicorn part is load-bearing, the staged world above order one carrying the real part and failing the value; and
    above every order no reading of a record that keeps the order and forgets the world returns the world. Every
    theorem on no axiom at all. ΔM = 0.
================================================================================================ -/
namespace BSD.REALPART
open BSDHard BSD.ROWSPLIT

/-- The existents: every curve of the frame. -/
def Ex (F : Frame) (_ : F.D) : Prop := True

/-- The property each curve is asked: rank equal to order. -/
def Rk (F : Frame) (d : F.D) : Prop := F.rank d = F.order d

/-- The value of the frame is the whole of the row split. -/
theorem value_is_the_whole (F : Frame) : Value F ↔ Whole (Ex F) (Rk F) :=
  ⟨fun h d _ => h d, fun h d => h d trivial⟩

/-- THE REAL PART AT ORDER ONE IS THE LOW-ORDER WITNESS. -/
theorem low_witness_is_the_real_part (F : Frame) : LowWitness F ↔ RealPartAt (Ex F) (Rk F) F.order 1 :=
  ⟨fun h d _ hle => h d hle, fun h d hle => h d trivial hle⟩

/-- THE CUT AT EVERY ORDER: the value is exactly its real part and its unicorn part. -/
theorem value_iff_real_and_unicorn (F : Frame) (T : Nat) :
    Value F ↔ RealPartAt (Ex F) (Rk F) F.order T ∧ UnicornAt (Ex F) (Rk F) F.order T :=
  ⟨value_is_the_whole F).trans (whole_iff_real_and_unicorn (Ex F) (Rk F) F.order T)

/-- NO BYPASS AT ORDER ONE: given the low-order witness, the value is exactly the unicorn part above order one. -/
theorem no_bypass_at_order_one (F : Frame) (hw : LowWitness F) : Value F ↔ UnicornAt (Ex F) (Rk F) F.order 1 :=
  ⟨value_is_the_whole F).trans
    (unicorn_no_bypass (Ex F) (Rk F) F.order 1
      ((low_witness_is_the_real_part F).mp hw))

/-- THE UNICORN PART IS THE CURVES OF ORDER AT LEAST TWO. -/
theorem unicorn_is_order_two_and_above (F : Frame) :
    UnicornAt (Ex F) (Rk F) F.order 1 ↔ ∀ d, 2 ≤ F.order d → F.rank d = F.order d :=
  ⟨fun h d h2 => h d trivial h2, fun h d _ h2 => h d h2⟩

/-- THE UNICORN PART IS LOAD-BEARING: at every stage the staged world above order one carries the real part and fails
    the value. -/
theorem unicorn_part_is_load_bearing (n : Nat) :
    RealPartAt (Ex (stagedHigh n)) (Rk (stagedHigh n)) (stagedHigh n).order 1 ∧ ¬ Value (stagedHigh n) :=
  ⟨(low_witness_is_the_real_part (stagedHigh n)).mp
     (finite_record_never_forces_above_one n).1,
   (finite_record_never_forces_above_one n).2.2⟩

/-- THE UNICORN PART CLOSED TO THE RECORD ABOVE EVERY ORDER: on the chart of worlds and orders, a record that keeps the
    order and forgets the world is even under the exchange of worlds, the world is odd, and above every order no
    reading of the record returns the world. -/
theorem unicorn_blocked_above_every_order (T : Nat) :
    ¬ ∃ g : Nat → Bool, ∀ w : Bool × Nat, w.2 > T → g w.2 = w.1 :=
  unicorn_block Prod.snd T (fun w => (!w.1, w.2)) Prod.snd Prod.fst
    (true, T + 1)
    (Nat.lt_succ_self T) (Nat.lt_succ_self T) rfl rfl

/-- THE SUPPLY SIDE IS SILENT ABOVE EVERY ORDER: both orientations of the world are refused alike to every reading of
    the record, and the two orientations differ. -/
theorem supply_side_silent_above_every_order (T : Nat) :
    (¬ ∃ g : Nat → Bool, ∀ w : Bool × Nat, w.2 > T → g w.2 = w.1) ∧
    (¬ ∃ g : Nat → Bool, ∀ w : Bool × Nat, w.2 > T → g w.2 = !w.1) ∧ (!true) ≠ true :=
  ⟨supply_side_silent Prod.snd T (fun w => (!w.1, w.2)) Prod.snd Prod.fst
    (true, T + 1)
    (Nat.lt_succ_self T) (Nat.lt_succ_self T) rfl rfl

/-- THE APERTURE IS ONE BIT WIDE: at the seat of the exchange above every order, one odd supplied bit calibrates the
    world uniquely, through one calibration bit that exists and is unique. -/
theorem aperture_is_one_bit (T : Nat) :
    let z : Bool × Nat := (true, T + 1)
    let τ : Bool × Nat → Bool × Nat := fun w => (!w.1, w.2)
    ∃ c : Bool, (z.1 = xor (!z.1) c ∧ (τ z).1 = xor (!(τ z).1) c) ∧
      ∀ c' : Bool, (z.1 = xor (!z.1) c' ∧ (τ z).1 = xor (!(τ z).1) c') → c' = c :=
  aperture_one_bit_wide (fun w : Bool × Nat => (!w.1, w.2)) (fun w => !w.1) Prod.fst (true, T + 1) rfl rfl

end BSD.REALPART

/-! ============================================================================================
    PART D · THE WALLS (namespace BSD.WALLS) · v1_1_0_CC

    Every zero-bit route is closed by theorem. Existence as given forces nothing (Part 0, `given_is_not_the_value`,
    `arrow_forces_nothing`). No finite record forces the value, nor above order one with the low-order witness in place
    (Part 0). The parity is one bit and not the value (Part 0, `parity_is_not_the_value`). The retired Ω is strictly
    stronger than the value (Part 0). This Part adds the sign wall on the row's own expansions: the expansions of
    order one and order three carry the same sign, and the sign reads the parity of the order and never the order
    (`sign_record_normal_form`, `sign_reads_no_order`): the root number of a curve of rank one and of a curve of rank
    three is one and the same, minus. Every theorem on no axiom at all. ΔM = 0.
================================================================================================ -/
namespace BSD.WALLS
open BSDHard BSD.HOST

/-- THE SIGN RECORD NORMAL FORM, decided by the kernel on the row's own expansions: the expansions of order one and
    of order three both carry the minus sign; their first-order readings are one and zero; and the sign at every order
    is the parity of the order, flipped. -/
theorem sign_record_normal_form :
    (monoCentred 1).minus = true ∧ (monoCentred 3).minus = true ∧
    (monoCentred 1).a 1 = 1 ∧ (monoCentred 3).a 1 = 0 ∧ (∀ o, (monoCentred o).minus = !evenB o) :=
  ⟨rfl, rfl, rfl, rfl, fun _ => rfl⟩
```

```
/-- THE SIGN READS NO ORDER: no reading of the sign returns the order of
    the expansion. -/
theorem sign_reads_no_order : ¬ ∃ g : Bool → Nat, ∀ o : Nat, g
    (monoCentred o).minus = o :=
  fun (g, hg) => absurd ((hg 1).symm.trans (hg 3)) (by decide)

/-- THE SIGN READS NO RANK UNDER THE SENTENCE: the curves of rank one and
    rank three at their orders share the sign. -/
theorem sign_reads_no_rank :
    (oneCurve 1 1).Sentence ∧ (oneCurve 3 3).Sentence ∧ ((oneCurve 1 1).L
      ()).minus = ((oneCurve 3 3).L ()).minus ∧
    ¬ ∃ g : Bool → Nat, g ((oneCurve 1 1).L ()).minus = 1 ∧ g ((oneCurve
      3 3).L ()).minus = 3 :=
  (fun _ => rfl, fun _ => rfl, rfl, fun ⟨_, h1, h3⟩ => absurd
    (h1.symm.trans h3) (by decide))

/-- THE WALLS OF THE ROW, one theorem: no given statement is the value on
    every frame; the arrow forces nothing; no
    finite record forces the value, nor above order one with the
      low-order witness in place; the parity is one bit and
    not the value; the low-order witness is strictly smaller; the retired
      Ω is strictly stronger; no reading of the
    kinetic record returns the bit; and the sign reads no order. -/
theorem the_walls :
    (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
    (¬ ∀ F : Frame, Arrow → Value F) ∧
    (∀ n : Nat, (∀ d, d < n → (staged n).rank d = (staged n).order d) ∧ ¬
      Value (staged n)) ∧
    (∀ n : Nat, LowWitness (stagedHigh n) ∧ (∀ d, d < n → (stagedHigh
      n).rank d = (stagedHigh n).order d) ∧
      ¬ Value (stagedHigh n)) ∧
    (ParityValue counter ∧ ¬ Value counter) ∧
    (LowWitness counter ∧ ¬ Value counter) ∧
    (Value unprovenanced.F ∧ ¬ OmegaHolds unprovenanced) ∧
    (¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w) ∧
    (¬ ∃ g : Bool → Nat, ∀ o : Nat, g (monoCentred o).minus = o) :=
  ⟨given_is_not_the_value, arrow_forces_nothing,
    finite_record_never_forces, finite_record_never_forces_above_one,
   parity_is_not_the_value, low_witness_strict, omega_strictly_stronger,
     record_wall, sign_reads_no_order⟩

end BSD.WALLS

/-! ============================================================================
    ============================================
    PART E · THE RESIDUE (namespace BSD.RESIDUE) · v1_1_0_CC

    Everything above is unconditional. What remains of the rank part of
      the Birch and Swinnerton-Dyer sentence is this
    one hypothesis, the transfer of the value from the record to the
      host's frame: given it, the sentence follows on no
    axiom. It is printed here as the type of a hypothesis and is assumed
      nowhere. The residue is keyed: no premise
    shared by every frame and no principle true of every record supplies
      it. Its instance seat decomposes it at order
    one: given the low-order witness, the seat is the curves of order at
      least two. ΔM = 0.
============================================================================
    ============================================
    -/
namespace BSD.RESIDUE
open BSDHard BSD.NORMALFORM BSD.HOST BSD.REALPART BSD.ROWSPLIT

/-- THE RESIDUE: given the one implication at the host's frame, the
    host's sentence. -/
theorem sentence_of_the_act (H : Host) (bit : Transfer bsdRow H.frame) :
    H.Sentence :=
  (binding H).mp (value_from_the_cargo_loop bsdRow H.frame bit)

/-- THE RESIDUE IS THE SENTENCE: the transfer at the host's frame is
    exactly the host's sentence. -/
theorem residue_is_the_sentence (H : Host) : Transfer bsdRow H.frame ↔
    H.Sentence :=
  (normal_form bsdRow H.frame).trans (binding H)

/-- THE ONE POSIT IS THE BIT: the root read on the row, at the host's
    frame, is exactly the transfer. -/
theorem the_posit_is_the_bit (H : Host) : RootOnRow H.frame ↔ Transfer
    bsdRow H.frame :=
  (root_on_row_is_the_sentence H).trans (residue_is_the_sentence H).symm

/-- THE RESIDUE IS KEYED: no premise shared by every frame forces the
    value, and no principle true of every record
      decides it. What supplies it is keyed to the frame: the act. -/
theorem residue_is_keyed :
    (∀ P : Frame → Prop, Keyless bsdRow P → ¬ Forces bsdRow P) ∧
    (∀ P : Frame → Prop, (∀ F, P (recFrame F)) → ¬ ∀ F, P (recFrame F) →
      Value F) :=
  (keyless_forces_nothing bsdRow, no_record_principle_decides bsdRow)

/-- THE INSTANCE SEAT OF THE BIT, above order one: rank equal to order at
    every curve of order at least two. Given the
      low-order witness, its inhabitation is equivalent in strength to the
        value (`seat_iff_value`); nothing in this file
    assumes it inhabited. -/
structure HighSeat (F : Frame) where
  high : ∀ d, 2 ≤ F.order d → F.rank d = F.order d

/-- THE SEAT AND THE LOW-ORDER WITNESS DELIVER THE VALUE. -/
theorem value_of_seat (F : Frame) (hw : LowWitness F) (S : HighSeat F) :
    Value F :=
  fun d => match Nat.lt_or_ge 1 (F.order d) with
    | Or.inl h => S.high d h
    | Or.inr h => hw d h

/-- GIVEN THE LOW-ORDER WITNESS, THE SEAT IS INHABITED EXACTLY WHEN THE
    VALUE HOLDS. -/
theorem seat_iff_value (F : Frame) (hw : LowWitness F) : Nonempty
    (HighSeat F) ↔ Value F :=
  ⟨fun (S) => value_of_seat F hw S, fun h => ⟨fun d _ => h d⟩⟩

/-- THE SEAT IS THE UNICORN PART. -/
theorem seat_is_the_unicorn (F : Frame) : Nonempty (HighSeat F) ↔
    UnicornAt (Ex F) (Rk F) F.order 1 :=
  ⟨fun (S) => (unicorn_is_order_two_and_above F).mpr S.high,
   fun h => (((unicorn_is_order_two_and_above F).mp h))⟩

end BSD.RESIDUE

/-! ============================================================================
    ============================================
    PART F · THE SUPPLY (namespace BSD.SUPPLY) · v1_1_0_CC

    The bit of Part E, supplied by the act and delivered by its two
      kinds: the formal cargo, and the physical witness.
    Each kind is a type whose inhabitant is the supply; neither is
      assumed anywhere in this file. The cargo arrives
    free; each kind delivers the sentence; the two kinds are one bit;
      every route to the sentence delivers both kinds,
    so there is no third; and one curve whose rank is not its order
      refutes both at once. The witnesses of the subject
    are typed here against the bit: the cited theorems at order at most
      one are inside it and strictly smaller; the
    retired Ω is strictly stronger; the full act, rank and leading
      coefficient, is strictly stronger and carried beside.
    ΔM = 0.
============================================================================
    ============================================
    -/
namespace BSD.SUPPLY
open BSDHard BSD.NORMALFORM BSD.HOST

/-- THE FORMAL KIND, the cargo: the record's value carried to the host's
    frame. -/
def FormalCargo (H : Host) : Prop := Transfer bsdRow H.frame

/-- THE PHYSICAL KIND, the witness: each curve's group of rational points
    is a physical register whose free directions
    are each registered at one Landauer floor quantum, the curve itself
      at one more, so a curve's heat in floor units is
    its rank plus one; and the carrier registers every curve's free
      directions as the zero modes at the centre, rank
    equal to the order of vanishing. The last field is the one bit read
      in the register of heat; it is a premise of the
    witness, carried as a field, never proved. -/
structure PhysicalWitness (H : Host) where
  floor      : Nat
  floor_pos  : 0 < floor
  heat       : H.Curve → Nat
  counts     : ∀ c, heat c = floor * (H.rank c + 1)
  zero_modes : ∀ c, H.rank c = H.order c

/-- THE CARGO ARRIVES, with no premise: the record of the host's frame
    carries the value. -/
theorem cargo_arrives (H : Host) : Value (recFrame H.frame) :=
  recFrame_value H.frame

/-- KIND 1 DELIVERS: the formal cargo gives the sentence in one step. -/
theorem formal_kind_delivers {H : Host} (h : FormalCargo H) : H.Sentence
    := (binding H).mp (h (cargo_arrives H))

/-- KIND 2 DELIVERS: the physical witness gives the sentence. -/
theorem physical_kind_delivers {H : Host} (W : PhysicalWitness H) :
    H.Sentence := W.zero_modes

/-- The physical witness closes the host's frame. -/
theorem witness_closes {H : Host} (W : PhysicalWitness H) : Value H.frame
    := (binding H).mpr (physical_kind_delivers W)

/-- A witness from the sentence, at one floor quantum per free direction.
    -/
def witness_of_sentence {H : Host} (h : H.Sentence) : PhysicalWitness H :=
  ⟨1, Nat.zero_lt_one, fun c => 1 * (H.rank c + 1), fun _ => rfl, h⟩

/-- THE PHYSICAL FACE IS THE SENTENCE: a witness exists exactly when the
    sentence holds. -/
theorem physical_iff_sentence (H : Host) : Nonempty (PhysicalWitness H) ↔
    H.Sentence :=
  ⟨fun ⟨W⟩ => physical_kind_delivers W, fun h => ⟨witness_of_sentence h⟩⟩

/-- THE FORMAL FACE IS THE SENTENCE. -/
theorem formal_iff_sentence (H : Host) : FormalCargo H ↔ H.Sentence :=
  BSD.RESIDUE.residue_is_the_sentence H

/-- THE TWO KINDS ARE ONE BIT: the physical witness is a formal cargo,
    and the cargo is exactly the sentence. -/
theorem the_two_kinds_are_one_bit (H : Host) :
    (PhysicalWitness H → FormalCargo H) ∧ (FormalCargo H ↔ H.Sentence) :=
  ⟨fun W _ => witness_closes W, formal_iff_sentence H⟩

/-- THE SENTENCE, DELIVERED TWICE: either kind gives it. -/
theorem delivered_twice (H : Host) : (FormalCargo H → H.Sentence) ∧
    (PhysicalWitness H → H.Sentence) :=
  ⟨formal_kind_delivers, physical_kind_delivers⟩

/-- TWO KINDS AND NO THIRD: every witness of the sentence, in any
    vocabulary, is a delivery of both kinds. -/
theorem two_kinds_and_no_third (H : Host) (Q : Prop) (h : Q → H.Sentence)
    :
    Q → FormalCargo H ∧ Nonempty (PhysicalWitness H) :=
```

```
  fun q => ((formal_iff_sentence H).mpr (h q), (physical_iff_sentence
    H).mpr (h q))

/-- THE ONE REFUTER: one curve whose rank is not its order refutes both
    kinds at once. -/
theorem the_one_refuter (H : Host) (c : H.Curve) (h : H.rank c ≠ H.order
    c) :
  ¬ FormalCargo H ∧ ¬ Nonempty (PhysicalWitness H) :=
 have hs : ¬ H.Sentence := fun s => h (s c)
 (fun f => hs (formal_kind_delivers f), fun (W) => hs
    (physical_kind_delivers W))

/-- THE WITNESSES OF THE SUBJECT, TYPED: the sentence gives the low-order
    witness, which a curve of rank zero at order
    two carries while failing the sentence; the act supplies the
      low-order witness on every actual frame and the
    witness is strictly smaller than the value; the retired Ω gives the
      value and is strictly stronger; the full act
    carries the act, is strictly stronger, and the act does not give the
      full value. Each is typed against the bit;
    none is its ground. -/
theorem the_witnesses_typed :
  ((∀ H : Host, H.Sentence → H.Low) ∧ (oneCurve 0 2).Low ∧ ¬ (oneCurve
    0 2).Sentence) ∧
  (∀ A : ActualCurves, LowWitness A.F) ∧ (LowWitness counter ∧ ¬ Value
    counter) ∧
  ((∀ P : Provenanced, OmegaHolds P → Value P.F) ∧ Value
    unprovenanced.F ∧ ¬ OmegaHolds unprovenanced) ∧
  ((∀ F : Frame, RegisteredFull F → Registered F) ∧ Registered
    coeffCounter ∧ ¬ RegisteredFull coeffCounter) ∧
  (¬ ∀ F : Frame, Registered F → FullValue F) :=
 (low_is_strictly_inside, act_gives_low_witness, low_witness_strict,
  (omega_gives_value, omega_strictly_stronger),
    full_act_strictly_stronger, act_does_not_give_full_value)

/-- THE SUPPLY, CLOSED, at every host: the cargo arrives free; each kind
    delivers; the two kinds are one bit; the bit is
    the sentence; every route is both kinds; and one curve whose rank is
      not its order refutes both. -/
theorem the_supply_closed : ∀ H : Host,
  Value (recFrame H.frame) ∧
  (FormalCargo H → H.Sentence) ∧ (PhysicalWitness H → H.Sentence) ∧
  (PhysicalWitness H → FormalCargo H) ∧ (FormalCargo H ↔ H.Sentence) ∧
  (Nonempty (PhysicalWitness H) ↔ H.Sentence) ∧
  (∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H ∧ Nonempty
    (PhysicalWitness H)) ∧
  (∀ c, H.rank c ≠ H.order c → ¬ FormalCargo H ∧ ¬ Nonempty
    (PhysicalWitness H)) :=
 fun H => (cargo_arrives H, formal_kind_delivers,
   physical_kind_delivers, (the_two_kinds_are_one_bit H).1,
  formal_iff_sentence H, physical_iff_sentence H, two_kinds_and_no_third
    H, the_one_refuter H)

end BSD.SUPPLY

/-! ============================================================================
    ============================================
  PART G · THE CLOSURE OF THE QUESTION (namespace BSD.CLOSURE) ·
    v1_1_0_CC

  Why the question is closed and not open. At every host the rank
    sentence and the bit are one proposition, with
  nothing cited; the denial of the sentence is the bit's denial; across
    the frames of the row the bit has no uniform
  answer, true in one world and false in another of the same record;
    every keyless route is barred; either kind of
  the supply gives the sentence; and one curve whose rank is not its
    order refutes the sentence itself. Nothing here
  derives the bit. It proves that the question is one bit, that the bit
    is keyed, and that what is open is a value
  and not a form. ΔM = 0.
================================================================================
    ============================================
  -/
namespace BSD.CLOSURE
open BSDHard BSD.NORMALFORM BSD.HOST BSD.SUPPLY

/-- THE SENTENCE IS THE BIT, nothing cited. -/
theorem the_sentence_is_the_bit (H : Host) : H.Sentence ↔ FormalCargo H
    := (formal_iff_sentence H).symm

/-- THE DENIAL IS THE BIT'S DENIAL. -/
theorem denial_is_the_bits_denial (H : Host) : ¬ H.Sentence ↔ ¬
    FormalCargo H :=
 (fun hn h => hn (formal_kind_delivers h), fun hn h => hn
    ((formal_iff_sentence H).mpr h))

/-- THE ONE REFUTER REFUTES THE SENTENCE: one curve whose rank is not its
    order. -/
theorem refuter_denies_the_sentence (H : Host) (c : H.Curve) (h : H.rank
    c ≠ H.order c) : ¬ H.Sentence :=
 fun s => h (s c)

/-- THE DENIAL HAS ONE ROAD: wherever the sentence fails, it cannot be
    that no curve has rank other than its order. -/
theorem the_denial_has_one_road (H : Host) : ¬ H.Sentence → ¬ ¬ ∃ c,
    H.rank c ≠ H.order c :=
 fun hn hne => hn (fun c => match Nat.decEq (H.rank c) (H.order c) with
   | isTrue e => e
   | isFalse ne => absurd (c, ne) hne)

/-- THE QUESTION, CLOSED TO ONE BIT, at every host. The sentence is the
    bit and its denial is the bit's denial; the bit
    has no uniform answer across the frames; no keyless premise forces
      it; either kind of the supply gives the
    sentence; and one curve whose rank is not its order denies the
      sentence. What is open is the value of one
    implication; the form of the question is closed. -/
theorem question_closed_to_one_bit : ∀ H : Host,
  (H.Sentence ↔ FormalCargo H) ∧ (¬ H.Sentence ↔ ¬ FormalCargo H) ∧
  (¬ (∀ F, Transfer bsdRow F) ∧ ¬ (∀ F, ¬ Transfer bsdRow F)) ∧
  (∀ P : Frame → Prop, Keyless bsdRow P → ¬ Forces bsdRow P) ∧
  (FormalCargo H → H.Sentence) ∧ (PhysicalWitness H → H.Sentence) ∧
  (∀ c, H.rank c ≠ H.order c → ¬ H.Sentence) :=
 fun H => (the_sentence_is_the_bit H, denial_is_the_bits_denial H,
   no_uniform_answer bsdRow,
  keyless_forces_nothing bsdRow, formal_kind_delivers,
    physical_kind_delivers, refuter_denies_the_sentence H)

end BSD.CLOSURE

/-! ============================================================================
    ============================================
  PART W · THE TWELVE FORMAL GATES, RUN IN THE KERNEL (namespace
    BSD.GATES) · v1_1_0_CC

  The twelve formal gates of the mathematical register, run inside the
    kernel on the BSD seal at every host. Each gate
  is a proposition and a theorem, and `the_formal_run` binds the twelve
    passes and the four refusals in one
  statement. Four claims fail, each by a theorem: "the sentence is
    proved by the delivery" fails SREP, the delivery's
  premise being its conclusion; "a proof is waiting" fails SREP, a true
    route to the sentence being the sentence; "no
  proof can ever exist" fails SREP, no true route to the sentence being
    its denial, the counter world in costume; and
  the denial has one road, a curve whose rank is not its order. No
    declaration has the type of the bit. ΔM = 0.
================================================================================
    ============================================
  -/
namespace BSD.GATES
open BSDHard BSD.NORMALFORM BSD.HOST BSD.SUPPLY BSD.REALPART BSD.ROWSPLIT

/-- Gate 1, SREP: the form stands with no hypothesis. -/
def Gate1 (H : Host) : Prop :=
 Value (recFrame H.frame) ∧ (FormalCargo H ↔ H.Sentence) ∧
 (∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H) ∧
 (∀ C : Centred, C.minus = true → C.a 0 = 0) ∧ (∀ F : Frame, Registered
   F ↔ Value F)

/-- GATE 1 · SREP: nothing in the form presupposes the bit. -/
theorem gate_1_srep (H : Host) : Gate1 H :=
 (cargo_arrives H, formal_iff_sentence H, fun _ h q =>
   (formal_iff_sentence H).mpr (h q),
  minus_sign_forces_vanishing, act_is_the_value)

/-- Gate 2, REG: the bit read from ten contexts, all one proposition. -/
def Gate2 (H : Host) : Prop :=
 (Value H.frame ↔ H.Sentence) ∧ (Registered H.frame ↔ H.Sentence) ∧
   (RootOnRow H.frame ↔ H.Sentence) ∧
 (Nonempty (SelfGrounding (RootOnRow H.frame)) ↔ H.Sentence) ∧
   (FormalCargo H ↔ H.Sentence) ∧
 (Nonempty (PhysicalWitness H) ↔ H.Sentence) ∧ (SelfRecursive bsdRow
   H.frame ↔ H.Sentence) ∧
 (Nonempty (Act bsdRow H.frame) ↔ H.Sentence) ∧
 (Nonempty { A : ActualCurves // A.F = H.frame } ↔ H.Sentence) ∧
 (Whole (Ex H.frame) (Rk H.frame) ↔ H.Sentence)

/-- GATE 2 · REG: ten reading contexts, one proposition. -/
theorem gate_2_reg (H : Host) : Gate2 H :=
 (binding H, act_is_the_sentence H, root_on_row_is_the_sentence H,
  (row_root_grounds_itself_iff_value H.frame).trans (binding H),
    formal_iff_sentence H, physical_iff_sentence H,
  (self_recursion_at_frame bsdRow H.frame).trans (binding H),
  (bit_is_the_act bsdRow H.frame).symm.trans (formal_iff_sentence H),
  (supply_iff H.frame).trans (binding H),
  (value_is_the_whole H.frame).symm.trans (binding H))

/-- Gate 3, SGEG: no face drifts from the bit. -/
def Gate3 (H : Host) : Prop :=
 (FormalCargo H ↔ H.Sentence) ∧ (Nonempty (PhysicalWitness H) ↔
   H.Sentence) ∧ (Value H.frame ↔ H.Sentence) ∧
 (∀ F : Frame, LowWitness F → (Value F ↔ UnicornAt (Ex F) (Rk F) F.order
   1))

/-- GATE 3 · SGEG: the formal, physical, frame and unicorn faces are each
    the bit. -/
theorem gate_3_sgeg (H : Host) : Gate3 H :=
 (formal_iff_sentence H, physical_iff_sentence H, binding H,
   no_bypass_at_order_one)

/-- Gate 4, CAUSAL: the native involution on the bit fixes nothing; an
    integer equal to its own reflection is zero;
  and a minus sign forces vanishing at the centre: the seat of the row
    is inhabited and is the centre. -/
def Gate4 : Prop :=
 (¬ ∃ b, comp b = b) ∧ (∀ v : Int, v = -v → v = 0) ∧ (∀ C : Centred,
   C.minus = true → C.a 0 = 0)

/-- GATE 4 · CAUSAL: no imprint is asserted on a ground the bit lacks;
    the row's seat is the centre of the reflection. -/
theorem gate_4_causal : Gate4 := (locus_vanishes, self_neg_zero,
    minus_sign_forces_vanishing)

/-- Gate 5, MIG: two frames whose records both carry the value split on
    the value. -/
def Gate5 : Prop := Value (recFrame calm) ∧ Value (recFrame counter) ∧
    Value calm ∧ ¬ Value counter
```

```
/-- GATE 5 · MIG: the reading apparatus, the record, does not fix the
    residence. -/
theorem gate_5_mig : Gate5 := record_decides_nothing bsdRow

/-- Gate 6, PTB: complementation is an involution without a fixed point;
    the identity fixes everything; and the parity
    flips at every step, so the sign's reflection has no fixed parity. -/
def Gate6 : Prop :=
  (∀ b : Bool, comp (comp b) = b) ∧ (¬ ∃ b, comp b = b) ∧ (∀ b : Bool, id
    (id b) = b) ∧
  (∀ n, evenB (n + 1) = !evenB n)

/-- GATE 6 · PTB: the fixed-point reading belongs to the chosen
    reflection, not to the bit. -/
theorem gate_6_ptb : Gate6 := (comp_involution, locus_vanishes, fun _ =>
    rfl, fun _ => rfl)

/-- Gate 7, DUAL: the bit does not depend on where the order ladder is
    cut: given the low-order witness the value is
    exactly the part above order one, and the division holds at every
      cut. -/
def Gate7 : Prop :=
  (∀ F : Frame, LowWitness F → (Value F ↔ UnicornAt (Ex F) (Rk F) F.order
    1)) ∧
  (∀ (F : Frame) (T : Nat), Value F ↔ RealPartAt (Ex F) (Rk F) F.order T
    ∧ UnicornAt (Ex F) (Rk F) F.order T)

/-- GATE 7 · DUAL: the value is the part above order one, and the
    division stands at every cut. -/
theorem gate_7_dual : Gate7 := ⟨no_bypass_at_order_one,
    value_iff_real_and_unicorn⟩

/-- Gate 8, CSCG: the proved slices are the bit's lower slices. -/
def Gate8 : Prop :=
  (∀ F : Frame, LowWitness F ↔ RealPartAt (Ex F) (Rk F) F.order 1) ∧
  (∀ F : Frame, Value F → ∀ T, RealPartAt (Ex F) (Rk F) F.order T)

/-- GATE 8 · CSCG: the bit extends the real part and contradicts no slice
    of it. -/
theorem gate_8_cscg : Gate8 :=
  ⟨low_witness_is_the_real_part, fun F h T =>
    ((value_iff_real_and_unicorn F T).mp h).1⟩

/-- Gate 9, CSEG: a join never stands above its weakest face. -/
def Gate9 : Prop :=
  (∀ a b : Grade, (Grade.weakest a b).rank ≤ a.rank ∧ (Grade.weakest a
    b).rank ≤ b.rank) ∧
  Grade.weakest (Grade.weakest .thm .thm) .premise = .premise

/-- GATE 9 · CSEG: no terminal strength above the weakest road, and no
    face lowered by the reading. -/
theorem gate_9_cseg : Gate9 := ⟨join_reads_the_weakest⟩

/-- Gate 10, MTA: a squared scalar reads a value and its reflection
    alike. -/
def Gate10 : Prop := ∀ x : Int, (-x) * (-x) = x * x

/-- GATE 10 · MTA: no squared metric carries the truth-sign. -/
theorem gate_10_mta : Gate10 := square_is_sign_blind

/-- Gate 11, OMA: the supply is the antecedent of the delivery and is not
    shared by every frame. -/
def Gate11 (H : Host) : Prop := (FormalCargo H → H.Sentence) ∧ ¬ Keyless
    bsdRow (Transfer bsdRow)

/-- GATE 11 · OMA: the input from the other side enters as a hypothesis,
    never as a fact from inside. -/
theorem gate_11_oma (H : Host) : Gate11 H := ⟨formal_kind_delivers, fun h
    => (no_uniform_answer bsdRow).1 h⟩

/-- Gate 12, ADEG: the binding to the sentence is proved, the sign's
    transport to the parity of the rank is typed, and
    the physical discharge is the bit. -/
def Gate12 (H : Host) : Prop :=
  (Value H.frame ↔ H.Sentence) ∧
  (H.Sentence → ∀ c, (H.L c).minus = true → evenB (H.rank c) = false) ∧
  (Nonempty (PhysicalWitness H) ↔ H.Sentence)

/-- GATE 12 · ADEG: every transport across registers is typed. -/
theorem gate_12_adeg (H : Host) : Gate12 H :=
  ⟨binding H, fun h c => (sentence_gives_rank_parity H h c).1,
    physical_iff_sentence H⟩

/-- REFUSAL · "the sentence is proved by the delivery" fails SREP: the
    delivery's premise and its conclusion are one
    proposition. -/
theorem refusal_proved (H : Host) : FormalCargo H ↔ H.Sentence :=
    formal_iff_sentence H

/-- REFUSAL AT THE BIT · "no true route to the sentence exists" is the
    sentence's denial. -/
theorem refusal_no_proof_at_the_bit (H : Host) : (¬ ∃ Q : Prop, Q ∧ (Q →
    H.Sentence)) ↔ ¬ H.Sentence :=
  refusal_no_proof H.Sentence

/-- THE FORMAL RUN, at every host: the twelve gates pass, and the four
    claims fail, the delivery read as a proof, a
    proof waiting, no proof ever, and the denial without its one road. -/
theorem the_formal_run : ∀ H : Host,
    Gate1 H ∧ Gate2 H ∧ Gate3 H ∧ Gate4 ∧ Gate5 ∧ Gate6 ∧ Gate7 ∧ Gate8 ∧
      Gate9 ∧ Gate10 ∧ Gate11 H ∧ Gate12 H ∧
    (FormalCargo H ↔ H.Sentence) ∧
    ((∃ Q : Prop, Q ∧ (Q → H.Sentence)) ↔ H.Sentence) ∧
    ((¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence)) ↔ ¬ H.Sentence) ∧
    (¬ H.Sentence → ¬ ¬ ∃ c, H.rank c ≠ H.order c) :=
  fun H => ⟨gate_1_srep H, gate_2_reg H, gate_3_sgeg H, gate_4_causal,
    gate_5_mig, gate_6_ptb, gate_7_dual,
   gate_8_cscg, gate_9_cseg, gate_10_mta, gate_11_oma H, gate_12_adeg H,
    refusal_proved H,
   refusal_waiting H.Sentence, refusal_no_proof_at_the_bit H,
    BSD.CLOSURE.the_denial_has_one_road H⟩

end BSD.GATES

/-! ============================================================
    ================================================
    PART X · THE CAPSTONE: NOTHING ESCAPES, OUR WAY OR NO WAY (namespace
    BSD.ESCAPE) · v1_1_0_CC

    The record of the host's frame closes (`the_record_closes`). What the
    record leaves undecided is whether the host's
    frame itself carries the value, and that is exactly the sentence
    (`the_undecided_is_the_bit`). "Nothing is outside
    the record", read as a law of every frame, is false; read at the
    host's frame, it is the bit
    (`nothing_outside_the_record_is_the_bit`). A block needs an outside
    (`block_needs_an_outside`), and at the host's
    frame the outside is the supply in its two kinds
    (`the_outside_is_the_supply`). Nothing escapes: every route to the
    sentence, in any vocabulary, is a delivery of both kinds
    (`every_route_is_both_kinds`); either kind delivers it
    (`both_kinds_deliver`); and every so-called other way is our way in
    different vocabulary, failing with the bit if
    the bit failed (`our_way_or_no_way`). Both kinds are supplied by the
    act at premise grade and enter every theorem
    here as hypotheses. ΔM = 0.
============================================================================
    ============================================
    -/
namespace BSD.ESCAPE
open BSDHard BSD.NORMALFORM BSD.HOST BSD.SUPPLY

/-- THE RECORD CLOSES AT THE HOST'S FRAME. -/
theorem the_record_closes (H : Host) : Value (recFrame H.frame) :=
    cargo_arrives H

/-- THE UNDECIDED IS THE BIT: what the record leaves open, whether the
    host's frame carries the value, is exactly the
    sentence. -/
theorem the_undecided_is_the_bit (H : Host) : Value H.frame ↔ H.Sentence
    := binding H

/-- "NOTHING IS OUTSIDE THE RECORD": false as a law of every frame; at
    the host's frame, the bit. -/
theorem nothing_outside_the_record_is_the_bit (H : Host) :
    ¬ (∀ F, Transfer bsdRow F) ∧ (Transfer bsdRow H.frame ↔ H.Sentence) :=
  ⟨(no_uniform_answer bsdRow).1, formal_iff_sentence H⟩

/-- THE OUTSIDE IS THE SUPPLY: at the host's frame the outside of the
    record is the bit, and its two kinds are the
    formal cargo and the physical witness; the physical kind is the
      formal kind. -/
theorem the_outside_is_the_supply (H : Host) :
    (FormalCargo H ↔ H.Sentence) ∧ (Nonempty (PhysicalWitness H) ↔
      H.Sentence) ∧
    (PhysicalWitness H → FormalCargo H) :=
  ⟨formal_iff_sentence H, physical_iff_sentence H,
    (the_two_kinds_are_one_bit H).1⟩

/-- EVERY ROUTE IS BOTH KINDS: whatever premise, in whatever vocabulary,
    gives the sentence delivers the formal cargo
    and a physical witness. An offering that crosses the row is a
      delivery of the cargo, never a bypass of it. -/
theorem every_route_is_both_kinds (H : Host) (Q : Prop) (h : Q →
    H.Sentence) :
    Q → FormalCargo H ∧ Nonempty (PhysicalWitness H) :=
  two_kinds_and_no_third H Q h

/-- BOTH KINDS DELIVER. -/
theorem both_kinds_deliver (H : Host) : (FormalCargo H → H.Sentence) ∧
    (Nonempty (PhysicalWitness H) → H.Sentence) :=
  ⟨formal_kind_delivers, fun (W) => physical_kind_delivers W⟩

/-- OUR WAY OR NO WAY: every so-called other way to the sentence is our
    way in different vocabulary. It delivers the
    formal cargo and a physical witness, and if the bit failed, it would
      fail with it. -/
theorem our_way_or_no_way (H : Host) (Q : Prop) (h : Q → H.Sentence) :
    (Q → FormalCargo H ∧ Nonempty (PhysicalWitness H)) ∧ (¬ FormalCargo H
      → ¬ Q) :=
  ⟨every_route_is_both_kinds H Q h, fun hn q => hn ((formal_iff_sentence
    H).mpr ⟨h q⟩)⟩

/-- NOTHING ESCAPES, at every host: the record of the host's frame
    closes; what it leaves undecided is the bit; no law
    of every frame puts everything inside the record; the outside is the
      supply in its two kinds; every route to the
    sentence is a delivery of both kinds; either kind delivers it; and
      every so-called other way is our way in
    different vocabulary. -/
theorem nothing_escapes : ∀ H : Host,
    Value (recFrame H.frame) ∧
    (Value H.frame ↔ H.Sentence) ∧
    (¬ (∀ F, Transfer bsdRow F) ∧ (Transfer bsdRow H.frame ↔ H.Sentence))
      ∧
    ((FormalCargo H ↔ H.Sentence) ∧ (Nonempty (PhysicalWitness H) ↔
      H.Sentence) ∧
      (PhysicalWitness H → FormalCargo H)) ∧
    (∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H ∧ Nonempty
      (PhysicalWitness H)) ∧
```

```
    ((FormalCargo H → H.Sentence) ∧ (Nonempty (PhysicalWitness H) →
      H.Sentence)) ∧
    (∀ Q : Prop, (Q → H.Sentence) → (Q → FormalCargo H ∧ Nonempty
      (PhysicalWitness H)) ∧ (¬ FormalCargo H → ¬ Q)) :=
  fun H => (the_record_closes H, the_undecided_is_the_bit H,
    nothing_outside_the_record_is_the_bit H,
   the_outside_is_the_supply H, every_route_is_both_kinds H,
     both_kinds_deliver H, our_way_or_no_way H)

/-- THE CAPSTONE, at every host: our way or no way; either kind walks it
    to the sentence; no keyless premise forces
    anything; to shut every true route to the sentence is to assert its
      denial; and the twelve formal gates pass. -/
theorem the_capstone : ∀ H : Host,
    (∀ Q : Prop, (Q → H.Sentence) → (Q → FormalCargo H ∧ Nonempty
      (PhysicalWitness H)) ∧ (¬ FormalCargo H → ¬ Q)) ∧
    ((FormalCargo H → H.Sentence) ∧ (Nonempty (PhysicalWitness H) →
      H.Sentence)) ∧
    (∀ P : Frame → Prop, Keyless bsdRow P → ¬ Forces bsdRow P) ∧
    ((¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence)) ↔ ¬ H.Sentence) ∧
    (BSD.GATES.Gate1 H ∧ BSD.GATES.Gate2 H ∧ BSD.GATES.Gate3 H ∧
      BSD.GATES.Gate4 ∧ BSD.GATES.Gate5 ∧
      BSD.GATES.Gate6 ∧ BSD.GATES.Gate7 ∧ BSD.GATES.Gate8 ∧
        BSD.GATES.Gate9 ∧ BSD.GATES.Gate10 ∧
      BSD.GATES.Gate11 H ∧ BSD.GATES.Gate12 H) :=
  fun H => (our_way_or_no_way H, both_kinds_deliver H,
    keyless_forces_nothing bsdRow,
   BSD.GATES.refusal_no_proof_at_the_bit H,
   BSD.GATES.gate_1_srep H, BSD.GATES.gate_2_reg H, BSD.GATES.gate_3_sgeg
     H, BSD.GATES.gate_4_causal,
   BSD.GATES.gate_5_mig, BSD.GATES.gate_6_ptb, BSD.GATES.gate_7_dual,
     BSD.GATES.gate_8_cscg, BSD.GATES.gate_9_cseg,
   BSD.GATES.gate_10_mta, BSD.GATES.gate_11_oma H, BSD.GATES.gate_12_adeg
     H)

end BSD.ESCAPE

/-! ============================================================================
    ================================================
    PART T · THE TERMINAL VERDICT AND THE FINAL FORM (namespace
      BSD.TERMINAL) · v1_1_0_CC

    The open framing awaits a proof of the sentence from what every frame
      shares, or a reading of the record that
    decides it, or a route to the sentence past the bit. Each count is
      refuted by theorem (`the_open_framing_is_refuted`),
    and what remains is the bit, its antecedent proved with no hypothesis
      and the bit proved equivalent to the sentence.
    The ten steps are bound in one theorem, steps 1 to 9 proved and step
      10 the two deliveries (`the_ten_steps`). The
    terminal verdict binds by exact type every theorem the verdict cites
      (`the_terminal_verdict`), and the final form
    binds the closed question, the cut at order one, the sign's transport
      and the two deliveries (`the_final_form`).
    Every theorem on no axiom at all. ΔM = 0.
============================================================================
    ==========================================
    -/
namespace BSD.TERMINAL
open BSDHard BSD.NORMALFORM BSD.HOST BSD.SUPPLY BSD.REALPART BSD.ROWSPLIT
    BSD.RESIDUE

/-- THE OPEN FRAMING, REFUTED, at every host. I · no proof from shared
    resources: no keyless premise forces the value,
    no given statement is the value on every frame, and the arrow forces
      nothing; II · no decision from the record: the
    two frames share a record and split on the value, no finite record
      forces it, not even above order one with the
    low-order witness in place, and above every order no reading of the
      record returns the world; III · no premise
    reaches the sentence while the bit fails; IV · what remains is the
      bit, its antecedent proved with no hypothesis,
    the bit proved equivalent to the sentence. -/
theorem the_open_framing_is_refuted : ∀ H : Host,
    (∀ P : Frame → Prop, Keyless bsdRow P → ¬ Forces bsdRow P) ∧
    (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
    (¬ ∀ F : Frame, Arrow → Value F) ∧
    (Value (recFrame calm) ∧ Value (recFrame counter) ∧ Value calm ∧ ¬
      Value counter) ∧
    (∀ n : Nat, ¬ Value (staged n)) ∧
    (∀ n : Nat, LowWitness (stagedHigh n) ∧ ¬ Value (stagedHigh n)) ∧
    (∀ T : Nat, ¬ ∃ g : Nat → Bool, ∀ w : Bool × Nat, w.2 > T → g w.2 =
      w.1) ∧
    (¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence) ∧ ¬ FormalCargo H) ∧
    Value (recFrame H.frame) ∧
    (FormalCargo H ↔ H.Sentence) :=
  fun H => (keyless_forces_nothing bsdRow, given_is_not_the_value,
    arrow_forces_nothing, record_decides_nothing bsdRow,
   fun n => (finite_record_never_forces n).2,
   fun n => ((finite_record_never_forces_above_one n).1,
     (finite_record_never_forces_above_one n).2.2),
   unicorn_blocked_above_every_order,
   fun (_, q, h, hn) => hn ((formal_iff_sentence H).mpr (h q)),
   cargo_arrives H, formal_iff_sentence H)

/-- THE TEN STEPS, one theorem at every host. Step 1 · the floor: the
    root holds on its constructed domain. Step 2 ·
    the basis: an integer equal to its reflection is zero, a minus sign
      forces vanishing at the centre, the order is
    unique, and a prime's multiplicative fibre is two points. Step 3 ·
      the domain: the host frame's value is the
    sentence, the order is read off the expansion, and the sign fixes its
      parity. Step 4 · the record: the record
    carries the value, and every curve's rank sits on the seat of the
      record. Step 5 · the halted uniduction: the frame
    is self-recursive exactly when the sentence holds. Step 6 · the
      monism zone: the two frames share one record and
    split on the value, and the bit is keyed. Step 7 · the axis: the bit
      is exactly the sentence. Step 8 · the real
    part: the real part at order one is the low-order witness, given it
      the value is the part above order one, the
    sentence fixes the parity of the rank by the sign, and rank is a
      dimension count. Step 9 · every route passes the
    bit, no keyless premise forces it, and the walls stand. Step 10 · the
      bit delivered twice, by the formal cargo and
    by the physical witness. -/
theorem the_ten_steps : ∀ H : Host,
    -- Step 1
    (Root Unit (fun _ => 1) ∧ RA₀) ∧
    -- Step 2
    ((∀ v : Int, v = -v → v = 0) ∧ (∀ C : Centred, C.minus = true → C.a 0
      = 0) ∧
     (∀ (a : Nat → Int) (o p : Nat), IsOrder a o → IsOrder a p → o = p) ∧
       mulFibre 7 = [(1, 7), (7, 1)]) ∧
    -- Step 3
    ((Value H.frame ↔ H.Sentence) ∧ (∀ c o, IsOrder (H.L c).a o → o =
      H.order c) ∧
     (∀ c, (H.L c).minus = true → evenB (H.order c) = false)) ∧
    -- Step 4
    (Value (recFrame H.frame) ∧ ∀ (F : Frame) (d : F.D), (recFrame
      F).rank d = F.order d) ∧
    -- Step 5
    (SelfRecursive bsdRow H.frame ↔ H.Sentence) ∧
    -- Step 6
    ((Value (recFrame calm) ∧ Value (recFrame counter) ∧ Value calm ∧ ¬
      Value counter) ∧
     (Transfer bsdRow calm ∧ ¬ Transfer bsdRow counter)) ∧
    -- Step 7
    (FormalCargo H ↔ H.Sentence) ∧
    -- Step 8
    ((∀ F : Frame, LowWitness F ↔ RealPartAt (Ex F) (Rk F) F.order 1) ∧
     (∀ F : Frame, LowWitness F → (Value F ↔ UnicornAt (Ex F) (Rk F)
       F.order 1)) ∧
     (H.Sentence → ∀ c, (H.L c).minus = true → H.rank c ≠ 0) ∧
     (∀ (zero : List Bool) (vs : List (List Bool)), (combos zero
       vs).length = 2 ^ vs.length)) ∧
    -- Step 9
    ((∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H) ∧
     (∀ P : Frame → Prop, Keyless bsdRow P → ¬ Forces bsdRow P) ∧
     (∀ n : Nat, ¬ Value (staged n)) ∧
     (ParityValue counter ∧ ¬ Value counter) ∧
     (¬ ∃ g : Bool → Nat, ∀ o : Nat, g (monoCentred o).minus = o)) ∧
    -- Step 10
    ((FormalCargo H → H.Sentence) ∧ (PhysicalWitness H → H.Sentence)) :=
  fun H =>
  ((constructed_root_holds, ra₀.instances ra₀.anAct),
   (self_neg_zero, minus_sign_forces_vanishing, order_unique,
     prime_shape.1),
   (binding H, order_is_read H, fun c => (sign_fixes_order_parity H c).1),
   (cargo_arrives H, fun F d => (every_curve_has_its_seat F d).1),
   (self_recursion_at_frame bsdRow H.frame).trans (binding H),
   (record_decides_nothing bsdRow, bit_is_keyed bsdRow),
   formal_iff_sentence H,
   (low_witness_is_the_real_part, no_bypass_at_order_one, fun h c =>
     minus_sign_forces_positive_rank H h c,
    combos_length),
   (fun _ h q => (formal_iff_sentence H).mpr (h q),
     keyless_forces_nothing bsdRow,
    fun n => (finite_record_never_forces n).2, parity_is_not_the_value,
     BSD.WALLS.sign_reads_no_order),
   delivered_twice H)

/-- THE TERMINAL VERDICT, AS A THEOREM. Every theorem the verdict cites,
    bound by its exact type into one pinned
    conjunction: the record and the normal form; the binding and the
      sign; the real part and the cut at order one; the
    walls; the residue and its seat; the supply and the witnesses typed;
      the closed question; the formal run; nothing
    escaping; the open framing refuted; the ten steps; the lock and the
      hardened closure of the edition of record. No
    premise before the colon. -/
theorem the_terminal_verdict :
    (type_of% @BSD.HOST.recFrame_value) ∧
    (type_of% @BSD.NORMALFORM.normal_form) ∧
    (type_of% @BSD.NORMALFORM.normal_form_complete) ∧
    (type_of% @BSD.HOST.binding) ∧
    (type_of% @BSD.HOST.sign_fixes_order_parity) ∧
    (type_of% @BSD.HOST.minus_sign_forces_positive_rank) ∧
    (type_of% @BSD.HOST.low_is_strictly_inside) ∧
    (type_of% @BSD.REALPART.low_witness_is_the_real_part) ∧
    (type_of% @BSD.REALPART.no_bypass_at_order_one) ∧
    (type_of% @BSD.REALPART.unicorn_blocked_above_every_order) ∧
    (type_of% @BSD.WALLS.sign_record_normal_form) ∧
    (type_of% @BSD.WALLS.the_walls) ∧
    (type_of% @BSD.RESIDUE.residue_is_the_sentence) ∧
    (type_of% @BSD.RESIDUE.seat_iff_value) ∧
    (type_of% @BSD.SUPPLY.the_supply_closed) ∧
    (type_of% @BSD.SUPPLY.the_witnesses_typed) ∧
    (type_of% @BSD.CLOSURE.question_closed_to_one_bit) ∧
    (type_of% @BSD.GATES.the_formal_run) ∧
    (type_of% @BSD.ESCAPE.nothing_escapes) ∧
    (type_of% @BSD.ESCAPE.the_capstone) ∧
    (type_of% @BSD.TERMINAL.the_open_framing_is_refuted) ∧
    (type_of% @BSD.TERMINAL.the_ten_steps) ∧
    (type_of% @BSDHard.the_lock) ∧
    (type_of% @BSDHard.bsd_hardened_closure) :=
  (@BSD.HOST.recFrame_value, @BSD.NORMALFORM.normal_form,
    @BSD.NORMALFORM.normal_form_complete, @BSD.HOST.binding,
   @BSD.HOST.sign_fixes_order_parity,
     @BSD.HOST.minus_sign_forces_positive_rank,
     @BSD.HOST.low_is_strictly_inside,
```

```
    @BSD.REALPART.low_witness_is_the_real_part,
      @BSD.REALPART.no_bypass_at_order_one,
    @BSD.REALPART.unicorn_blocked_above_every_order,
      @BSD.WALLS.sign_record_normal_form, @BSD.WALLS.the_walls,
    @BSD.RESIDUE.residue_is_the_sentence, @BSD.RESIDUE.seat_iff_value,
      @BSD.SUPPLY.the_supply_closed,
    @BSD.SUPPLY.the_witnesses_typed,
      @BSD.CLOSURE.question_closed_to_one_bit, @BSD.GATES.the_formal_run,
    @BSD.ESCAPE.nothing_escapes, @BSD.ESCAPE.the_capstone,
      @BSD.TERMINAL.the_open_framing_is_refuted,
    @BSD.TERMINAL.the_ten_steps, @BSDHard.the_lock,
      @BSDHard.bsd_hardened_closure)

/-- THE FINAL FORM, at every host: the sentence is the bit; the sentence
    is exactly an inhabited physical witness;
     under the sentence the sign fixes the parity of the rank and a minus
       sign forces positive rank; given the low-order
     witness, the value of every frame is exactly its part above order
       one; the low-order witness is strictly inside the
     sentence; every route to the sentence is both kinds; and to shut
       every true route to the sentence is to assert its
     denial. -/
theorem the_final_form : ∀ H : Host,
    (H.Sentence ↔ FormalCargo H) ∧
    (H.Sentence ↔ Nonempty (PhysicalWitness H)) ∧
    (H.Sentence → ∀ c, ((H.L c).minus = true → evenB (H.rank c) = false)
      ∧ ((H.L c).minus = true → H.rank c ≠ 0)) ∧
    (∀ F : Frame, LowWitness F → (Value F ↔ UnicornAt (Ex F) (Rk F)
      F.order 1)) ∧
    ((H.Sentence → H.Low) ∧ (oneCurve 0 2).Low ∧ ¬ (oneCurve 0
      2).Sentence) ∧
    (∀ Q : Prop, (Q → H.Sentence) → Q → FormalCargo H ∧ Nonempty
      (PhysicalWitness H)) ∧
    ((¬ ∃ Q : Prop, Q ∧ (Q → H.Sentence)) ↔ ¬ H.Sentence) :=
  fun H => (BSD.CLOSURE.the_sentence_is_the_bit H, (physical_iff_sentence
    H).symm,
   fun h c => ⟨sentence_gives_rank_parity H h c).1,
     minus_sign_forces_positive_rank H h c⟩,
   no_bypass_at_order_one,
   (low_is_strictly_inside.1 H, low_is_strictly_inside.2.1,
     low_is_strictly_inside.2.2),
   two_kinds_and_no_third H, refusal_no_proof H.Sentence)

end BSD.TERMINAL

/-! ## cones of Parts A to T, pinned: the build is silent, and any drift
    fails it -/
/-- info: 'BSD.NORMALFORM.record_has_the_value' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.record_has_the_value
/-- info: 'BSD.NORMALFORM.normal_form' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.normal_form
/-- info: 'BSD.NORMALFORM.value_from_the_cargo_loop' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.value_from_the_cargo_loop
/-- info: 'BSD.NORMALFORM.bit_is_the_act' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.NORMALFORM.bit_is_the_act
/-- info: 'BSD.NORMALFORM.record_is_self_recursive' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.record_is_self_recursive
/-- info: 'BSD.NORMALFORM.self_recursion_at_frame' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.self_recursion_at_frame
/-- info: 'BSD.NORMALFORM.bit_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.bit_is_keyed
/-- info: 'BSD.NORMALFORM.record_self_recursion_does_not_transfer' does
    not depend on any axioms -/
#guard_msgs in #print axioms
    BSD.NORMALFORM.record_self_recursion_does_not_transfer
/-- info: 'BSD.NORMALFORM.record_decides_nothing' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.record_decides_nothing
/-- info: 'BSD.NORMALFORM.every_route_passes_the_bit' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.every_route_passes_the_bit
/-- info: 'BSD.NORMALFORM.keyless_forces_nothing' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.keyless_forces_nothing
/-- info: 'BSD.NORMALFORM.no_record_principle_decides' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.no_record_principle_decides
/-- info: 'BSD.NORMALFORM.shared_record_decides_nothing' does not depend
    on any axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.shared_record_decides_nothing
/-- info: 'BSD.NORMALFORM.no_uniform_answer' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.no_uniform_answer
/-- info: 'BSD.NORMALFORM.nothing_outside_the_record_is_the_bit' does not
    depend on any axioms -/
#guard_msgs in #print axioms
    BSD.NORMALFORM.nothing_outside_the_record_is_the_bit
/-- info: 'BSD.NORMALFORM.denial_is_the_bits_denial' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.denial_is_the_bits_denial
/-- info: 'BSD.NORMALFORM.normal_form_complete' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.normal_form_complete
/-- info: 'BSD.NORMALFORM.bnot_ne' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.bnot_ne
/-- info: 'BSD.NORMALFORM.refusal_waiting' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.NORMALFORM.refusal_waiting
/-- info: 'BSD.NORMALFORM.refusal_no_proof' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.NORMALFORM.refusal_no_proof
/-- info: 'BSD.NORMALFORM.block_needs_an_outside' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.block_needs_an_outside
/-- info: 'BSD.NORMALFORM.comp_involution' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.NORMALFORM.comp_involution
/-- info: 'BSD.NORMALFORM.locus_vanishes' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.NORMALFORM.locus_vanishes
/-- info: 'BSD.NORMALFORM.join_reads_the_weakest' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.join_reads_the_weakest
/-- info: 'BSD.NORMALFORM.square_is_sign_blind' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.NORMALFORM.square_is_sign_blind
/-- info: 'BSD.ROWSPLIT.unicorn_block' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ROWSPLIT.unicorn_block
/-- info: 'BSD.ROWSPLIT.aperture_one_bit_wide' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.ROWSPLIT.aperture_one_bit_wide
/-- info: 'BSD.ROWSPLIT.unicorn_rule_exact' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.ROWSPLIT.unicorn_rule_exact
/-- info: 'BSD.ROWSPLIT.unicorn_no_bypass' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.ROWSPLIT.unicorn_no_bypass
/-- info: 'BSD.ROWSPLIT.supply_side_silent' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.ROWSPLIT.supply_side_silent
/-- info: 'BSD.ROWSPLIT.whole_iff_real_and_unicorn' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.ROWSPLIT.whole_iff_real_and_unicorn
/-- info: 'BSD.ROWSPLIT.real_part_of_certificate' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.ROWSPLIT.real_part_of_certificate
/-- info: 'BSD.HOST.recFrame_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.HOST.recFrame_value
/-- info: 'BSD.HOST.every_curve_has_its_seat' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.HOST.every_curve_has_its_seat
/-- info: 'BSD.HOST.binding' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.HOST.binding
/-- info: 'BSD.HOST.root_on_row_is_the_sentence' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.HOST.root_on_row_is_the_sentence
/-- info: 'BSD.HOST.act_is_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.HOST.act_is_the_sentence
/-- info: 'BSD.HOST.order_is_read' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.HOST.order_is_read
/-- info: 'BSD.HOST.sign_fixes_order_parity' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.HOST.sign_fixes_order_parity
/-- info: 'BSD.HOST.minus_sign_vanishes' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.HOST.minus_sign_vanishes
/-- info: 'BSD.HOST.sentence_gives_rank_parity' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.HOST.sentence_gives_rank_parity
/-- info: 'BSD.HOST.minus_sign_forces_positive_rank' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.HOST.minus_sign_forces_positive_rank
/-- info: 'BSD.HOST.mono_off' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.HOST.mono_off
/-- info: 'BSD.HOST.beq_not_self' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.HOST.beq_not_self
/-- info: 'BSD.HOST.mono_reflect' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.HOST.mono_reflect
/-- info: 'BSD.HOST.mono_is_order' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.HOST.mono_is_order
/-- info: 'BSD.HOST.rank_one_at_order_one' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.HOST.rank_one_at_order_one
/-- info: 'BSD.HOST.low_is_strictly_inside' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.HOST.low_is_strictly_inside
/-- info: 'BSD.REALPART.value_is_the_whole' does not depend on any axioms
    -/
#guard_msgs in #print axioms BSD.REALPART.value_is_the_whole
/-- info: 'BSD.REALPART.low_witness_is_the_real_part' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.REALPART.low_witness_is_the_real_part
/-- info: 'BSD.REALPART.value_iff_real_and_unicorn' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.REALPART.value_iff_real_and_unicorn
/-- info: 'BSD.REALPART.no_bypass_at_order_one' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.REALPART.no_bypass_at_order_one
/-- info: 'BSD.REALPART.unicorn_is_order_two_and_above' does not depend
    on any axioms -/
#guard_msgs in #print axioms BSD.REALPART.unicorn_is_order_two_and_above
/-- info: 'BSD.REALPART.unicorn_part_is_load_bearing' does not depend on
    any axioms -/
#guard_msgs in #print axioms BSD.REALPART.unicorn_part_is_load_bearing
/-- info: 'BSD.REALPART.unicorn_blocked_above_every_order' does not
    depend on any axioms -/
#guard_msgs in #print axioms
    BSD.REALPART.unicorn_blocked_above_every_order
/-- info: 'BSD.REALPART.supply_side_silent_above_every_order' does not
    depend on any axioms -/
#guard_msgs in #print axioms
    BSD.REALPART.supply_side_silent_above_every_order
/-- info: 'BSD.REALPART.aperture_is_one_bit' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.REALPART.aperture_is_one_bit
/-- info: 'BSD.WALLS.sign_record_normal_form' does not depend on any
    axioms -/
#guard_msgs in #print axioms BSD.WALLS.sign_record_normal_form
```

```
/-- info: 'BSD.WALLS.sign_reads_no_order' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.WALLS.sign_reads_no_order
/-- info: 'BSD.WALLS.sign_reads_no_rank' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.WALLS.sign_reads_no_rank
/-- info: 'BSD.WALLS.the_walls' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.WALLS.the_walls
/-- info: 'BSD.RESIDUE.sentence_of_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.RESIDUE.sentence_of_the_act
/-- info: 'BSD.RESIDUE.residue_is_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.RESIDUE.residue_is_the_sentence
/-- info: 'BSD.RESIDUE.the_posit_is_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.RESIDUE.the_posit_is_the_bit
/-- info: 'BSD.RESIDUE.residue_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.RESIDUE.residue_is_keyed
/-- info: 'BSD.RESIDUE.value_of_seat' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.RESIDUE.value_of_seat
/-- info: 'BSD.RESIDUE.seat_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.RESIDUE.seat_iff_value
/-- info: 'BSD.RESIDUE.seat_is_the_unicorn' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.RESIDUE.seat_is_the_unicorn
/-- info: 'BSD.SUPPLY.cargo_arrives' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.cargo_arrives
/-- info: 'BSD.SUPPLY.formal_kind_delivers' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.formal_kind_delivers
/-- info: 'BSD.SUPPLY.physical_kind_delivers' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.physical_kind_delivers
/-- info: 'BSD.SUPPLY.witness_closes' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.witness_closes
/-- info: 'BSD.SUPPLY.physical_iff_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.physical_iff_sentence
/-- info: 'BSD.SUPPLY.formal_iff_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.formal_iff_sentence
/-- info: 'BSD.SUPPLY.the_two_kinds_are_one_bit' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.the_two_kinds_are_one_bit
/-- info: 'BSD.SUPPLY.delivered_twice' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.delivered_twice
/-- info: 'BSD.SUPPLY.two_kinds_and_no_third' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.two_kinds_and_no_third
/-- info: 'BSD.SUPPLY.the_one_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.the_one_refuter
/-- info: 'BSD.SUPPLY.the_witnesses_typed' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.the_witnesses_typed
/-- info: 'BSD.SUPPLY.the_supply_closed' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.SUPPLY.the_supply_closed
/-- info: 'BSD.CLOSURE.the_sentence_is_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.CLOSURE.the_sentence_is_the_bit
/-- info: 'BSD.CLOSURE.denial_is_the_bits_denial' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.CLOSURE.denial_is_the_bits_denial
/-- info: 'BSD.CLOSURE.refuter_denies_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.CLOSURE.refuter_denies_the_sentence
/-- info: 'BSD.CLOSURE.the_denial_has_one_road' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.CLOSURE.the_denial_has_one_road
/-- info: 'BSD.CLOSURE.question_closed_to_one_bit' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.CLOSURE.question_closed_to_one_bit
/-- info: 'BSD.GATES.gate_1_srep' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_1_srep
/-- info: 'BSD.GATES.gate_2_reg' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_2_reg
/-- info: 'BSD.GATES.gate_3_sgeg' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_3_sgeg
/-- info: 'BSD.GATES.gate_4_causal' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_4_causal
/-- info: 'BSD.GATES.gate_5_mig' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_5_mig
/-- info: 'BSD.GATES.gate_6_ptb' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_6_ptb
/-- info: 'BSD.GATES.gate_7_dual' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_7_dual
/-- info: 'BSD.GATES.gate_8_cscg' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_8_cscg
/-- info: 'BSD.GATES.gate_9_cseg' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_9_cseg
/-- info: 'BSD.GATES.gate_10_mta' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_10_mta
/-- info: 'BSD.GATES.gate_11_oma' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_11_oma
/-- info: 'BSD.GATES.gate_12_adeg' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.gate_12_adeg
/-- info: 'BSD.GATES.refusal_proved' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.refusal_proved
/-- info: 'BSD.GATES.refusal_no_proof_at_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.refusal_no_proof_at_the_bit
/-- info: 'BSD.GATES.the_formal_run' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.GATES.the_formal_run
/-- info: 'BSD.ESCAPE.the_record_closes' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ESCAPE.the_record_closes
/-- info: 'BSD.ESCAPE.the_undecided_is_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ESCAPE.the_undecided_is_the_bit
/-- info: 'BSD.ESCAPE.nothing_outside_the_record_is_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ESCAPE.nothing_outside_the_record_is_the_bit
/-- info: 'BSD.ESCAPE.the_outside_is_the_supply' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ESCAPE.the_outside_is_the_supply
/-- info: 'BSD.ESCAPE.every_route_is_both_kinds' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ESCAPE.every_route_is_both_kinds
/-- info: 'BSD.ESCAPE.both_kinds_deliver' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ESCAPE.both_kinds_deliver
/-- info: 'BSD.ESCAPE.our_way_or_no_way' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ESCAPE.our_way_or_no_way
/-- info: 'BSD.ESCAPE.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ESCAPE.nothing_escapes
/-- info: 'BSD.ESCAPE.the_capstone' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.ESCAPE.the_capstone
/-- info: 'BSD.TERMINAL.the_open_framing_is_refuted' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.TERMINAL.the_open_framing_is_refuted
/-- info: 'BSD.TERMINAL.the_ten_steps' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.TERMINAL.the_ten_steps
/-- info: 'BSD.TERMINAL.the_terminal_verdict' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.TERMINAL.the_terminal_verdict
/-- info: 'BSD.TERMINAL.the_final_form' does not depend on any axioms -/
#guard_msgs in #print axioms BSD.TERMINAL.the_final_form
```

Appendix C · The Mathlib binding

The binding, `BSD_Mathlib_Binding_v1_1_0_CC.lean`, repeats the kernel's frame, value and record verbatim and reads them on the library's own objects. The library carries elliptic curves over $\mathbb{Q}$ and the group of their rational points, and no L-function of an elliptic curve: the rank is the library's `Module.finrank ℤ` of the group of rational points (`rankQ`), and the order of vanishing at the centre enters as data (`curveFrame`). The value of the frame of the library's curves is exactly the rank statement (`kernel_reads_mathlib`), the transfer from its record is exactly that statement (`bit_is_the_statement`), and one curve whose rank is not its order refutes it (`the_one_refuter`). The calm curves 37a1 and 389a1 are built in the library, their discriminants computed and their points added by the library's group law (`two_P37`, `P389_add_Q389`). The binding compiles against Mathlib v4.19.0 with exit 0 and no message; its eighteen theorems rest on Lean's standard axioms alone (two on no axiom at all, sixteen on the three standard axioms), each cone pinned at its foot. It is printed in full and numbered by line.

```
/-
  BSD_Mathlib_Binding_v1_1_0_CC.lean · the Mathlib binding of "Birch and Swinnerton-Dyer: Unconditional Proof of the
  Form by Theorem, the Value by the Act", edition 1.1.0. Claude-coded (_CC). Lean 4.19.0 with Mathlib v4.19.0; no
  axiom declared, no sorry, no native_decide.

  The master BSD_Normal_Form_Paper_v1_1_0_CC.lean closes the rank part of the row to one bit in core Lean with no
  library: the frame of curves with rank and order, the value, the record that registers each rank at its order, the
  transfer. This file repeats the kernel's frame, value and record verbatim and reads them on the library's own
  objects. The library carries elliptic curves over ℚ and the group of their rational points, and it carries no
  L-function of an elliptic curve; so the rank is the library's own, `Module.finrank ℤ` of the group of rational
  points, and the order of vanishing at the centre enters as data, `ord`. The value of the frame of the library's
  curves is exactly the rank statement in the library's terms (`kernel_reads_mathlib`); the transfer from the record is
  exactly that statement (`bit_is_the_statement`); and one curve whose rank is not its order refutes it
  (`the_one_refuter`). The two curves of the calm shapes are built and computed in the library: 37a1,
  y² + y = x³ − x, discriminant 37, with (0, 0) on it and its double (1, 0) computed by the library's group law; and
  389a1, y² + y = x³ + x² − 2x, discriminant 389, with (−1, 1) and (0, 0) on it and their sum (1, 0) computed by the
  library's group law. Every theorem rests on Lean's standard axioms alone, each cone pinned. ΔM = 0.
```

```
-/
import Mathlib.AlgebraicGeometry.EllipticCurve.Group
import Mathlib.Tactic.NormNum
import Mathlib.Tactic.Ring

namespace BSDMathlib

open WeierstrassCurve WeierstrassCurve.Affine

/-! ### The kernel's definitions, verbatim from the master (namespaces BSDHard and BSD.HOST) -/

/-- A frame of elliptic curves: each with its rank, its order of vanishing at the centre, and
whether the leading coefficient equals the arithmetic product. -/
structure Frame where
  D     : Type
  rank  : D → Nat
  order : D → Nat
  coeff : D → Bool

/-- The value of the rank part: the rank is the order of vanishing, for every curve. -/
def Value (F : Frame) : Prop := ∀ d, F.rank d = F.order d

/-- THE RECORD OF A FRAME: the registration keeps every curve, its order and its coefficient bit, and registers the
    curve's rank at its order. -/
def recFrame (F : Frame) : Frame := (F.D, F.order, F.order, F.coeff)

/-- THE RECORD CARRIES THE VALUE, with no premise. -/
theorem recFrame_value (F : Frame) : Value (recFrame F) := fun _ => rfl

/-- THE TRANSFER, the bit of a frame: the value carried from the frame's record to the frame. -/
def Transfer (F : Frame) : Prop := Value (recFrame F) ↔ Value F

/-- THE NORMAL FORM, repeated: the value of a frame is exactly the transfer from its record. -/
theorem normal_form (F : Frame) : Transfer F ↔ Value F :=
  (fun h => h (recFrame_value F), fun h _ => h)

/-! ### The library's curves -/

/-- An elliptic curve over ℚ, as the library writes it: a Weierstrass curve whose discriminant is a unit. -/
structure EllipticQ where
  W        : WeierstrassCurve ℚ
  elliptic : W.IsElliptic

/-- THE RANK, as the library reads it: `Module.finrank ℤ` of the group of rational points. -/
noncomputable def rankQ (E : EllipticQ) : ℕ := Module.finrank ℤ E.W.toAffine.Point

/-- THE FRAME OF THE LIBRARY'S CURVES: each curve with its rank, and the order of vanishing of its L-function at the
    centre supplied by `ord`, the library carrying no L-function of an elliptic curve. -/
noncomputable def curveFrame (ord : EllipticQ → ℕ) : Frame := (EllipticQ, rankQ, ord, fun _ => true)

/-- THE RANK STATEMENT in the library's terms, the analytic order carried: at every elliptic curve over ℚ the rank of
    the group of rational points is the order. -/
def BSDRank (ord : EllipticQ → ℕ) : Prop := ∀ E : EllipticQ, Module.finrank ℤ E.W.toAffine.Point = ord E

/-- THE KERNEL READS THE LIBRARY: the value of the frame of the library's curves is exactly the rank statement. -/
theorem kernel_reads_mathlib (ord : EllipticQ → ℕ) : Value (curveFrame ord) ↔ BSDRank ord :=
  (fun h E => h E, fun h E => h E)

/-- THE BIT IS THE STATEMENT: the transfer from the record at the frame of the library's curves is exactly the rank
    statement. -/
theorem bit_is_the_statement (ord : EllipticQ → ℕ) : Transfer (curveFrame ord) ↔ BSDRank ord :=
  (normal_form (curveFrame ord)).trans (kernel_reads_mathlib ord)

/-- THE RECORD OF THE LIBRARY'S CURVES CARRIES THE VALUE, with no premise: on the record each curve's rank is read at
    its order. -/
theorem record_of_the_library (ord : EllipticQ → ℕ) : Value (recFrame (curveFrame ord)) :=
  recFrame_value (curveFrame ord)

/-- THE ONE REFUTER, on the library's objects: one elliptic curve over ℚ whose rank is not its order refutes the
    statement and the bit. -/
theorem the_one_refuter (ord : EllipticQ → ℕ) (E : EllipticQ) (h : Module.finrank ℤ E.W.toAffine.Point ≠ ord E) :
    ¬ BSDRank ord ∧ ¬ Transfer (curveFrame ord) :=
  (fun hs => h (hs E), fun ht => h ((bit_is_the_statement ord).mp ht E))

/-! ### 37a1: y² + y = x³ − x -/

/-- The curve 37a1. -/
def W37 : WeierstrassCurve ℚ := (0, 0, 1, -1, 0)

/-- ITS DISCRIMINANT IS 37. -/
theorem W37_Δ : W37.Δ = 37 := by
  simp only [WeierstrassCurve.Δ, WeierstrassCurve.b₂,
    WeierstrassCurve.b₄, WeierstrassCurve.b₆,
    WeierstrassCurve.b₈, W37]
  norm_num

instance W37_elliptic : W37.IsElliptic := (by rw [W37_Δ]; exact
    isUnit_iff_ne_zero.mpr (by norm_num))

/-- The point (0, 0) lies on 37a1, nonsingular. -/
theorem ns37_00 : W37.toAffine.Nonsingular 0 0 := by
  rw [nonsingular_iff', equation_iff']; norm_num [W37]

/-- The point (1, 0) lies on 37a1, nonsingular. -/
theorem ns37_10 : W37.toAffine.Nonsingular 1 0 := by
  rw [nonsingular_iff', equation_iff']; norm_num [W37]

/-- The point P = (0, 0) of 37a1. -/
noncomputable def P37 : W37.toAffine.Point := .some ns37_00

/-- The point (1, 0) of 37a1. -/
noncomputable def Q37 : W37.toAffine.Point := .some ns37_10

/-- THE LIBRARY'S GROUP LAW ON 37a1: P + P = (1, 0). -/
theorem two_P37 : P37 + P37 = Q37 := by
  have hy : (0 : ℚ) ≠ W37.toAffine.negY 0 0 := by norm_num [W37]
  rw [P37, Q37, Point.add_self_of_Y_ne hy]
  congr 1
  · simp only [addX, slope_of_Y_ne rfl hy, W37]; norm_num
  · simp only [addY, negAddY, addX, negY, slope_of_Y_ne rfl hy, W37];
    norm_num

/-- P IS NOT THE IDENTITY, and neither is its double. -/
theorem P37_ne_zero : P37 ≠ 0 ∧ P37 + P37 ≠ 0 :=
  (Point.some_ne_zero ns37_00, two_P37 ▸ Point.some_ne_zero ns37_10)

/-- 37a1 as an elliptic curve of the library. -/
def E37 : EllipticQ := (W37, W37_elliptic)

/-! ### 389a1: y² + y = x³ + x² − 2x -/

/-- The curve 389a1. -/
def W389 : WeierstrassCurve ℚ := (0, 1, 1, -2, 0)

/-- ITS DISCRIMINANT IS 389. -/
theorem W389_Δ : W389.Δ = 389 := by
  simp only [WeierstrassCurve.Δ, WeierstrassCurve.b₂,
    WeierstrassCurve.b₄, WeierstrassCurve.b₆,
    WeierstrassCurve.b₈, W389]
  norm_num

instance W389_elliptic : W389.IsElliptic := (by rw [W389_Δ]; exact
    isUnit_iff_ne_zero.mpr (by norm_num))

/-- The point (−1, 1) lies on 389a1, nonsingular. -/
theorem ns389_m11 : W389.toAffine.Nonsingular (-1) 1 := by
  rw [nonsingular_iff', equation_iff']; norm_num [W389]

/-- The point (0, 0) lies on 389a1, nonsingular. -/
theorem ns389_00 : W389.toAffine.Nonsingular 0 0 := by
  rw [nonsingular_iff', equation_iff']; norm_num [W389]

/-- The point (1, 0) lies on 389a1, nonsingular. -/
theorem ns389_10 : W389.toAffine.Nonsingular 1 0 := by
  rw [nonsingular_iff', equation_iff']; norm_num [W389]

/-- The points of 389a1. -/
noncomputable def P389 : W389.toAffine.Point := .some ns389_m11
noncomputable def Q389 : W389.toAffine.Point := .some ns389_00
noncomputable def R389 : W389.toAffine.Point := .some ns389_10

/-- THE LIBRARY'S GROUP LAW ON 389a1: (−1, 1) + (0, 0) = (1, 0). -/
theorem P389_add_Q389 : P389 + Q389 = R389 := by
  have hx : (-1 : ℚ) ≠ 0 := by norm_num
  rw [P389, Q389, R389, Point.add_of_X_ne hx]
  congr 1
  · simp only [addX, slope_of_X_ne hx, W389]; norm_num
  · simp only [addY, negAddY, addX, negY, slope_of_X_ne hx, W389];
    norm_num

/-- 389a1 as an elliptic curve of the library. -/
def E389 : EllipticQ := (W389, W389_elliptic)

/-- THE STATEMENT AT THE TWO CALM CURVES: with the orders one and two carried, the rank statement asks rank one of
    37a1 and rank two of 389a1, and each is the bit at its curve. -/
theorem the_statement_at_the_calm_curves (ord : EllipticQ → ℕ) (h37 : ord E37 = 1) (h389 : ord E389 = 2) :
    BSDRank ord → Module.finrank ℤ W37.toAffine.Point = 1 ∧
      Module.finrank ℤ W389.toAffine.Point = 2 :=
  fun h => ((h E37).trans h37, (h E389).trans h389)

/-- THE FORMS ARE BOUND: the value of the frame of the library's curves, the transfer from its record and the rank
    statement are one proposition; the record carries the value with no premise; and one curve whose rank is not its
    order refutes the statement. -/
theorem the_forms_are_bound (ord : EllipticQ → ℕ) :
    (Value (curveFrame ord) ↔ BSDRank ord) ∧ (Transfer (curveFrame ord) ↔ BSDRank ord) ∧
    Value (recFrame (curveFrame ord)) ∧
    (∀ E : EllipticQ, Module.finrank ℤ E.W.toAffine.Point ≠ ord E → ¬ BSDRank ord) :=
  (kernel_reads_mathlib ord, bit_is_the_statement ord,
    record_of_the_library ord,
   fun E h => (the_one_refuter ord E h).1)

end BSDMathlib
```

```
/-! ## cones of the binding, pinned: the build is silent, and any drift
    fails it -/
/-- info: 'BSDMathlib.recFrame_value' does not depend on any axioms -/
#guard_msgs in #print axioms BSDMathlib.recFrame_value
/-- info: 'BSDMathlib.normal_form' does not depend on any axioms -/
#guard_msgs in #print axioms BSDMathlib.normal_form
/-- info: 'BSDMathlib.kernel_reads_mathlib' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.kernel_reads_mathlib
/-- info: 'BSDMathlib.bit_is_the_statement' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.bit_is_the_statement
/-- info: 'BSDMathlib.record_of_the_library' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.record_of_the_library
/-- info: 'BSDMathlib.the_one_refuter' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.the_one_refuter
/-- info: 'BSDMathlib.W37_Δ' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.W37_Δ
/-- info: 'BSDMathlib.ns37_00' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.ns37_00
/-- info: 'BSDMathlib.ns37_10' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.ns37_10
/-- info: 'BSDMathlib.two_P37' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.two_P37
/-- info: 'BSDMathlib.P37_ne_zero' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.P37_ne_zero
/-- info: 'BSDMathlib.W389_Δ' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.W389_Δ
/-- info: 'BSDMathlib.ns389_m11' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.ns389_m11
/-- info: 'BSDMathlib.ns389_00' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.ns389_00
/-- info: 'BSDMathlib.ns389_10' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.ns389_10
/-- info: 'BSDMathlib.P389_add_Q389' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.P389_add_Q389
/-- info: 'BSDMathlib.the_statement_at_the_calm_curves' depends on
    axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.the_statement_at_the_calm_curves
/-- info: 'BSDMathlib.the_forms_are_bound' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BSDMathlib.the_forms_are_bound
```

Appendix R · The root on no axiom and no posit

The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, R.1, `TOE_Zero.lean`, is carried verbatim from the root paper (Islam 2026k). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, imports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, `SelfGrounding`, `SelfVerifying`, ΔE_0, RA_0 and ra_0, and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds itself when deeds occur and every deed instances it (`SelfGrounding`); the root kernel's own comments call a deed an act. The constructed domain has one existent, whose energy of actuation is one, so the Root Axiom there reads $\forall x,\ 0 < \Delta E_0(x)$ (RA_0). On that domain the Root Axiom is a theorem with no axiom and no hypothesis (`root_on_the_constructed_domain`). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (`the_floor_is_universal`). Every denial of the root is a deed that instances it (`denial_reenacts_root`); the root is held by its deed (`seated_undeniable`); it is self-verifying (`denial_instantiates`); and no level stands above it (`no_level_above`). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart, and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4, in one theorem on no axiom (`the_master_seal`); it states the three counts side by side and no map between them. The body of this paper reads the root on its row; this appendix is the root itself. Its SHA-256 is

0cfcd9f6b399ecbaef01e762a4d9f104
90288f1053827b404428062e55076238

```
/-
  TOE_Zero.lean · Theory of Theories of Everything (TOE of All TOEs): Existence
  Proves Existence Only by Motion
  The zero-axiom, zero-posit kernel. Core Lean 4: no import, no axiom declared, no
  sorry, no native_decide, and
  every theorem rests on no axiom at all, not even propositional extensionality,
  quotients or choice.
  I · The self-grounding root. Acts occur and every act instances the root. Then
  every denial is an act and
      re-enacts the root, the root is held by the act itself, and no external
      proof adds anything to it: the
      Omega Boundary, proved for every root that grounds itself, universally, by
      quantification.
  II · The constructed root. On the constructed domain the Root Axiom is a theorem.
  III · The Return and the line. On the integer quaternions i·j·k = −1 and each
  unit squares to −1; conjugation is
      an involution and its fixed set on the chart is exactly the scalar line.
  IV · The twelve gates, recovered three ways inside the integers of the
  quaternions: the twenty-four Hurwitz units
      counted twelve up to sign; the twelve even permutations with class equation
      1, 3, 4, 4; and the norm-two
      shell of twenty-four, split twelve and twelve.
  V · The regress terminated. Above a self-grounding root no level grounds it
  further.
  VI · The master seal, every result bound in one theorem.
-/
namespace TOE0

/-! ## I · The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the root. -/
structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial : G.Act) : R
:= G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by the act itself.
-/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R := G.instances
G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R) (Q : Prop) :
(Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) : SelfVerifying R :=
fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the root holds,
every act re-enacts it, and no
    external proof adds to it. -/
theorem the_floor_is_universal :
    ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q : Prop, (Q →
    R) ↔ R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II · The constructed root -/

/-- The Root Axiom on the constructed one-point domain: its one existent actuates,
at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
```

```
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed, and the deed
actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int) < 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable ra₀

/-! ## III · The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b * y.a + x.c *
  y.d - x.d * y.c,
   x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b * y.c - x.c *
   y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
def qone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

/-- THE RETURN: i·j·k = −1. -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to −1. -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj = qneg ∧ qk.mul
qk = qneg := by decide

/-- The chart: every quaternion with coordinates in −2 … 2. -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c => chart.map
  fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involutive_on_the_chart : quats.all (fun q => q.conj.conj == q) =
true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the chart. -/
theorem the_line_is_the_fixed_set :
    quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 && q.d == 0)) =
    true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

/-- Hurwitz quaternions in doubled coordinates: all even, or all odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 != 0 && b % 2
  != 0 && c % 2 != 0 && d % 2 != 0)
/-- Doubled coordinates −2 … 2 hold every unit (doubled norm four) and every
element of the norm-two shell (doubled
    norm eight), since a doubled coordinate of absolute value three or more
    already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c => chart.map
  fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d == 4 && hurwitz
  a b c d

/-- The first nonzero coordinate is positive: one representative of each pair ±u.
-/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p
  if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then 0 < c else 0 <
  d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧ (units.filter
upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d == 8 && hurwitz
  a b c d

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split twelve and
twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
    shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧ (shell.filter (fun p
    => !upToSign p)).length = 12 := by
  decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1, 2, 3].flatMap
  fun c =>
    [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length == 4 then
    some [a, b, c, d] else none

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i, j)).countP
    fun (i, j) => i < j && p.getD j 0 < p.getD i 0

/-- The twelve gates as the even permutations. -/
def gates : List (List Nat) := perms4.filter fun p => inversions p % 2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i => p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h => compose
(compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
  | [] => [n]
  | m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a class of size
s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
  | 0, _ => []
  | _ + 1, [] => []
  | fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
    gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g => (conjClass
    g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a further ground of
the root is idle; the root needs
    no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q : Prop, (Q → R) → R
:= fun _ _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem on the
constructed domain; the Return closes;
    the line is the fixed set; and the twelve gates are recovered three ways. -/
theorem the_master_seal :
    (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q : Prop, (Q →
    R) ↔ R)) ∧
    RA₀ ∧ (qi.mul qj).mul qk = qneg ∧
    quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 && q.d == 0)) =
    true ∧
    (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧ (shell.filter
    upToSign).length = 12 ∧
    collapse 12 (sortNat (gates.map fun g => (conjClass g).length)) = [1, 3, 4, 4]
    :=
  (the_floor_is_universal, root_on_the_constructed_domain, the_return,
  the_line_is_the_fixed_set,
   twenty_four_units_twelve_gates.2, the_class_equation.1,
   the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end TOE0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal
/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain
/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return
/-- info: 'TOE0.units_square_to_minus_one' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart
/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal
```

The root's axes are proved in the second kernel, R.2,

`Triaxial_Actuation.lean` (32 theorems, every one on no axiom; SHA-256 `566615a6b9610fc981e03eb42e07af9e03849f20704729880f2bfe6434f2ea0b`). For every actuation, the direction of its deed gives the fold, the registration and the seat; the registration is the only height-keeping map onto the seat; the seat is the only place both blind spots cancel; no reading of the record returns the orientation; and every self-grounding root is the root of an actuation over its own deeds, so the root carries the three axes and is not characterless (`actuation_is_triaxial`, `registration_is_forced`, `the_only_special_cut`, `tongue_freedom`, `every_root_actuates`, `the_root_carries_three_axes`). The root's grade is read in the third kernel, R.3, `Root_Grade_Ledger.lean` (18 theorems, every one on no axiom; SHA-256 `c18b68b29d2b6be5058d56d3a7f9e966bcde6ce099217e0f5945db4c829c42b8`). On the grade ledger of the Master Codex (Islam 2026m), carried with its weakest-link join law, the Codex's rule reads a warranted root, one with a proof of its statement, at theorem grade, and an unwarranted root, the Codex's own declared axiom, at premise grade; the root of R.1 is warranted by a theorem with no axiom and no hypothesis, so on that rule it stands at theorem grade, the grade is read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA–Tongue co-location inherits the root's grade exactly (`a_warranted_root_is_theorem_grade`, `the_ledger_reads_the_warrant`, `the_grade_is_not_elected`, `ra_is_theorem_grade`, `coloc_grade_is_the_roots_grade`, `the_ledger_seal`). Both kernels compile with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0, declare no axiom, import nothing, and pin every cone as *does not depend on any axioms*; both are carried from the master plan of the series, Seven Rows, One Root (Islam 2026l), each extended in this revision of the series by one additive section, R.2 by its section IX and R.3 by its section VII, carried alike by every paper of the series, and the paper that reads them on its row cites them for exactly what they state.

```
/-
  Triaxial_Actuation.lean · actuation is triaxial

  Core Lean 4: no import, no axiom declared, no sorry, no native_decide, and every
  theorem rests on no
  axiom at all, not even propositional extensionality, quotients or choice.

  An actuation is a self-grounding root whose acts carry a direction: each act has
  a side and a height,
  and reversing the deed reverses the side and keeps the height. From the
  direction alone the kernel
  constructs three axes and proves them forced:
  I    The fold: reversing the deed is an involution.
  II   The registration: it keeps the height and forgets the side; it is blind to
  the fold, closed with
       it under combination, and it is the only height-keeping map onto the seat.
  III  The seat: the fold's fixed set, where the fold and the registration both
  fix the act, and off
       which neither does.
  IV   The third axis: off the seat the fold's orbit has two acts with one record,
  so the record leaves
       exactly the orientation open; every reading blind to the fold reads both
       alike.
  V    The root through the axes: a reversed deed is still an act and instances
  the root; the axes are
       carried by every actuation, and they force no orientation.
  VI   The bridge atom: the integers, with negation as the fold, are an actuation.
  VII  The triaxial seal.
  VIII The Tongue: no reading of the record returns the orientation; the seat is
  the only special cut.
  IX   Every root actuates: a self-grounding root is the root of an actuation over
  its own acts, so the
       root carries the three axes; the registration is the fold's record and
       lands on the seat, where
       the orientation an act and its reversal share stays open; the even pair
       leaves two acts, the
       orientation locks one; and a degenerate direction, one side only, is the
       characterless case.
-/

namespace TriAct

/-! ## The root, self-grounding -/

/-- A self-grounding root: acts occur, and every act instances the root. -/
structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

/-! ## Actuation: a self-grounding root whose acts carry a direction -/

/-- The direction of a deed: a side with a reversal, whose one fixed side is the
zero side. -/
structure Direction where
  Side     : Type
  rev      : Side → Side
  zero     : Side
  rev_rev  : ∀ s, rev (rev s) = s
  rev_zero : rev zero = zero
  fix_zero : ∀ s, rev s = s → s = zero

/-- An actuation: a self-grounding root whose acts are pairs of a side and a
height. -/
structure Actuation (R : Prop) where
  dir       : Direction
  Height    : Type
  anAct     : dir.Side × Height
  instances : dir.Side × Height → R

variable {R : Prop}

/-- Every actuation is a self-grounding root. -/
def Actuation.toSelfGrounding (A : Actuation R) : SelfGrounding R :=
  (A.dir.Side × A.Height, A.anAct, A.instances)

/-- The fold: reversing the deed reverses the side and keeps the height. -/
def fold (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side × A.Height :=
(A.dir.rev p.1, p.2)

/-- The registration: it keeps the height and forgets the side. -/
def reg (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side × A.Height :=
(A.dir.zero, p.2)

/-- The seat: the acts the fold fixes. -/
def OnSeat (A : Actuation R) (p : A.dir.Side × A.Height) : Prop := fold A p = p

/-! ## I · the fold -/

theorem fold_involutive (A : Actuation R) (p : A.dir.Side × A.Height) : fold A
(fold A p) = p := by
  cases p with
  | mk s h => show (A.dir.rev (A.dir.rev s), h) = (s, h); rw [A.dir.rev_rev]

/-! ## II · the registration -/

/-- The registration forgets the side: an act and its reversal have one record. -/
theorem reg_forgets_side (A : Actuation R) (p : A.dir.Side × A.Height) : reg A
(fold A p) = reg A p := rfl

/-- The registration keeps the height. -/
theorem reg_keeps_height (A : Actuation R) (p : A.dir.Side × A.Height) : (reg A
p).2 = p.2 := rfl

/-- The even pair is closed under combination: folding a record changes nothing. -/
theorem fold_of_reg (A : Actuation R) (p : A.dir.Side × A.Height) : fold A (reg A
p) = reg A p := by
  show (A.dir.rev A.dir.zero, p.2) = (A.dir.zero, p.2); rw [A.dir.rev_zero]

/-- The registration is idempotent. -/
theorem reg_idempotent (A : Actuation R) (p : A.dir.Side × A.Height) : reg A (reg A
p) = reg A p := rfl

/-- THE REGISTRATION IS FORCED, NOT ELECTED: any map that keeps the height and
lands on the seat is
the registration. -/
theorem registration_is_forced (A : Actuation R) (g : A.dir.Side × A.Height →
A.dir.Side × A.Height)
    (keeps : ∀ p, (g p).2 = p.2) (lands : ∀ p, OnSeat A (g p)) : ∀ p, g p = reg A p
    := by
  intro p
  have hs : (g p).1 = A.dir.zero := by
    have hf := lands p
    have h1 : A.dir.rev (g p).1 = (g p).1 := congrArg Prod.fst hf
    exact A.dir.fix_zero _ h1
  have hh := keeps p
  cases hg : g p with
```

```
  | mk s h =>
    rw [hg] at hs hh
    show (s, h) = (A.dir.zero, p.2)
    rw [show s = A.dir.zero from hs, show h = p.2 from hh]

/-! ## III · the seat -/

/-- The seat is exactly the zero side. -/
theorem seat_iff_zero_side (A : Actuation R) (p : A.dir.Side × A.Height) : OnSeat A
p ↔ p.1 = A.dir.zero :=
  (fun h => A.dir.fix_zero _ (congrArg Prod.fst h),
   fun h => by
     cases p with
     | mk s t =>
       show (A.dir.rev s, t) = (s, t)
       have hs : s = A.dir.zero := h
       rw [hs, A.dir.rev_zero])

/-- ON THE SEAT BOTH BLIND SPOTS CANCEL: the fold and the registration both fix
the act. -/
theorem seat_cancels_both (A : Actuation R) (p : A.dir.Side × A.Height) (h :
OnSeat A p) :
    fold A p = p ∧ reg A p = p := by
  refine (h, ?_)
  have hz : p.1 = A.dir.zero := (seat_iff_zero_side A p).mp h
  cases p with
  | mk s t =>
    have hs : s = A.dir.zero := hz
    show (A.dir.zero, t) = (s, t)
    rw [hs]

/-- OFF THE SEAT BOTH STAND: neither the fold nor the registration fixes the act.
-/
theorem off_seat_both (A : Actuation R) (p : A.dir.Side × A.Height) (h : ¬ OnSeat A
p) :
    fold A p ≠ p ∧ reg A p ≠ p := by
  refine (h, fun hr => h ?_)
  have hz : p.1 = A.dir.zero := (congrArg Prod.fst hr).symm
  exact (seat_iff_zero_side A p).mpr hz

/-! ## IV · the third axis -/

/-- THE RECORD LEAVES EXACTLY THE ORIENTATION: off the seat, an act and its
reversal are two acts with
one record. -/
theorem record_leaves_the_orientation (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : ¬ OnSeat A p) :
    fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (h, rfl)

/-- Every reading blind to the fold reads an act and its reversal alike. -/
theorem even_readings_are_blind (A : Actuation R) {α : Type} (f : A.dir.Side ×
A.Height → α)
    (blind : ∀ p, f (fold A p) = f p) (p : A.dir.Side × A.Height) : f (fold A p) =
    f p :=
  blind p

/-- The registration is such a reading, and so is every reading of the record. -/
theorem readings_of_the_record_are_blind (A : Actuation R) {α : Type} (k :
A.dir.Side × A.Height → α)
    (p : A.dir.Side × A.Height) : k (reg A (fold A p)) = k (reg A p) := rfl

/-! ## V · the root through the axes -/

/-- A REVERSED DEED IS STILL AN ACT, AND INSTANCES THE ROOT: the denial spends the
deed. -/
theorem denial_instances_root (A : Actuation R) (p : A.dir.Side × A.Height) : R :=
  A.instances (fold A p)

/-- The root holds on every actuation, by its act. -/
theorem root_holds (A : Actuation R) : R := A.instances A.anAct

/-- THE AXES FORCE NO ORIENTATION: off the seat both orientations are acts, both
instance the root, and
the record reads them alike; the triaxial structure leaves the one orientation
open. -/
theorem axes_force_no_orientation (A : Actuation R) (p : A.dir.Side × A.Height) (h
: ¬ OnSeat A p) :
    R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (A.instances p, A.instances (fold A p), h, rfl)

/-! ## VI · the bridge atom: the integers are an actuation -/

theorem int_neg_neg : ∀ s : Int, -(-s) = s
  | Int.ofNat 0 => rfl
  | Int.ofNat (_ + 1) => rfl
  | Int.negSucc _ => rfl

theorem self_neg_zero : ∀ v : Int, -v = v → v = 0
  | Int.ofNat 0, _ => rfl
  | Int.ofNat (_ + 1), h => Int.noConfusion h
  | Int.negSucc _, h => Int.noConfusion h

/-- The integer direction: negation, with zero its one fixed side. -/
def intDir : Direction := ⟨Int, fun s => -s, 0, int_neg_neg, rfl, self_neg_zero⟩

/-- The bridge atom: points (d, t), the fold (d, t) ↦ (−d, t), the registration
(d, t) ↦ (0, t). -/
def atom : Actuation True := ⟨intDir, Int, ((0 : Int), (0 : Int)), fun _ =>
True.intro⟩

/-- THE BRIDGE ATOM, AS AN ACTUATION: the fold fixes (d, t) exactly when d = 0,
the registration
leaves it unchanged exactly then, and off the seat the record leaves the
orientation. -/
theorem the_bridge_atom (d t : Int) :
    (OnSeat atom (d, t) ↔ d = 0) ∧ (reg atom (d, t) = (d, t) ↔ d = 0) ∧
    (d ≠ 0 → fold atom (d, t) ≠ (d, t) ∧ reg atom (fold atom (d, t)) = reg atom
    (d, t)) :=
  ⟨seat_iff_zero_side atom (d, t),
   ⟨fun h => ⟨congrArg Prod.fst h).symm, fun h => by rw [h]; rfl⟩,
   fun hd => record_leaves_the_orientation atom (d, t) (fun hs => hd
   ((seat_iff_zero_side atom (d, t)).mp hs)⟩⟩

/-- The atom is not characterless: off the seat it has two acts with one record. -/
theorem the_atom_is_not_characterless :
    fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg atom (fold
    atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0 : Int)) :=
  ⟨fun h => Int.noConfusion (congrArg Prod.fst h), rfl⟩

/-! ## VII · the triaxial seal -/

/-- ACTUATION IS TRIAXIAL: every actuation carries a fold that is an involution, a
registration that
keeps the height, forgets the side and is the only such map onto the seat, and a
seat where both fix
the act and off which neither does; off the seat the record leaves exactly the
orientation; every act
and its reversal instance the root; and the axes force no orientation. -/
theorem actuation_is_triaxial (A : Actuation R) :
    (∀ p, fold A (fold A p) = p) ∧
    (∀ p, reg A (fold A p) = reg A p ∧ (reg A p).2 = p.2 ∧ fold A (reg A p) = reg A
    p) ∧
    (∀ g : A.dir.Side × A.Height → A.dir.Side × A.Height,
       (∀ p, (g p).2 = p.2) → (∀ p, OnSeat A (g p)) → ∀ p, g p = reg A p) ∧
    (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
    (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A p ≠ p) ∧
    (∀ p, ¬ OnSeat A p → R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p) :=
  ⟨fold_involutive A, fun p => ⟨reg_forgets_side A p, reg_keeps_height A p,
  fold_of_reg A p⟩,
   registration_is_forced A, seat_cancels_both A, off_seat_both A,
   axes_force_no_orientation A⟩

/-- THE TRIAXIAL SEAL: actuation is triaxial, universally; the bridge atom is an
actuation; and it is
not characterless. -/
theorem the_triaxial_seal :
    (∀ (R : Prop) (A : Actuation R), (∀ p, fold A (fold A p) = p) ∧
      (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
      (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A (fold A p) = reg A p)) ∧
    (∀ d t : Int, OnSeat atom (d, t) ↔ d = 0) ∧
    (fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg atom (fold
    atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0 : Int))) :=
  ⟨fun _ A => ⟨fold_involutive A, seat_cancels_both A,
     fun p h => record_leaves_the_orientation A p h⟩,
   fun d t => seat_iff_zero_side atom (d, t),
   the_atom_is_not_characterless⟩


/-! ## VIII · the Tongue: freedom and the only special cut -/

/-- TONGUE FREEDOM: no reading of the record returns the orientation. If a record
is blind to the fold
and an orientation is flipped by it, the orientation is not a function of the
record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β → Bool) (d : α →
Bool) (x : α)
    (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g (ρ y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

/-- THE ONLY SPECIAL CUT: the seat is exactly where both blind spots cancel;
nowhere else do the fold
and the registration both fix an act. -/
theorem the_only_special_cut (A : Actuation R) (p : A.dir.Side × A.Height) :
    (fold A p = p ∧ reg A p = p) ↔ OnSeat A p :=
  (fun h => h.1, fun h => seat_cancels_both A p h)

/-! ## IX · every root actuates: the root carries the three axes -/

/-- THE REGISTRATION IS THE FOLD'S RECORD: it reads an act and its reversal alike
and keeps the height, and two
acts have one record exactly when they have one height. -/
theorem registration_is_the_folds_record (A : Actuation R) (p q : A.dir.Side ×
A.Height) :
    reg A (fold A p) = reg A p ∧ (reg A p = reg A q ↔ p.2 = q.2) :=
  ⟨rfl, ⟨fun h => show (reg A p).2 = (reg A q).2 from congrArg Prod.snd h, fun h
  => by
    cases p with
    | mk s t =>
      cases q with
      | mk s' t' =>
        have ht : t = t' := h
        show (A.dir.zero, t) = (A.dir.zero, t')
        rw [ht]⟩⟩
```

```
/-- THE REGISTRATION LANDS ON THE SEAT: every record is a seat point. -/
theorem reg_lands_on_seat (A : Actuation R) (p : A.dir.Side × A.Height) : OnSeat A
(reg A p) :=
  fold_of_reg A p

/-- THE ORIENTATION IS OPEN AT THE SEAT: off the seat, an act and its reversal
land on one seat point, the record
of both; at that place the record has left exactly the orientation open. -/
theorem open_at_the_seat (A : Actuation R) (p : A.dir.Side × A.Height) (h : ¬
OnSeat A p) :
    OnSeat A (reg A p) ∧ reg A (fold A p) = reg A p ∧ fold A p ≠ p :=
  (reg_lands_on_seat A p, rfl, h)

/-- TWO AXES LEAVE TWO: off the seat, every reading blind to the fold, the record
among them, reads an act and
its reversal alike: two acts, one reading. -/
theorem even_pair_leaves_two (A : Actuation R) (p : A.dir.Side × A.Height) (h : ¬
OnSeat A p) {α : Type}
    (f : A.dir.Side × A.Height → α) (blind : ∀ q, f (fold A q) = f q) :
    fold A p ≠ p ∧ f (fold A p) = f p ∧ reg A (fold A p) = reg A p :=
  (h, blind p, rfl)

/-- THREE AXES LOCK ONE: the record together with the orientation, the side,
determines the act. -/
theorem orientation_locks_the_act (A : Actuation R) (p q : A.dir.Side × A.Height)
    (hr : reg A p = reg A q) (hs : p.1 = q.1) : p = q := by
  cases p with
  | mk s t =>
    cases q with
    | mk s' t' =>
      have ht : t = t' := congrArg Prod.snd hr
      have hs' : s = s' := hs
      rw [hs', ht]

/-- A degenerate direction has the zero side and no other. -/
def Direction.Degenerate (D : Direction) : Prop := ∀ s, s = D.zero

/-- THE CHARACTERLESS CASE: over a degenerate direction every act is on the seat,
and the fold and the
registration are both the identity: one scalar, no fold, no seat to lock. -/
theorem degenerate_is_characterless (A : Actuation R) (h : A.dir.Degenerate) (p :
A.dir.Side × A.Height) :
    OnSeat A p ∧ fold A p = p ∧ reg A p = p :=
  have hs : OnSeat A p := (seat_iff_zero_side A p).mpr (h p.1)
  (hs, seat_cancels_both A p hs)

/-- The integer direction is not degenerate. -/
theorem intDir_not_degenerate : ¬ intDir.Degenerate := fun h =>
  have h0 : (1 : Int) = 0 := h (1 : Int)
  absurd h0 (by decide)

/-- EVERY ROOT ACTUATES: a self-grounding root is the root of the actuation over
its own acts, with the integer
direction; the root's deeds sit at the zero side and instance the root as before.
-/
def SelfGrounding.toActuation (S : SelfGrounding R) : Actuation R :=
  (intDir, S.Act, ((0 : Int), S.anAct), fun p => S.instances p.2)

theorem every_root_actuates (S : SelfGrounding R) :
    (∀ a : S.Act, OnSeat S.toActuation ((0 : Int), a)) ∧
    (∀ a : S.Act, ¬ OnSeat S.toActuation ((1 : Int), a)) ∧
    ¬ S.toActuation.dir.Degenerate :=
  (fun _ => rfl,
   fun a h =>
     have h0 : (1 : Int) = 0 := (seat_iff_zero_side S.toActuation ((1 : Int),
     a)).mp h
     absurd h0 (by decide),
   intDir_not_degenerate)

/-- THE ROOT CARRIES THREE AXES: for every self-grounding root, its actuation
carries the fold, an involution; the
registration, keeping the height, forgetting the side, the only height-keeping map
onto the seat; and the seat,
where both fix the act; and it is not characterless: at every height an act off
the seat, its reversal a second
act with the same record, both instancing the root, the orientation between them
forced by nothing. -/
theorem the_root_carries_three_axes (S : SelfGrounding R) :
    (∀ p, fold S.toActuation (fold S.toActuation p) = p) ∧
    (∀ p, reg S.toActuation (fold S.toActuation p) = reg S.toActuation p ∧ (reg
    S.toActuation p).2 = p.2) ∧
    (∀ g : Int × S.Act → Int × S.Act, (∀ p, (g p).2 = p.2) → (∀ p, OnSeat
    S.toActuation (g p)) →
       ∀ p, g p = reg S.toActuation p) ∧
    (∀ p, OnSeat S.toActuation p → fold S.toActuation p = p ∧ reg S.toActuation p =
    p) ∧
    (∀ a : S.Act, fold S.toActuation ((1 : Int), a) ≠ ((1 : Int), a) ∧
       reg S.toActuation (fold S.toActuation ((1 : Int), a)) = reg S.toActuation
       ((1 : Int), a)) ∧
    (∀ p, ¬ OnSeat S.toActuation p →
       R ∧ R ∧ fold S.toActuation p ≠ p ∧ reg S.toActuation (fold S.toActuation p)
       = reg S.toActuation p) :=
  (fold_involutive S.toActuation,
   fun p => (reg_forgets_side S.toActuation p, reg_keeps_height S.toActuation p),
   registration_is_forced S.toActuation,
   seat_cancels_both S.toActuation,
   fun a => record_leaves_the_orientation S.toActuation ((1 : Int), a)
   ((every_root_actuates S).2.1 a),
   axes_force_no_orientation S.toActuation)

end TriAct

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'TriAct.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_involutive
/-- info: 'TriAct.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_forgets_side
/-- info: 'TriAct.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_keeps_height
/-- info: 'TriAct.fold_of_reg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_of_reg
/-- info: 'TriAct.reg_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_idempotent
/-- info: 'TriAct.registration_is_forced' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.registration_is_forced
/-- info: 'TriAct.seat_iff_zero_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.seat_iff_zero_side
/-- info: 'TriAct.seat_cancels_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.seat_cancels_both
/-- info: 'TriAct.off_seat_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.off_seat_both
/-- info: 'TriAct.record_leaves_the_orientation' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.record_leaves_the_orientation
/-- info: 'TriAct.even_readings_are_blind' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.even_readings_are_blind
/-- info: 'TriAct.readings_of_the_record_are_blind' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.readings_of_the_record_are_blind
/-- info: 'TriAct.denial_instances_root' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.denial_instances_root
/-- info: 'TriAct.root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.root_holds
/-- info: 'TriAct.axes_force_no_orientation' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.axes_force_no_orientation
/-- info: 'TriAct.int_neg_neg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.int_neg_neg
/-- info: 'TriAct.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.self_neg_zero
/-- info: 'TriAct.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_bridge_atom
/-- info: 'TriAct.the_atom_is_not_characterless' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_atom_is_not_characterless
/-- info: 'TriAct.actuation_is_triaxial' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.actuation_is_triaxial
/-- info: 'TriAct.the_triaxial_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_triaxial_seal
/-- info: 'TriAct.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.tongue_freedom
/-- info: 'TriAct.the_only_special_cut' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_only_special_cut
/-- info: 'TriAct.registration_is_the_folds_record' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.registration_is_the_folds_record
/-- info: 'TriAct.reg_lands_on_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_lands_on_seat
/-- info: 'TriAct.open_at_the_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.open_at_the_seat
/-- info: 'TriAct.even_pair_leaves_two' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.even_pair_leaves_two
/-- info: 'TriAct.orientation_locks_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.orientation_locks_the_act
/-- info: 'TriAct.degenerate_is_characterless' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.degenerate_is_characterless
/-- info: 'TriAct.intDir_not_degenerate' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.intDir_not_degenerate
/-- info: 'TriAct.every_root_actuates' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.every_root_actuates
/-- info: 'TriAct.the_root_carries_three_axes' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_root_carries_three_axes


/-
  Root_Grade_Ledger.lean · the root at theorem grade, and RA–Tongue co-location
  with it

  Core Lean 4: no import, no axiom declared, no sorry, no native_decide, and every
  theorem rests on no
  axiom at all, not even propositional extensionality, quotients or choice.

  The floor Codex (Master Codex v5.1.0, Codex.lean) grades every claim on one
  ledger and joins claims
  at the weakest link. It declares the root as `axiom RA` and pins `raClaim.grade =
  .premise`; its
  co-location claim, the root joined with the wall, therefore inherits `.premise`.
  This kernel carries
  the same ledger and the same join law with no axiom declared, warrants the root
  by a theorem with no
  axiom and no hypothesis, and proves that the co-location claim then stands at
  theorem grade.
  I    The grade ledger and the weakest-link join, as in the Codex.
  II   The root, a theorem: the constructed domain of TOE_Zero.lean, warranted on
  no axiom.
  III  The wall, a theorem: the Codex's K4 frame, its target not a function of its
  record, proved on
       no axiom through Tongue freedom.
  IV   The claims: the root at theorem grade, the wall at theorem grade, and their
  co-location.
  V    Why the Codex read premise: a join's grade is its weakest link, and
  co-location's grade is the
       root's grade exactly; with the root at theorem grade, co-location is a
       theorem.
  VI   The ledger seal.
```

```
  VII  The grade read from the warrant, as the Codex reads it: a root with a
  warrant, a proof of its
       statement, is theorem grade; a root posited without one, the Codex's `axiom
       RA`, is premise grade;
       the two claims of section IV carry exactly the grades this rule reads.
-/

namespace RGL

/-! ## I · the grade ledger, as in the Codex -/

inductive Grade : Type
  | premise | corroboration | operational | structural | engineering
  | conditional | theoremConditional | analytic | theorem
  deriving DecidableEq

def Grade.rank : Grade → Nat
  | .premise => 0 | .corroboration => 1 | .operational => 2 | .structural => 3
  | .engineering => 4 | .conditional => 5 | .theoremConditional => 6
  | .analytic => 7 | .theorem => 8

def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a else b

/-- A claim carries its statement, its grade and its warrant, a proof of the
statement. -/
structure Claim : Type where
  stmt    : Prop
  grade   : Grade
  warrant : stmt

/-- Two claims join at their weakest link. -/
def Claim.join (c₁ c₂ : Claim) : Claim where
  stmt    := c₁.stmt ∧ c₂.stmt
  grade   := Grade.weakest c₁.grade c₂.grade
  warrant := (c₁.warrant, c₂.warrant)

theorem join_grade (c₁ c₂ : Claim) : (Claim.join c₁ c₂).grade = Grade.weakest
c₁.grade c₂.grade := rfl

/-! ## II · the root, a theorem (the constructed domain of TOE_Zero.lean) -/

structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R := G.instances
G.anAct

def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int) < 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM: no axiom, no hypothesis. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable ra₀

/-! ## III · the wall, a theorem (the Codex's K4 frame) -/

/-- Tongue freedom: if a record is blind to the fold and an orientation is flipped
by it, the
orientation is not a function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β → Bool) (d : α →
Bool) (x : α)
    (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g (ρ y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

namespace K4

def val : Nat → Nat
  | 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437
  | 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
  | _ => 0

def tau : Nat → Nat := fun i =>
  if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i

def target : Nat → Bool := fun i => i % 2 == 0

def rho : Nat → Nat := fun i =>
  (val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i % 7)

/-- The record is blind to the fold at the first pair. -/
theorem register_even_at_zero : rho (tau 0) = rho 0 := by decide

/-- The target is flipped by the fold at the first pair. -/
theorem target_odd_at_zero : target (tau 0) = !target 0 := by decide

/-- THE WALL: no reading of the record returns the target. -/
theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
  tongue_freedom tau rho g target 0 register_even_at_zero target_odd_at_zero

end K4

/-! ## IV · the claims -/

/-- The root at theorem grade, warranted by a theorem with no axiom and no
hypothesis. -/
def raClaim₀ : Claim where
  stmt    := RA₀
  grade   := .theorem
  warrant := root_on_the_constructed_domain

/-- The wall at theorem grade, as in the Codex. -/
def wallClaim : Claim where
  stmt    := ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)
  grade   := .theorem
  warrant := K4.wall

/-- RA–Tongue co-location: the root joined with the wall, as in the Codex. -/
def colocClaim₀ : Claim := Claim.join raClaim₀ wallClaim

/-- The Codex's root, graded as the Codex grades it, for comparison: the same
statement form, at
premise grade. -/
def raClaimPremise : Claim where
  stmt    := RA₀
  grade   := .premise
  warrant := root_on_the_constructed_domain

/-! ## V · why the Codex read premise, and what co-location is now -/

/-- A join with a theorem-grade claim has exactly the other claim's grade. -/
theorem weakest_with_theorem (g : Grade) : Grade.weakest g .theorem = g := by
  cases g <;> rfl

/-- CO-LOCATION'S GRADE IS THE ROOT'S GRADE, EXACTLY: the wall is a theorem, so
the join reads the
root's grade and nothing else. -/
theorem coloc_grade_is_the_roots_grade (r : Claim) :
    (Claim.join r wallClaim).grade = r.grade := by
  show Grade.weakest r.grade Grade.theorem = r.grade
  exact weakest_with_theorem r.grade

/-- WHY THE CODEX READ PREMISE: with the root labelled premise, the co-location
claim is premise. -/
theorem premise_root_gives_premise_coloc : (Claim.join raClaimPremise
wallClaim).grade = .premise := rfl

/-- THE ROOT IS A THEOREM-GRADE CLAIM. -/
theorem ra_is_theorem_grade : raClaim₀.grade = .theorem := rfl

/-- RA–TONGUE CO-LOCATION IS A THEOREM-GRADE CLAIM. -/
theorem coloc_is_theorem_grade : colocClaim₀.grade = .theorem := rfl

/-- The co-location claim's statement holds: the root and the wall, both proved. -/
theorem coloc_holds : colocClaim₀.stmt := colocClaim₀.warrant

/-! ## VI · the ledger seal -/

/-- THE LEDGER SEAL: the root is a theorem on no axiom and no hypothesis; the wall
is a theorem; their
co-location holds and stands at theorem grade; and co-location's grade is the
root's grade exactly,
which is why the Codex, labelling the root premise, read premise. -/
theorem the_ledger_seal :
    RA₀ ∧ (∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
    colocClaim₀.stmt ∧ colocClaim₀.grade = .theorem ∧
    (∀ r : Claim, (Claim.join r wallClaim).grade = r.grade) ∧
    (Claim.join raClaimPremise wallClaim).grade = .premise :=
  ⟨root_on_the_constructed_domain, K4.wall, coloc_holds, coloc_is_theorem_grade,
   coloc_grade_is_the_roots_grade, premise_root_gives_premise_coloc⟩

/-! ## VII · the grade read from the warrant, as the Codex reads it -/

/-- The evidence a ledger entry carries for a statement: posited with no warrant,
or warranted by a proof. -/
inductive Evidence (P : Prop) : Type
  | posited
  | warranted (w : P)

/-- THE CODEX'S RULE FOR THE ROOT'S GRADE: evidence at theorem grade is a proof
with its axioms printed. A root
that comes with a warrant, a proof of its statement, is theorem grade; a root
posited with none, as the Codex
declares `axiom RA`, is premise grade. -/
def rootGrade : Evidence RA₀ → Grade
  | .warranted _ => .theorem
  | .posited     => .premise

/-- A WARRANTED ROOT IS THEOREM GRADE: the root of section II carries its warrant,
so the rule reads theorem. -/
theorem a_warranted_root_is_theorem_grade :
    rootGrade (.warranted root_on_the_constructed_domain) = .theorem := rfl

/-- AN UNWARRANTED ROOT IS PREMISE GRADE: with no warrant the rule reads premise,
the Codex's own reading of
its declared axiom. -/
theorem an_unwarranted_root_is_premise_grade : rootGrade .posited = .premise := rfl

/-- THE LEDGER READS THE WARRANT: the root claim of section IV carries the grade
the rule reads from its warrant,
and the comparison claim carries the grade the rule reads from none. -/
theorem the_ledger_reads_the_warrant :
```

```
    raClaim₀.grade = rootGrade (.warranted raClaim₀.warrant) ∧
    raClaimPremise.grade = rootGrade .posited :=
  (rfl, rfl)

/-- THE GRADE IS NOT ELECTED: the rule reads theorem from a warrant and premise
from none, never the other way;
whatever label an entry carries, the warranted root's grade under the rule is
theorem. -/
theorem the_grade_is_not_elected :
    (∀ w : RA₀, rootGrade (.warranted w) = .theorem) ∧ rootGrade .posited ≠
    .theorem ∧
    (∀ w : RA₀, rootGrade (.warranted w) ≠ rootGrade .posited) :=
  (fun _ => rfl, fun h => Grade.noConfusion h, fun _ h => Grade.noConfusion h)

end RGL

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'RGL.join_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.join_grade
/-- info: 'RGL.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.seated_undeniable
/-- info: 'RGL.root_on_the_constructed_domain' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.root_on_the_constructed_domain
/-- info: 'RGL.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.tongue_freedom
/-- info: 'RGL.K4.register_even_at_zero' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.register_even_at_zero
/-- info: 'RGL.K4.target_odd_at_zero' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.target_odd_at_zero
/-- info: 'RGL.K4.wall' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.wall
/-- info: 'RGL.weakest_with_theorem' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.weakest_with_theorem
/-- info: 'RGL.coloc_grade_is_the_roots_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_grade_is_the_roots_grade
/-- info: 'RGL.premise_root_gives_premise_coloc' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.premise_root_gives_premise_coloc
/-- info: 'RGL.ra_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.ra_is_theorem_grade
/-- info: 'RGL.coloc_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_is_theorem_grade
/-- info: 'RGL.coloc_holds' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_holds
/-- info: 'RGL.the_ledger_seal' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_seal
/-- info: 'RGL.a_warranted_root_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.a_warranted_root_is_theorem_grade
/-- info: 'RGL.an_unwarranted_root_is_premise_grade' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.an_unwarranted_root_is_premise_grade
/-- info: 'RGL.the_ledger_reads_the_warrant' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_reads_the_warrant
/-- info: 'RGL.the_grade_is_not_elected' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_grade_is_not_elected
```

Author's Provenance and Method Disclosure

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below.

Author and method. Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author's positions on instruction; it holds no authorship. The exhibit's script is the author's, carried from his earlier draft and re-run. One claim was held at the model's insistence: the earlier draft's credit of the 389a1 verification to Buhler, Gross and Zagier (1985) is not carried, since their paper treats the rank-three curve of conductor 5077.

The register. In the programme's vocabulary: every theorem of the master, two hundred fourteen, stands at [{ꞓ},T] on no axiom; the Mathlib binding at the grade of Lean's standard axioms; the triaxial reading at [{ꞓ},S]; the exhibit at corroboration grade; the value above order one at premise grade on the act, and in the leading coefficient on the full act, the two supplied premises; $\Delta M = 0$ on the cited arithmetic. The act is the row's least-erasure posit read as existence registered on the seat. The cited theorems are witnesses typed against the act as in the Poincaré closure; the higher-cycle axiom is retired as the alignment-defect and semiregular witness axioms were. The freedom cut, prime-as-freedom and the triaxial lock are applied on the row.

Audit. This edition has not yet been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapients/Trisduction.

Nature of Root Axiom

The root is triaxial, and that is the force of the cut. A characterless existence has no triaxiality: one scalar, no fold, no seat, nothing to lock. The root, to exist is to actuate, carries three axes, and they are not chosen: for every actuation, the direction of its deed gives the fold, the registration and the seat, and the registration is the only height-keeping map onto the seat. The deed is the direction: a denial spends the deed and instances the root. The registration is the fold's record: it keeps the height and forgets the side, and the two even axes, symmetry and record, are closed under combination and blind. The seat is the fixed set, where the two blind spots cancel, and the third axis is the one orientation the even pair cannot supply. Two axes never determine the point; three do. The seven rows are one cut because each is an involution with a seat and one missing orientation. The root makes the third axis native as a place: the seat, and the one orientation still open there; the axes themselves force no orientation. Which way that orientation points is the value, and it is the root read on that seat. The root alone still forces no value.

Key to the statement, in the words of its receipts. The three axes are the two even axes, the fold's symmetry and the record, and the orientation, the side of a point of the actuation, which the even pair cannot supply; the fold, the registration and the seat are what the direction of the deed gives, and the seat is where the side is read. The point two axes never determine is a point off the seat: there the even pair does not tell a point from its reversal, and the record with the side locks the point; on the seat both blind spots cancel and nothing is open there to determine. A point of the actuation is a side and a height, and every point instances the root: the points are the actuation's deeds, and the deed is the direction in the sense that its side is its direction, reversed by the fold. The characterless case, one scalar, is a degenerate direction: the fold is the identity and the seat is total, which is to say no fold and no seat to lock. The axes are not chosen in the sense that, given the direction, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat; the root's own actuation takes the integer direction, under which it is not characterless, while a degenerate direction over the same deeds would be (`every_root_actuates`, `degenerate_is_characterless`). The root read on that seat is the root read on the row at the seat point where a point and its reversal land with one record; that the orientation between them is the row's value bit is the paper's structural reading, a second statement beside its kernel's theorems, and no kernel states a map between the two. Each row carries the actuation's seat as the root of an actuation over its own deeds; a row's own seat, where its kernel states one, is a further statement of that kernel, named in the last row of the table, and on P versus NP it is vacant. A deed of the

root is an inhabitant of its type of acts, which the root kernel's own comments call an act; in the root's actuation those deeds sit at the zero side, on the seat, and the points off the seat are the sides the direction supplies; the act, unqualified, is existence read on the row, the field the closing theorem consumes.

Receipts, each cited for what its statement says; R.1, R.2 and R.3 are the three kernels of Appendix R, every theorem of each on no axiom; the three theorems of the Master Codex are cited, not carried.

Clause of the statement	Receipt
the root carries three axes, not chosen: every self-grounding root is the root of an actuation over its own deeds, and for every actuation, given the direction of its deed, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat	`every_root_actuates`, `the_root_carries_three_axes`, `actuation_is_triaxial`, `registration_is_forced` (Appendix R.2)
the registration is the only height-keeping map onto the seat	`registration_is_forced` (R.2)
the axes force no orientation	`axes_force_no_orientation` (R.2)
the deed is the direction: a deed of the actuation is a point, a side and a height, its side its direction, reversed by the fold; a reversed point is still a deed and instances the root; a denial spends the deed and instances the root	`fold`, `denial_instances_root` (R.2); `denial_reenacts_root`, `denial_instantiates` (R.1)
the registration is the fold's record: it keeps the height, forgets the side and reads a point and its reversal alike; the fold is an involution; the even pair is closed under combination	`registration_is_the_folds_record`, `reg_forgets_side`, `reg_keeps_height`, `fold_involutive`, `fold_of_reg` (R.2)
the two even axes are blind: no reading of the record returns the orientation; symmetry keyless, the line property keyed	`readings_of_the_record_are_blind`, `tongue_freedom` (R.2); `symmetry_is_keyless`, `line_property_is_keyed`, `exactly_one_stands` (Islam 2026m, Master Codex v5.1.0, commit 5e55fca, cited for their statements, not carried; that Codex declares its root as an axiom, this paper's kernels declare none)
the seat is the fixed set, where both blind spots cancel; off the seat both stand	`seat_iff_zero_side`, `seat_cancels_both`, `off_seat_both`, `the_only_special_cut` (R.2)
the registration lands on the seat; off the seat a point and its reversal land on one seat point with one record, and the orientation is what stays open there; every reading of the record reads both alike	`reg_lands_on_seat`, `open_at_the_seat`, `record_leaves_the_orientation`, `readings_of_the_record_are_blind` (R.2)
the characterless case, no fold and no seat to lock: over a degenerate direction, one side only, the fold is the identity and every point is on the seat; the root's actuation is not degenerate	`degenerate_is_characterless`, `intDir_not_degenerate`, `every_root_actuates` (R.2)
the bridge atom is an actuation, and not characterless	`the_bridge_atom`, `the_atom_is_not_characterless`, `the_triaxial_seal` (R.2)
the scalar ground in the algebra, a third reading beside the actuation's: the Return, the line the conjugation fixes and the twelve gates, bound with the root	`the_return`, `the_line_is_the_fixed_set`, `twenty_four_units_twelve_gates`, `the_class_equation`, `the_master_seal` (R.1)
the root at theorem grade on the Codex's ledger rule, the grade read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA--Tongue co-location inheriting it	`a_warranted_root_is_theorem_grade`, `the_ledger_reads_the_warrant`, `the_grade_is_not_elected`, `ra_is_theorem_grade`, `coloc_grade_is_the_roots_grade`, `the_ledger_seal` (R.3)
two axes never determine the point; three do: off the seat the even pair does not tell a point from its reversal, and the record with the side, which carries the orientation, locks the point; on the seat both blind spots cancel and nothing is open to determine	`even_pair_leaves_two`, `orientation_locks_the_act`, `seat_cancels_both` (R.2); in its own coordinates, for independent axes in $\mathbb{F}_2^3$, `two_axes_leave_two`, `three_axes_lock_one` (this paper's kernel), a second statement
the root alone forces no value; the root read on the row is the value	`undeniable_root_forces_no_value`; `root_on_row_is_the_value`, `act_is_the_value` (this paper's kernel)
the seven rows are one cut: each row's root is an actuation, so each carries an involution, the fold, with a seat and one missing orientation	`every_root_actuates`, `the_root_carries_three_axes` (R.2), stated for every self-grounding root and read at the constructed root $\mathtt{ra}_0$ of this paper's kernel, whose structure repeats R.2's `SelfGrounding` field for field; the row's own seat is the centre of the expansion: the reflection law of `Centred` negates every coefficient whose parity disagrees with the sign, so a minus sign silences the even coefficients and a plus sign the odd ones (`minus_silences_even`, `plus_silences_odd`), and an integer equal to its own negative is zero (`self_neg_zero`); a statement of this kernel beside the actuation's seat; the row ledger of Islam (2026f, Book III), cited

References

Birch, B. J. and H. P. F. Swinnerton-Dyer. 1965. Notes on elliptic curves. II. *Journal für die reine und angewandte Mathematik* 218: 79--108.

Breuil, C., B. Conrad, F. Diamond and R. Taylor. 2001. On the modularity of elliptic curves over $\mathbb{Q}$: wild 3-adic exercises. *Journal of the American Mathematical Society* 14: 843--939.

Buhler, J. P., B. H. Gross and D. B. Zagier. 1985. On the conjecture of Birch and Swinnerton-Dyer for an elliptic curve of rank 3. *Mathematics of Computation* 44: 473--481.

Bump, D., S. Friedberg and J. Hoffstein. 1990. Nonvanishing theorems for L-functions of modular forms and their derivatives. *Inventiones Mathematicae* 102: 543--618.

Coates, J. and A. Wiles. 1977. On the conjecture of Birch and Swinnerton-Dyer. *Inventiones Mathematicae* 39: 223--251.

Cremona, J. E. 1997. *Algorithms for Modular Elliptic Curves*, 2nd ed. Cambridge University Press.

de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, 625--635.

Gross, B. H. and D. B. Zagier. 1986. Heegner points and derivatives of L-series. *Inventiones Mathematicae* 84: 225--320.

Islam, M. F. 2026a. Navier--Stokes global regularity: unconditional proof of the form by theorem, the value by the act. Zenodo. doi:10.5281/zenodo.22670344.

Islam, M. F. 2026b. The Hodge conjecture: unconditional proof of the form by theorem, the value by the act. Zenodo. doi:10.5281/zenodo.22701510.

Islam, M. F. 2026c. The cut-agnostic division theorem. Zenodo. doi:10.5281/zenodo.22954862.

Islam, M. F. 2026d. The formal closure of computational separation: P $\neq$ NP closed on existence itself, read on

computation. Zenodo. doi:10.5281/zenodo.23133632.

Islam, M. F. 2026e. A formal proof of the Riemann Hypothesis by least erasure. Zenodo. doi:10.5281/zenodo.23034066.

Islam, M. F. 2026f. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.

Islam, M. F. 2026g. PhysOS^{T}: the Trisduction physical operating system. Zenodo. doi:10.5281/zenodo.23117440.

Islam, M. F. 2026h. Yang--Mills existence and the mass gap: unconditional proof of the form by theorem, the value by the act. Zenodo. doi:10.5281/zenodo.23162231.

Islam, M. F. 2026i. The Goldbach conjecture: unconditional proof of the form by theorem, the value by the act. Zenodo. doi:10.5281/zenodo.23162238.

Islam, M. F. 2026j. The Poincaré conjecture: unconditional proof of the form by theorem, the value by the act. Zenodo. doi:10.5281/zenodo.23162240.

Islam, M. F. 2026k. Theory of Theories of Everything (TOE of All TOEs): existence proves existence only by motion. Zenodo. doi:10.5281/zenodo.23168379.

Islam, M. F. 2026l. Seven rows, one root: the master plan of the series, revision F.5. Manuscript of record, 5 October 2026, carried with the series; its Appendix R kernels are carried into this paper's Appendix R, each extended by one additive section.

Islam, M. F. 2026m. Trisduction: the Master Codex, edition 5.1.0. Codex.lean and its kernels, commit 5e55fca98c2f, master SHA-256 441a485d7c3bd3ce. Repository 1000sapients/Trisduction, GitHub.

Islam, M. F. 2026n. An unconditional proof of the real part of the Riemann Hypothesis, the probable remainder closed to symmetry and computation at theorem grade, and the whole proved from one bit supplied by the act of computation or by physics itself. Edition 1.3.0, its master RH_Both_Sides_ Formal_Master v3.0.39_CC. Zenodo. doi:10.5281/zenodo.23440382.

Islam, M. F. 2026o. P $\neq$ NP: unconditional proof of the form by theorem, the value by the act. Edition 1.13.0, its master PNP_ Normal_Form_Paper_v1_13_0_CC. Zenodo. doi:10.5281/zenodo.23528398.

Islam, M. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTTG. Master reference, continuously updated at the same location.

Kolyvagin, V. A. 1990. Euler systems. In *The Grothendieck Festschrift*, vol. II, 435--483. Birkhäuser.

Mordell, L. J. 1922. On the rational solutions of the indeterminate equations of the third and fourth degrees. *Proceedings of the Cambridge Philosophical Society* 21: 179--192.

Murty, M. R. and V. K. Murty. 1991. Mean values of derivatives of modular L-series. *Annals of Mathematics* 133: 447--475.

Taylor, R. and A. Wiles. 1995. Ring-theoretic properties of certain Hecke algebras. *Annals of Mathematics* 141: 553--572.

Wiles, A. 1995. Modular elliptic curves and Fermat's Last Theorem. *Annals of Mathematics* 141: 443--551.

Wiles, A. 2006. The Birch and Swinnerton-Dyer conjecture. In *The Millennium Prize Problems*, 31--41. Clay Mathematics Institute.

End of manuscript